

NovaMart

Multi-Vendor Fresh Vegetable Marketplace

Project Documentation Report

Prepared for: Zidio Development — UI/UX & Full-Stack Design Domain

Project Codename: NovaMart

Document Version: 1.0

Date: 25-06-2026

Statement of Declaration

I hereby declare that this project documentation report, titled “NovaMart — Multi-Vendor Fresh Vegetable Marketplace,” is my own original work, carried out during the course of this development and documentation programme. All designs, diagrams, source code excerpts, and analysis presented in this report were prepared specifically for the NovaMart project unless otherwise cited.

Where information, tools, or concepts from external sources have been used, they have been acknowledged appropriately in the References section. This report has not been submitted, in whole or in part, for any other qualification or project.

I understand that any misrepresentation of this declaration may result in disqualification from the associated evaluation or programme.

Signed: Mr Kirthan

Name: NovaMart Project Team – Jilla Kirthan

Date: 25-06-2026

Abstract

Fresh vegetable retail in most urban markets is still dominated by fragmented, informal channels: physical mandis, unaffiliated pushcart vendors, and single-store apps that list only one seller's inventory. Customers face inconsistent pricing, no visibility into order status, and limited payment flexibility, while small farmers and local sellers struggle to reach digital-first customers without building their own storefronts.

NovaMart is a multi-vendor e-commerce marketplace purpose-built for fresh vegetables. It brings customers, independent sellers (farmers and local vendors), and platform administrators onto a single, coherent digital storefront. The platform provides a searchable catalog of 38 vegetables across five categories (Leafy Greens, Root Vegetables, Gourds & Squashes, Exotic & Others, and Herbs & Aromatics), a fully functional cart with live quantity and price recalculation, an editable delivery-address checkout flow, dual payment options (Cash on Delivery and Online Payment via UPI, Card, or Net Banking), and real-time order tracking from placement through delivery.

This report documents the complete design and development lifecycle of NovaMart: the problem analysis and justification for building a custom platform rather than adopting an existing solution, the functional and non-functional requirements, the system architecture and data model, the implementation approach with representative source code, the testing performed, and a full technical manual and user guide. A comprehensive set of 49 supporting diagrams covering system architecture, data modelling, process flows, state management, project planning, and analytical views is included to give a complete, multi-perspective picture of the system.

The result is a working, browser-based prototype that demonstrates the core customer journey (browse → cart → checkout → track) and the seller-facing order management view, built with a lightweight HTML/CSS/JavaScript front end that can be progressively connected to the backend services described in this report.

Acknowledgement

I would like to express my sincere gratitude to Zidio Development for the opportunity to design, build, and document the NovaMart platform as part of this project programme. The structured guidelines, submission checklist, and evaluation rubric provided a clear framework that shaped both the design process and this documentation.

I am also grateful to the open-source communities behind the tools used throughout this project including Mermaid.js for diagramming, Matplotlib for analytical charting, and the broader HTML/CSS/JavaScript ecosystem without which the prototyping and documentation work would have taken considerably longer.

Finally, I thank everyone who reviewed early iterations of the NovaMart prototype and provided the direct, practical feedback that shaped the cart, checkout, and navigation flows into their current, tested form.

Table of Contents	
Statement of Declaration	2
Abstract	3
Acknowledgement	4
Analysis	9
Introduction	9
Purpose	9
Background	9
Problem Statement.....	9
Description of Existing Solution	10
Advantages of the Existing Solution	10
Disadvantages.....	10
Other Possible Solution.....	10
Hiring a Web Designer and/or Developer to create a website or mobile application	10
Advantages.....	10
Disadvantages	10
Using other already made softwares or other websites	10
The Advantages.....	10
Disadvantages	10
Evaluation of Suggested Solution	11
NovaMart Innovated Website Use Case.....	11
NovaMart Innovative Website Project Objectives	11
Advantages	11
Disadvantages.....	11
Requirements.....	12
Functional Requirements	12
Non-Functional Requirements.....	12
Safety Requirements	12
Software Quality Attributes.....	12
Development Platforms.....	13
Hardware	13
Software	13
Humanware	13
Hardware, Software and Humanware Requirements for the System	13
Design	13
Approach (Methodology).....	14
Activity Plan	14

Systems Flow Chart and Description of Solution	14
System Architecture	14
Client–Server Architecture	14
Static Web Architecture	17
System Architecture for Customer Platform	18
System Architecture for Seller Platform	18
Context and Container Views	18
Data Model	21
Process Flows — Sequence Diagrams	24
State Management	26
Business Process Flowcharts	29
Cross-Functional Process and Customer Experience	32
Development Workflow	33
Project Planning.....	34
Requirements Traceability and Navigation Analysis.....	36
Analytics and Business Charts	37
Stakeholder Relationships and Root-Cause Analysis.....	41
Event Modelling and Strategic Views	43
Hierarchy, Network, and Additional Views.....	44
Consolidated Overview Diagrams.....	45
Database and Tables.....	48
Implementation.....	49
Programming Code	49
Front-End Code	49
1. Design Tokens (CSS Custom Properties).....	49
2. Product Data Model	49
3. Navigation Controller	49
4. Catalog Rendering and Search	50
5. Cart Module.....	50
6. Checkout Module — Address and Payment.....	51
7. Authentication and Verification Flow.....	52
Back-End Code.....	52
1. Server Entry Point and Middleware Stack	53
2. Database Models.....	53
3. Authentication Middleware	54
4. Authentication Routes.....	55
5. Catalog Routes	56
6. Cart and Order Routes.....	56
7. Payment Routes	57

Database and Tables	58
Method of Solution Related to the Problem	59
Accurate Method of Solution.....	59
Testing and Result.....	59
Unit Testing.....	59
Functionality Testing	60
Documentation	61
Technical Manual	61
Approach (Methodology)	61
Systems Flow Chart and Description of Solution.....	61
System Architecture	61
System Architecture for Customer Platform	62
System Architecture for Seller Platform.....	62
Database and Tables.....	63
Implementation	63
programming Code.....	63
Database and Tables	63
Method of Solution Related to the Problem	63
Accurate Method of Solution	63
Testing and Result	63
Unit Testing	63
Functionality Testing	63
User Guide	63
Development	64
System Evaluation	65
References	66
Appendix: Diagram Source Code	66

Analysis

Introduction

NovaMart is a multi-vendor online marketplace focused exclusively on fresh vegetables. Unlike general e-commerce platforms that treat groceries as one category among many, NovaMart is designed end-to-end around the specific needs of vegetable retail: same-day freshness, per-kilogram and per-bunch pricing, small local sellers rather than large warehousing operations, and a delivery promise measured in hours rather than days.

The platform connects three user roles Customers who browse and purchase, Sellers (local farmers and vendors) who list and fulfil produce, and Administrators who moderate the catalog and oversee platform health through a single web application. This report documents the analysis, design, implementation, testing, and evaluation of that system.

Purpose

The purpose of this project is to design and build a digital marketplace that:

- Gives customers a single, trustworthy place to browse, compare, and buy fresh vegetables from multiple local sellers.
- Gives small sellers and farmers a low-friction way to reach digital customers without building their own storefront or logistics network.
- Reduces the operational friction unclear pricing, no order tracking, limited payment options that pushes both customers and sellers back toward informal, offline channels.
- Produces a working prototype and complete technical documentation suitable for handoff to a development team or for direct evaluation.

Background

Vegetable retail sits at an awkward midpoint in the e-commerce landscape. Large general grocery platforms carry vegetables as a small part of a much larger catalog, which means the shopping experience search, filtering, unit handling (kg / bunch / piece) is not optimised for produce. At the other end, informal local vendors offer freshness and trust but no digital presence, no order history, and no way for a customer to plan ahead or pay online.

This gap is what NovaMart is built to close: a platform narrow enough in scope to get the produce-specific details right (pack sizes, per-unit pricing, category browsing by leafy greens / root vegetables / gourds / exotic vegetables / herbs), but broad enough in vendor participation to offer real selection and competitive pricing.

Problem Statement

Customers who want to buy fresh vegetables online today face three recurring problems: (1) fragmented supply no single app lists multiple local sellers' produce side by side (2) opaque logistics

once an order is placed, there is often no visibility into its status until it arrives; and (3) restrictive payment options many small-seller channels only support cash, while customers increasingly expect UPI, card, or net-banking options. Sellers, meanwhile, have no affordable way to list produce, manage incoming orders, or reach customers beyond their immediate physical location.

NovaMart addresses all three problems in a single platform: a unified multi-vendor catalog, live order tracking from placement to delivery, and a checkout flow that supports both Cash on Delivery and Online Payment.

Description of Existing Solution

Prior to NovaMart, the closest available options were: (a) general grocery delivery apps that include vegetables as one of many categories (b) single-vendor local vegetable delivery apps run by one shop or farm; and (c) informal channels phone/WhatsApp ordering from a known local vendor, or in-person purchase at a physical mandi.

Advantages of the Existing Solution

- General grocery apps offer broad product ranges and established payment/delivery infrastructure.
- Single-vendor apps offer a direct, trusted relationship between the customer and one known seller.
- Informal/offline channels offer maximum freshness (same-day harvest) and zero platform fees for the seller.

Disadvantages

- General grocery apps rarely optimise the browsing experience for produce-specific needs (unit types, category depth, freshness cues).
- Single-vendor apps limit customer choice to whatever one seller stocks, with no price or quality comparison.
- Informal channels offer no order history, no digital payment trail, no delivery tracking, and do not scale beyond the seller's personal network.

Other Possible Solution

Hiring a Web Designer and/or Developer to create a website or mobile application

Advantages

- Fully custom-built to exact specifications from day one.
- Direct control over design decisions and technology choices throughout development.

Disadvantages

- Significantly higher upfront cost compared with adapting an existing template or platform.
- Longer time-to-market, since every component catalog, cart, checkout, tracking must be built from zero.
- Ongoing dependency on the hired designer/developer for maintenance and future changes.

Using other already made software's or other websites

The Advantages

- Faster initial launch using pre-built e-commerce platforms (e.g. generic storefront builders).
- Lower upfront development cost and access to existing plugin ecosystems.

Disadvantages

- Generic storefront platforms are not designed around vegetable-specific needs (per-kg pricing, pack sizes, category structure, freshness messaging).
- Multi-vendor support, if available at all, is usually a paid add-on with rigid workflows that do not match NovaMart's Customer / Seller / Admin model.
- Limited ability to customise the checkout and payment flow to the exact Cash-on-Delivery-plus-Online-Payment model required here.

Evaluation of Suggested Solution

Weighing the three alternatives above, a custom-built, purpose-specific platform was chosen over both outsourcing the entire build and adapting a generic storefront product. This decision was driven by three factors: the need for a data model and UI genuinely built around vegetables (units, categories, freshness), the need for a three-role (Customer / Seller / Admin) architecture from the outset, and the need for full control over the checkout and payment experience described in this report.

NovaMart Innovated Website Use Case

The core use case is straightforward: a customer opens NovaMart, browses or searches the vegetable catalog (optionally filtered by category), adds items to a live-updating cart, proceeds to a checkout screen where they confirm or edit their delivery address and choose between Cash on Delivery or Online Payment, places the order, and tracks it through a delivery timeline (Placed → Packed → Out for Delivery → Delivered). In parallel, a Seller receives the order in their portal, confirms and packs it, and hands it to a delivery partner. An Administrator oversees the catalog and user accounts across the platform.

NovaMart Innovative Website Project Objectives

- Provide a unified, searchable, category-filtered catalog of fresh vegetables from multiple sellers.
- Provide a fully functional cart supporting add, remove, and live quantity adjustment with accurate running totals.
- Provide an editable delivery-address checkout flow with two payment paths: Cash on Delivery and Online Payment (UPI / Card / Net Banking).
- Provide real-time order tracking from placement through delivery, visible to the customer.
- Provide a seller-facing dashboard for order and inventory oversight.
- Ensure the interface is fast, mobile-first, and requires no prior training to use.

Advantages

- Single destination for multi-vendor vegetable shopping, improving customer choice and price transparency.

- Lower barrier to entry for small sellers who cannot afford to build independent storefronts.
- Transparent, trackable order lifecycle builds customer trust.
- Flexible payment options remove a common purchase blocker.

Disadvantages

- Platform requires ongoing moderation to maintain catalog quality and seller accountability.
- Multi-vendor logistics (multiple sellers fulfilling one region) are more complex to coordinate than single-vendor delivery.
- Dependence on third-party payment gateways and delivery partners introduces external points of failure.

Requirements

Functional Requirements

1. Customers shall be able to search and filter the vegetable catalog by category and keyword.
2. Customers shall be able to add, remove, and adjust the quantity of items in their cart, with totals recalculating live.
3. Customers shall be able to view and edit their delivery address during checkout.
4. Customers shall be able to choose between Cash on Delivery and Online Payment (UPI, Card, or Net Banking) at checkout.
5. Customers shall be able to track the status of a placed order through a defined delivery lifecycle.
6. Customers shall be able to sign in, sign up, verify their email, and reset a forgotten password.
7. Sellers shall be able to list products, manage stock, and view/update incoming orders.
8. Administrators shall be able to moderate product listings and manage user accounts.

Non-Functional Requirements

- Performance: catalog and cart interactions should respond within 200 ms under normal load.
- Usability: the interface should require no onboarding or training for a first-time customer.
- Responsiveness: the UI must adapt cleanly from mobile screens to desktop widths.
- Availability: the platform should target 99.5% uptime for customer-facing services.
- Maintainability: the codebase should separate catalog, cart, checkout, and order-tracking concerns to simplify future changes.

Safety Requirements

- All payment data must be transmitted over encrypted (HTTPS) connections and never stored in plaintext.
- User passwords must be stored as salted hashes, never in plaintext.
- Session tokens must expire after a defined period of inactivity.
- Delivery addresses and personal details must only be visible to the customer, the assigned seller, and the assigned delivery partner for that order.

Software Quality Attributes

- Reliability — consistent, correct order and payment processing with no silent failures.
- Scalability — architecture should support growth in sellers, products, and concurrent customers without redesign.
- Portability — the front end runs in any modern browser without platform-specific dependencies.
- Security — authentication, authorization, and data-handling practices follow industry-standard practices.
- Testability — core modules (cart, checkout, order state) are structured to support isolated unit testing.

Development Platforms

Hardware

- Developer workstation: quad-core CPU or better, 8 GB+ RAM, for local development and testing.
- Production application servers: cloud-hosted virtual machines or containers, horizontally scalable.
- Client devices: any modern smartphone, tablet, or desktop computer with a web browser.

Software

- Front end: HTML5, CSS3, JavaScript (ES6+).
- Backend (target architecture): Node.js with Express, REST API design.
- Database: relational database (e.g. PostgreSQL) for transactional data; Redis for caching.
- Diagramming and documentation tooling: Mermaid.js, Matplotlib, Word/DOCX tooling.

Human ware

- Product/UI designer — responsible for catalog, cart, and checkout UX.
- Front-end developer — responsible for the customer and seller-facing web application.
- Backend developer — responsible for API, database, and payment integration.
- QA/Tester — responsible for unit and functionality testing described in this report.
- Project documentation owner — responsible for this report and its diagram set.

Hardware, Software and Human ware Requirements for the System

Category	Requirement	Justification
Hardware	Cloud application servers (2+ vCPU, 4GB+ RAM each)	Serves concurrent customer and seller traffic
Hardware	Managed database instance with automated backups	Protects transactional order and payment data
Software	Node.js / Express API layer	Implements catalog, cart, order and payment services
Software	PostgreSQL relational database	Stores users, products, orders, payments with referential integrity

Category	Requirement	Justification
Software	Redis cache	Speeds up catalog reads and session lookups
Human ware	1 UI/UX designer, 2 developers, 1 QA	Minimum viable team to build and test the described scope

Design

Approach (Methodology)

NovaMart was designed and built using an iterative, prototype-first methodology: each customer-facing flow (catalog → cart → checkout → tracking) was built as a working interactive prototype first, then refined against direct feedback before the underlying architecture and data model were formalised into the diagrams presented in this chapter. This mirrors an Agile approach short iteration, continuous feedback, incremental delivery rather than a single up-front waterfall design phase.

The methodology proceeded through four broad phases: Research & Discovery (competitor review, persona and journey mapping), UI Design & Visual System (design tokens, high-fidelity screens), Prototyping & Interaction (working cart/checkout logic, accessibility pass), and Testing, Handoff & Documentation (usability testing, this report, and the diagram set).

Activity Plan

The activity plan below reflects the phased delivery schedule used for NovaMart, spanning requirement gathering through deployment. A Gantt-chart and timeline view of this same plan appear later in this chapter (Figures 27–29) alongside the Kanban board used to track day-to-day development tasks (Figure 26).

Phase	Key Activities	Duration
Research & Discovery	Competitor analysis, persona & journey mapping, information architecture	2 weeks
Design	Design system, wireframes, high-fidelity screens	2 weeks
Development	Catalog & cart, checkout & payment, seller portal	3 weeks
Testing	Unit testing, functionality testing, usability testing	1 week
Deployment	Staging, QA sign-off, production release	3 days

Systems Flow Chart and Description of Solution

This section presents the complete set of 49 diagrams describing NovaMart's architecture, data model, process flows, and planning views, grouped by purpose. Each diagram is numbered and captioned for cross-reference throughout the rest of this report. Figures 1–47 form the detailed diagram set; Figures 48–49, presented at the end of this section, are two consolidated master overview pictures that combine several of those perspectives into one picture each.

Diagrams 1–37 and 41, 44–49 are authored in the Mermaid diagramming language (mermaid.js.org) a text-based syntax that renders as version-controllable, reproducible source rather than a static drawing. Diagrams 38–40, 42, and 43 (Tree Map, Venn, Ishikawa/Fishbone, Wardley Map, and Cynefin Framework) do not have a native Mermaid syntax and are instead generated programmatically from structured data using Matplotlib, following the same NovaMart colour system for visual consistency.

System Architecture

The system follows a layered client–server architecture: a client layer (customer web app, mobile-responsive PWA, and seller portal), an application layer of independent services behind an API gateway, and a data layer combining a relational database with a cache. Figure 1 shows this overall structure.

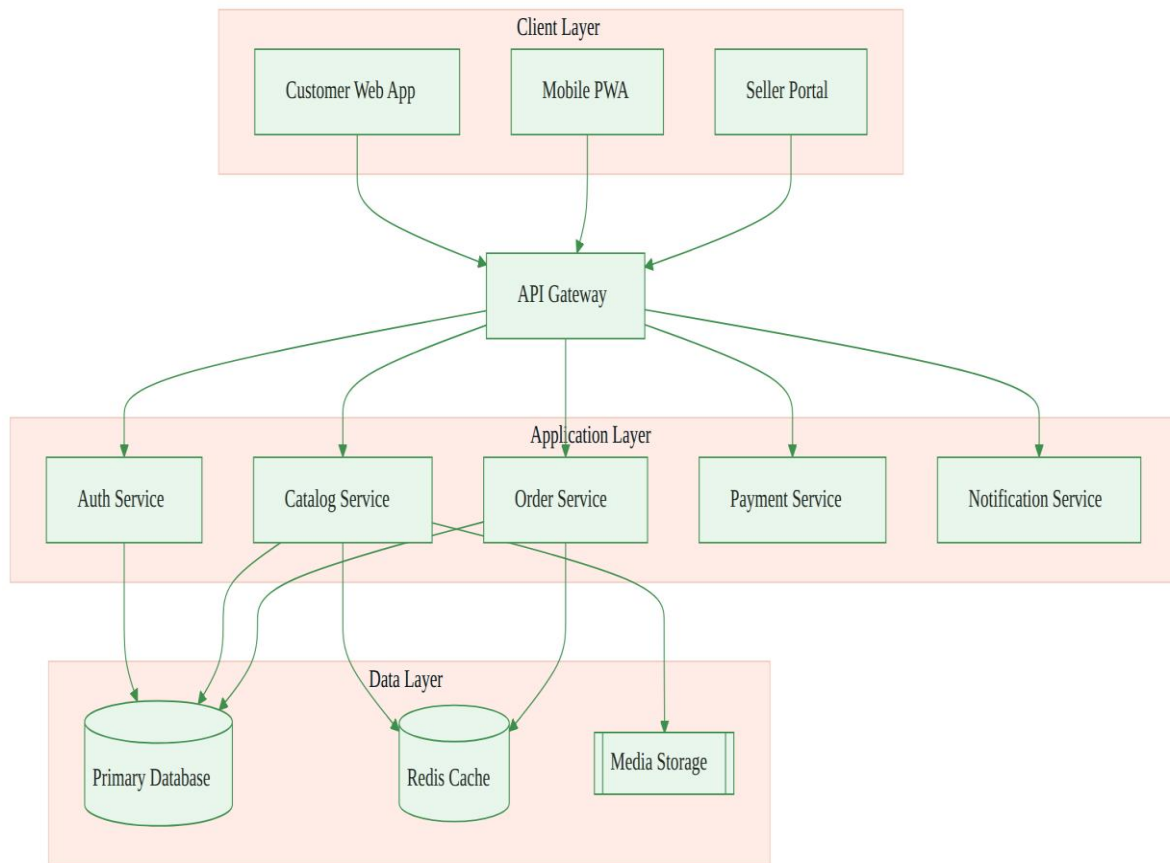


Figure 1: NovaMart System Architecture

Figure 2 focuses on the internal split between the front end and back end the front-end handles UI rendering and state, communicating over HTTPS/JSON with a backend API layer that enforces authentication before delegating to business logic and a data-access layer.

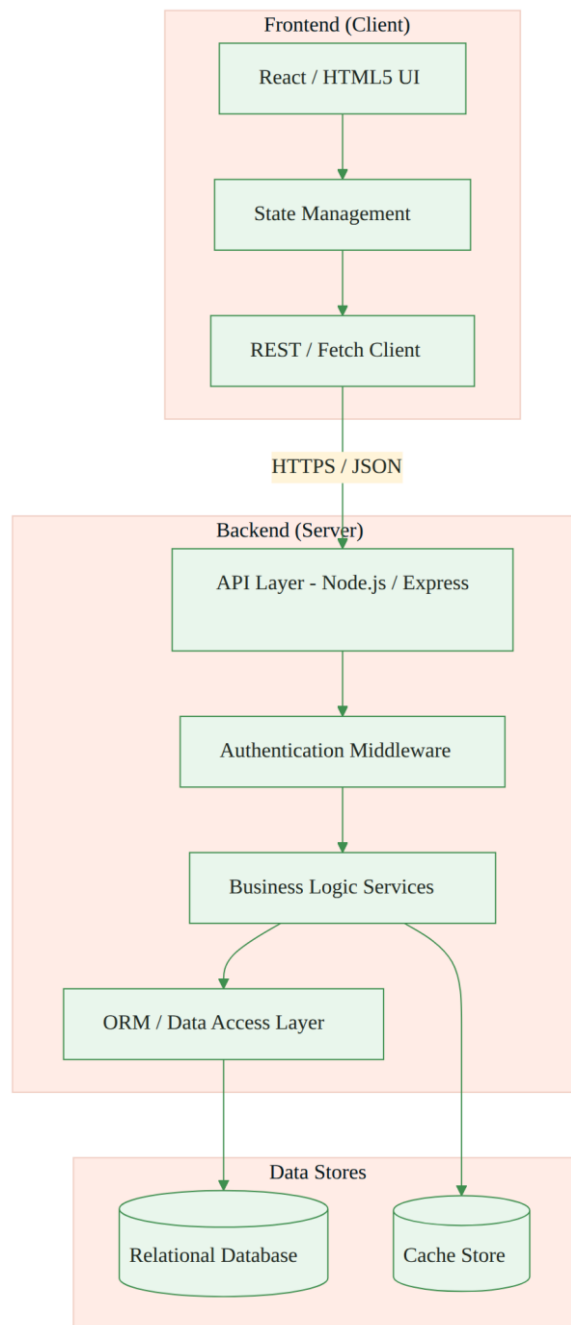


Figure 2: NovaMart Web Application Architecture (Frontend & Backend)

Figure 3 shows how the platform scales horizontally in production: a DNS/CDN edge routes traffic to a load balancer, which distributes requests across multiple application server instances sharing a common database and cache.

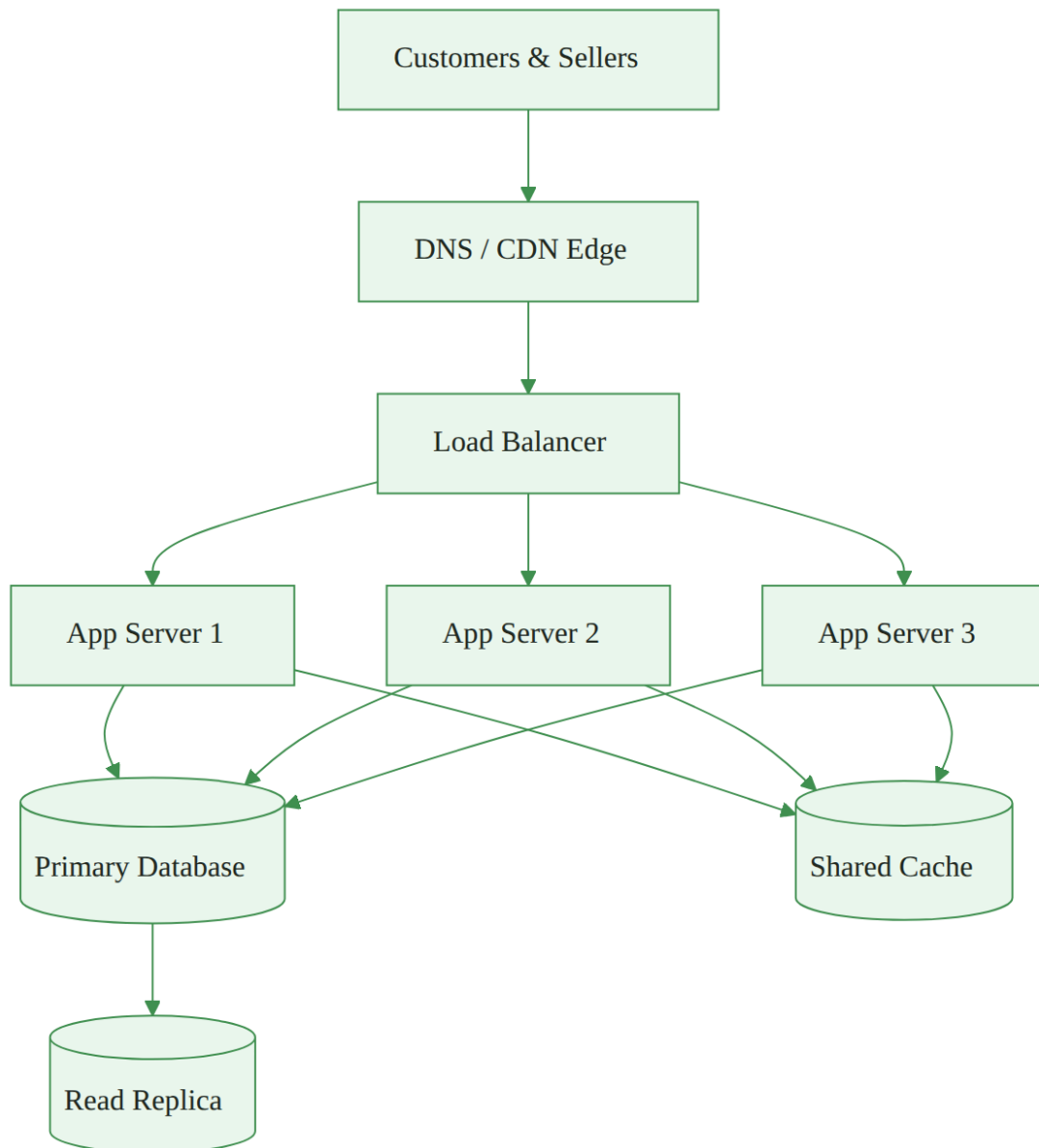


Figure 3: NovaMart Load Balancing Architecture

Client–Server Architecture

NovaMart follows a thin-client, thick-server model. The client (browser) is responsible only for rendering and local interaction state (cart contents, form inputs); all authoritative data product prices, stock levels, order status, and payment state lives on the server and is fetched or mutated through the API layer shown in Figures 1 and 2. This keeps the client lightweight and ensures a single source of truth for pricing and inventory, which is essential in a multi-vendor marketplace where several sellers' stock levels must stay consistent across every customer session.

Static Web Architecture

Marketing and informational pages (About, Seller Onboarding Info, Help/FAQ) are served as static content pre-rendered HTML/CSS with no server-side data dependency which can be cached at the CDN edge shown in Figure 3. This is distinct from the dynamic catalog, cart, and checkout views, which always require a live API call to reflect current stock and pricing.

System Architecture for Customer Platform

The customer-facing platform is the Client Layer's "Customer Web App" node in Figure 1, expanded in Figure 2's frontend layer. It is responsible for the catalog browse/search UI, the product detail view, the cart, the checkout flow (address + payment method), and order tracking all of which are documented in detail through the process and sequence diagrams later in this chapter (Figures 10, 11, 17, 18, 19).

System Architecture for Seller Platform

The seller-facing platform corresponds to the "Seller Portal" node in Figure 1. It provides product listing and stock management, an incoming-orders queue, and order status updates (Confirmed → Packed → Handed to Delivery), which feed into the same Order Service used by the customer platform. The seller's role in the end-to-end order lifecycle is documented in the Order Processing Sequence (Figure 13) and the Swimlane diagram (Figure 22).

Context and Container Views

Figure 4 places NovaMart within its wider ecosystem customers, sellers, administrators, and the external payment gateway, SMS/email provider, and delivery partner network it depends on.

NovaMart System Context Diagram

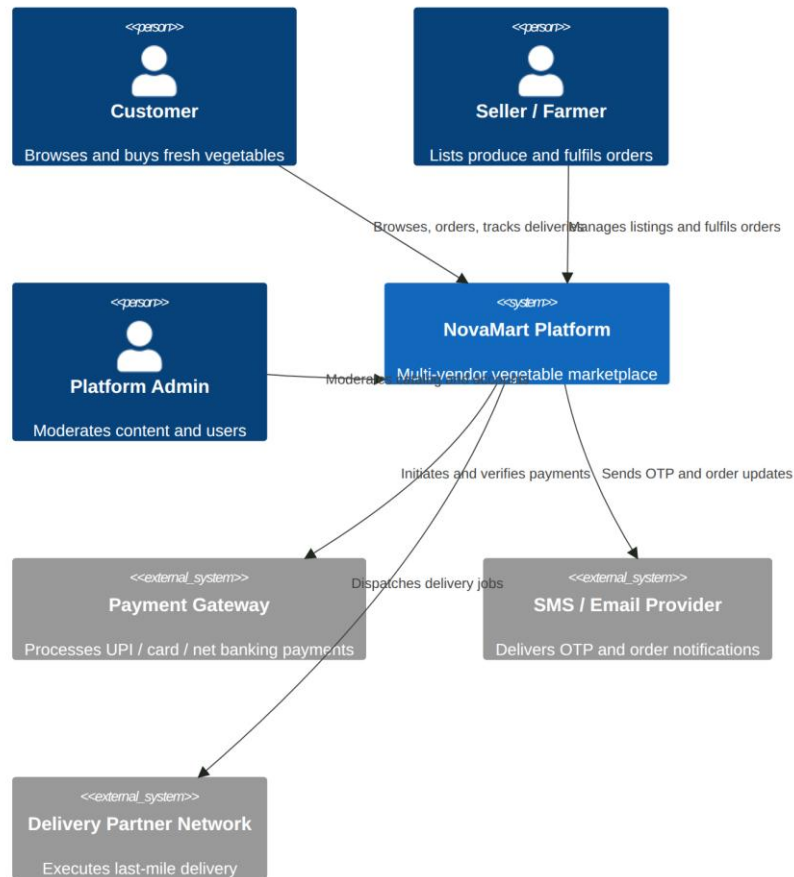


Figure 4: NovaMart System Context Diagram

Figure 5 zooms one level in, showing the containers (deployable units) that make up the platform itself and how they communicate.

NovaMart Container Architecture

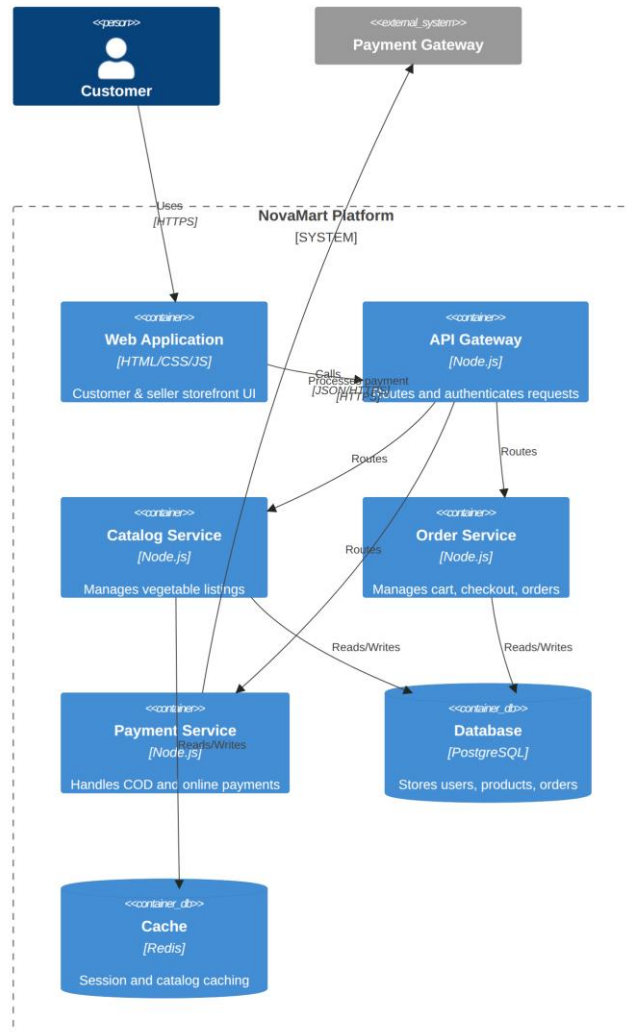


Figure 5: NovaMart Container Architecture

Figure 6 gives a simplified block-level view of the same system, useful as a quick-reference architecture summary.

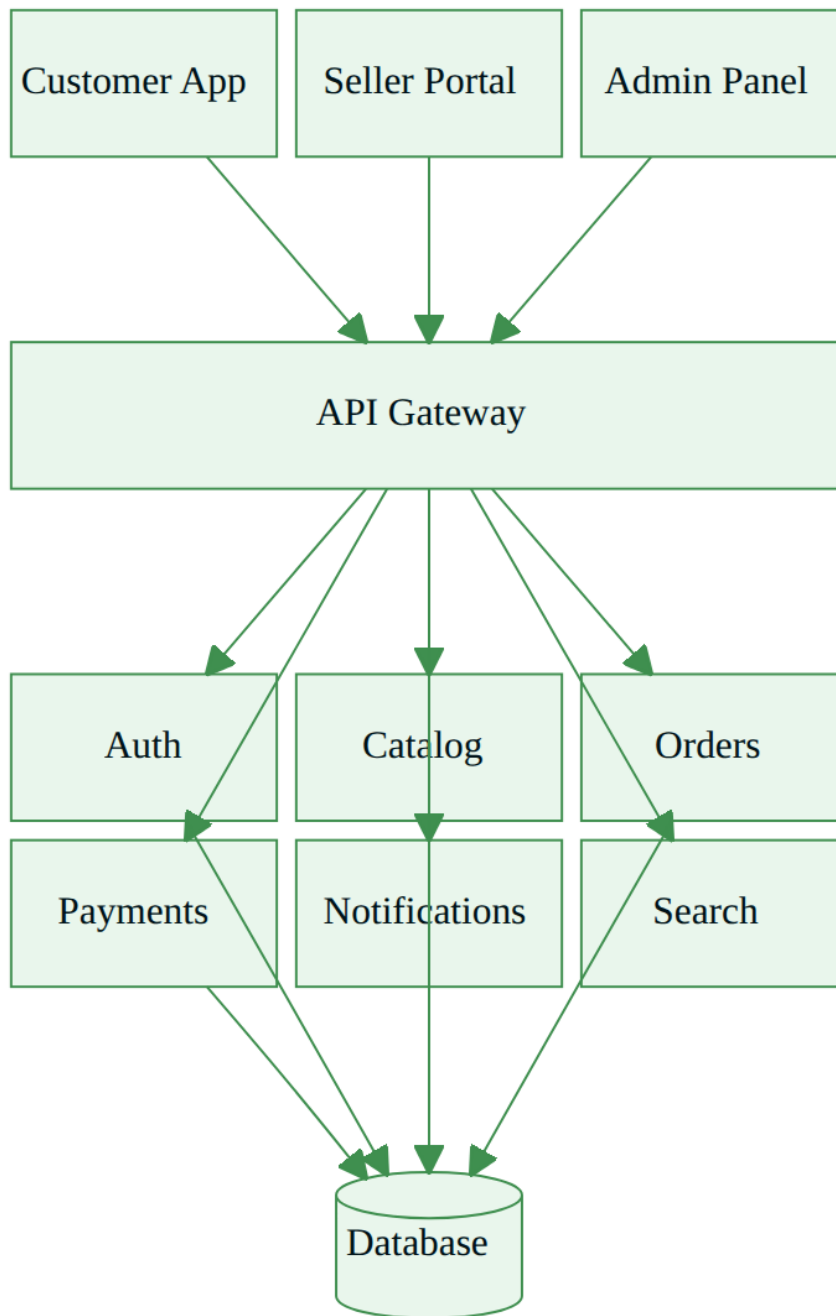


Figure 6: NovaMart System Block Diagram

Data Model

Figure 7 defines the relational schema underpinning NovaMart: users, sellers, products, orders, order items, payments, and addresses, with the key relationships between them.

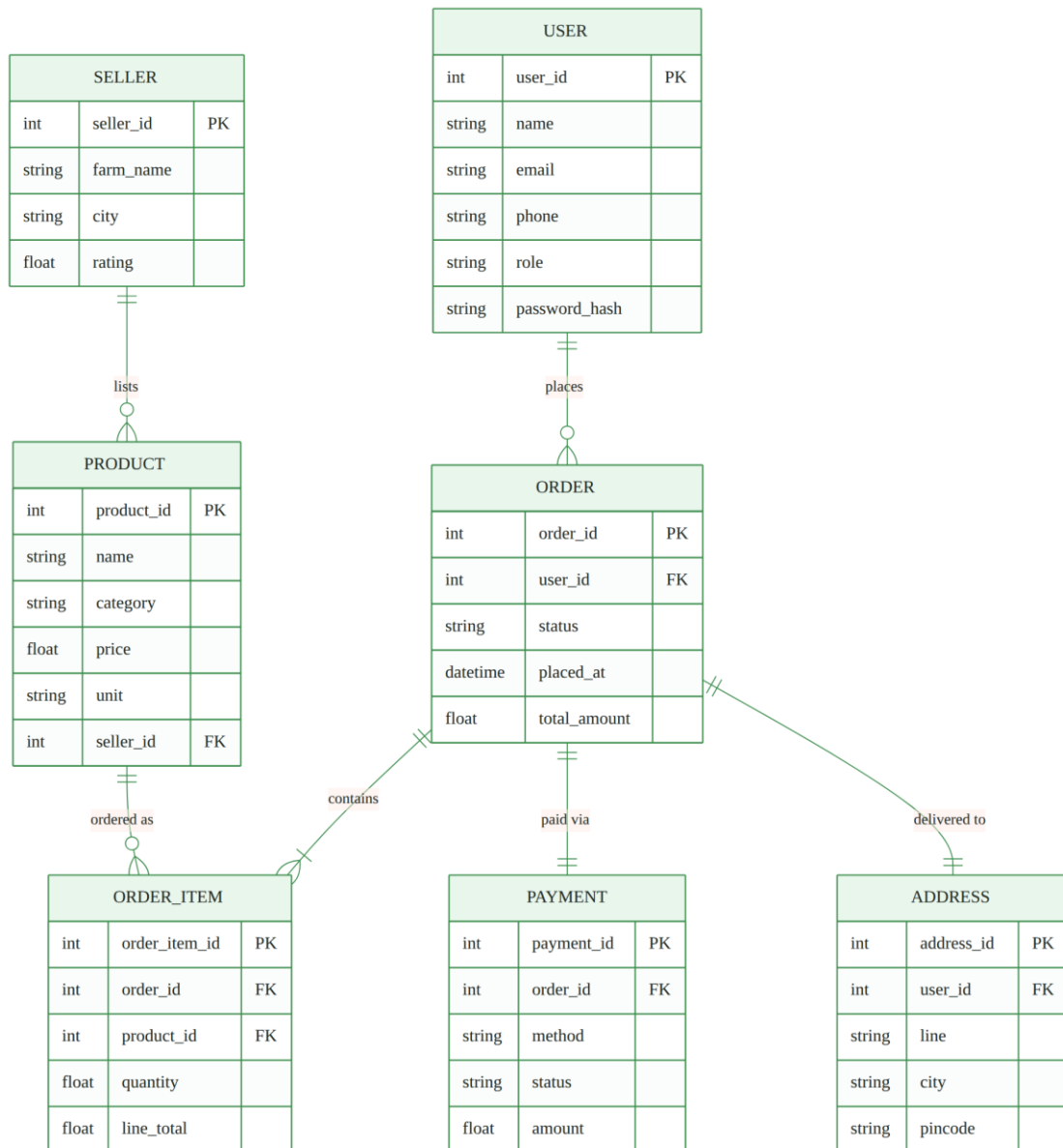


Figure 7: NovaMart Database Entity Relationship Diagram

Figures 8 and 9 present the same domain from an object-oriented perspective, useful for the application-layer implementation: user/customer/seller inheritance and cart composition (Figure 8), and the order/payment/delivery/review relationships (Figure 9).

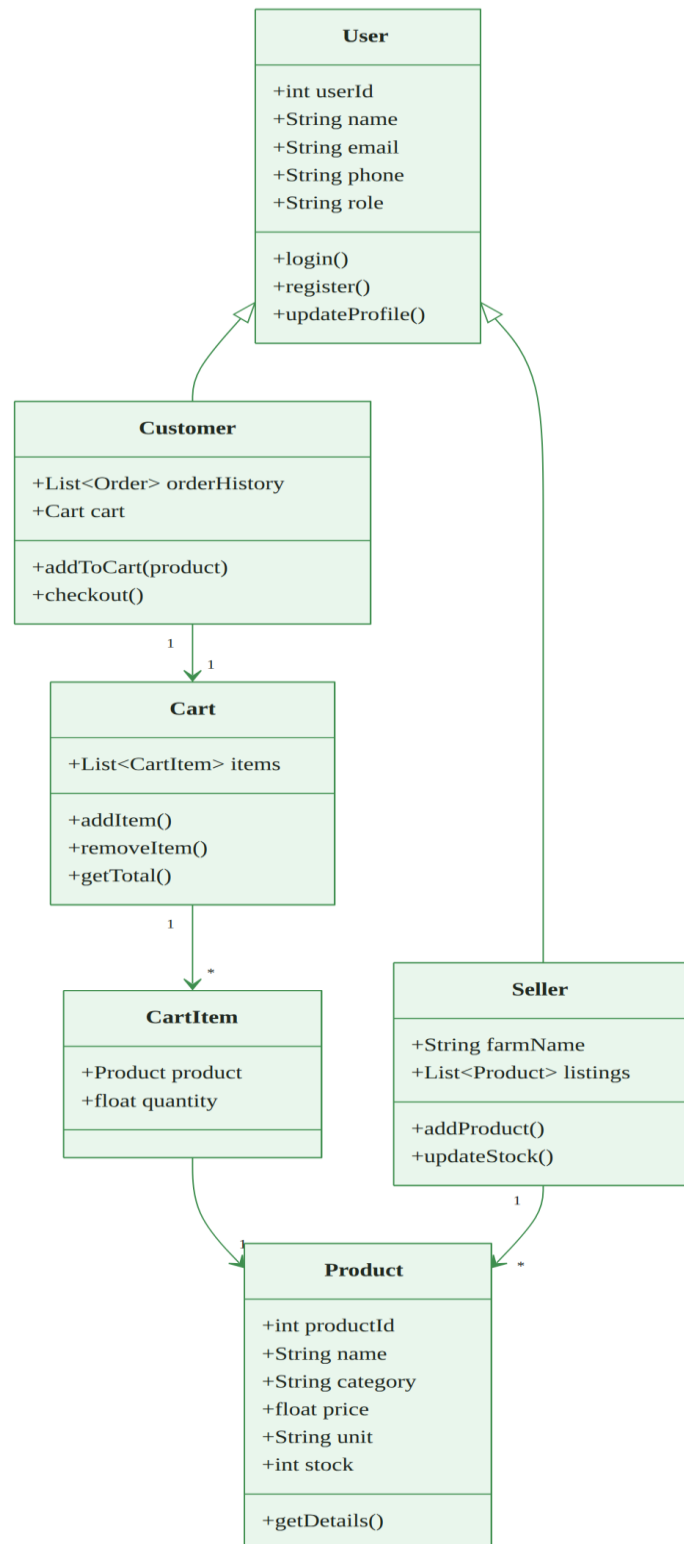


Figure 8: NovaMart User and Product Class Structure

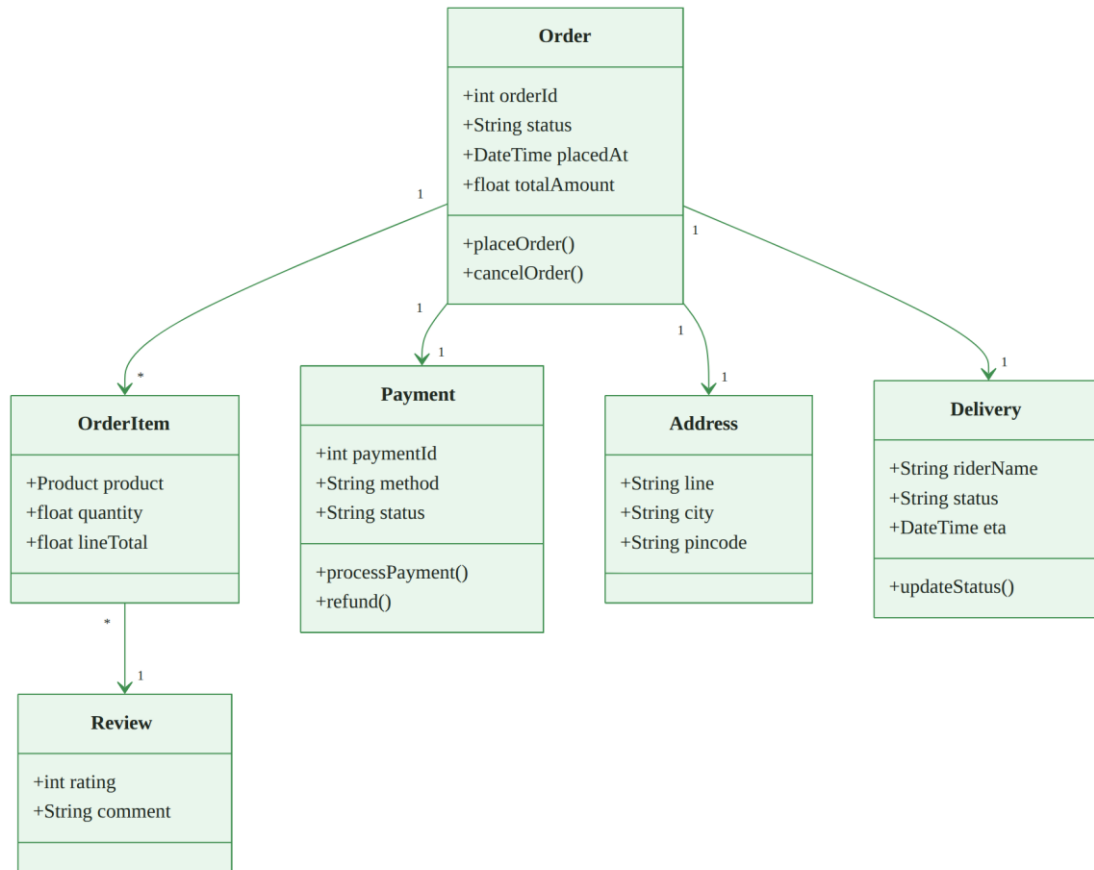


Figure 9: NovaMart E-Commerce Domain Model

Process Flows — Sequence Diagrams

The following four sequence diagrams trace the exact message flow for NovaMart's most critical interactions: signing in (Figure 10), browsing through to placing an order (Figure 11), completing an online payment (Figure 12), and the seller/delivery handling of a confirmed order (Figure 13).

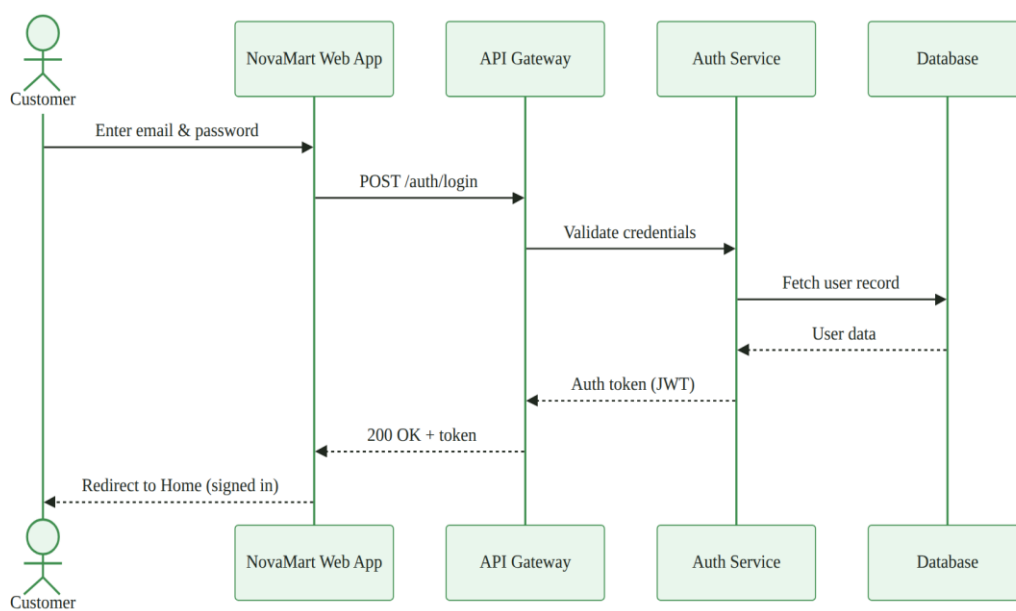


Figure 10: NovaMart User Login Sequence

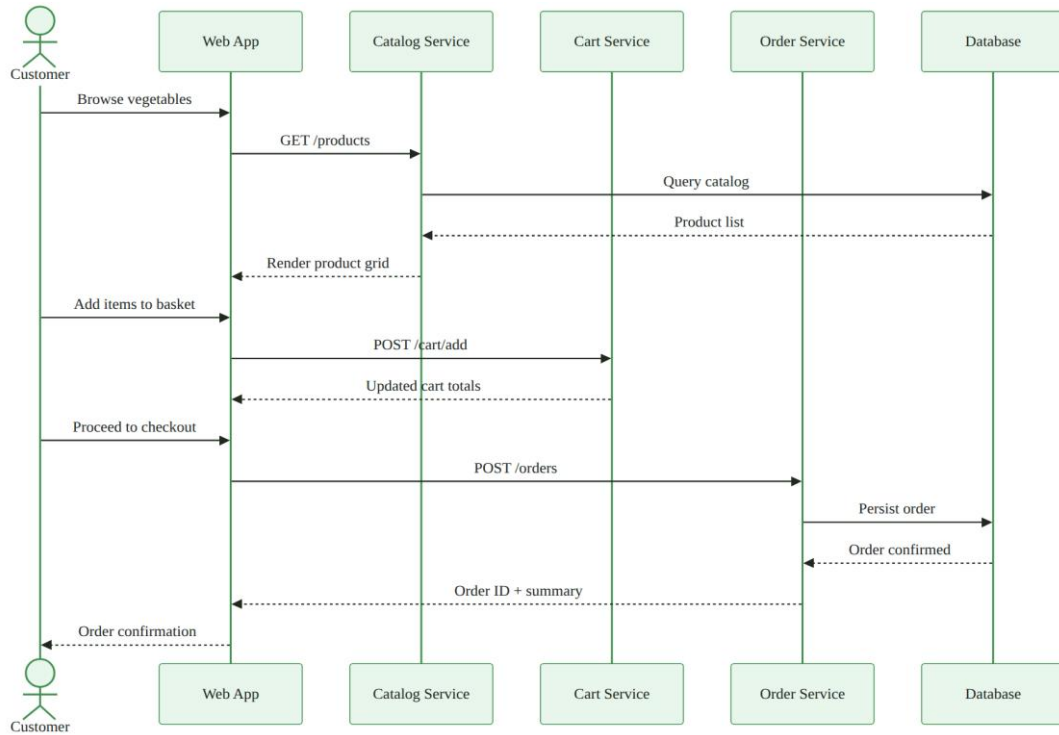


Figure 11: NovaMart Customer Purchase Sequence

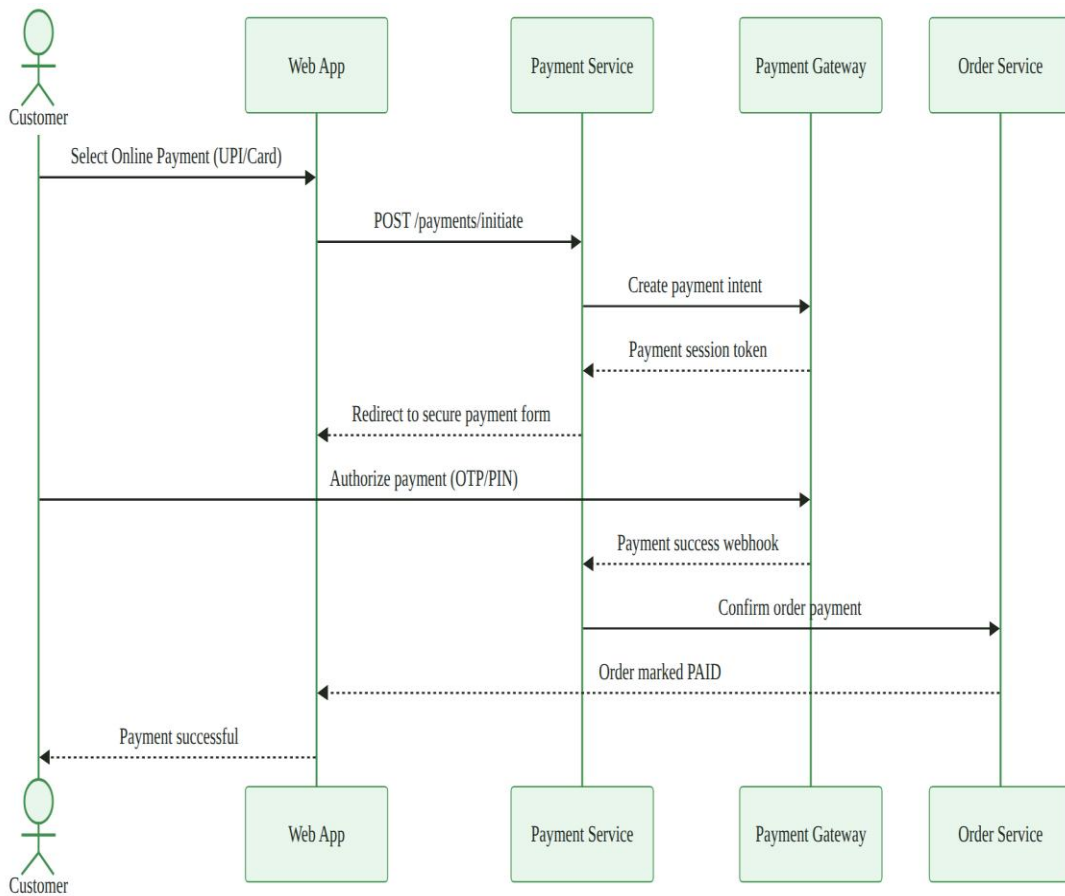


Figure 12: NovaMart Online Payment Sequence

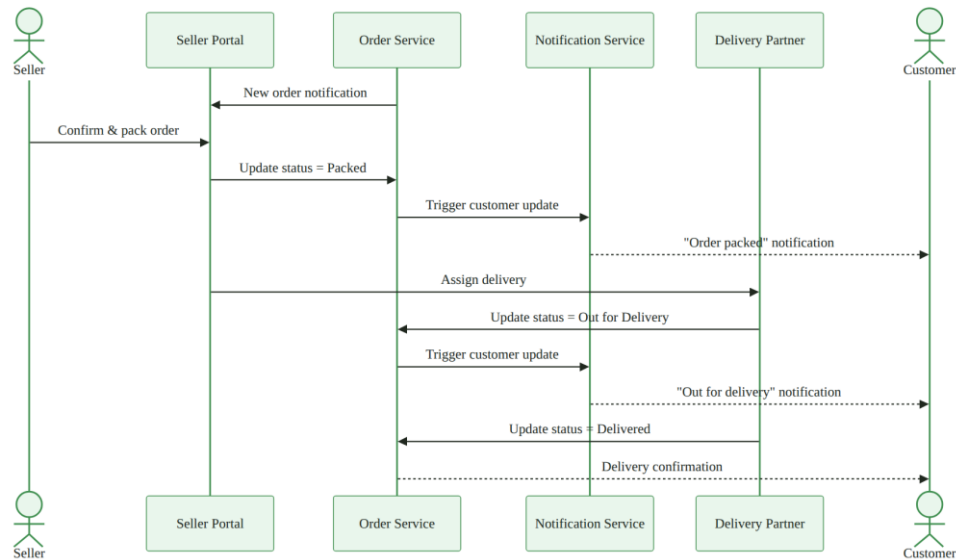


Figure 13: NovaMart Order Processing Sequence

State Management

Figure 14 defines the order status states surfaced to the customer in the tracking view (Placed → Confirmed → Packed → Out for Delivery → Delivered, with a Cancelled branch). Figure 15 extends this into the full internal order lifecycle, including payment failure and refund paths. Figure 16 shows how the Payment and Order state machines interact an order only advances to “Processing” once its linked payment reaches “Success.”

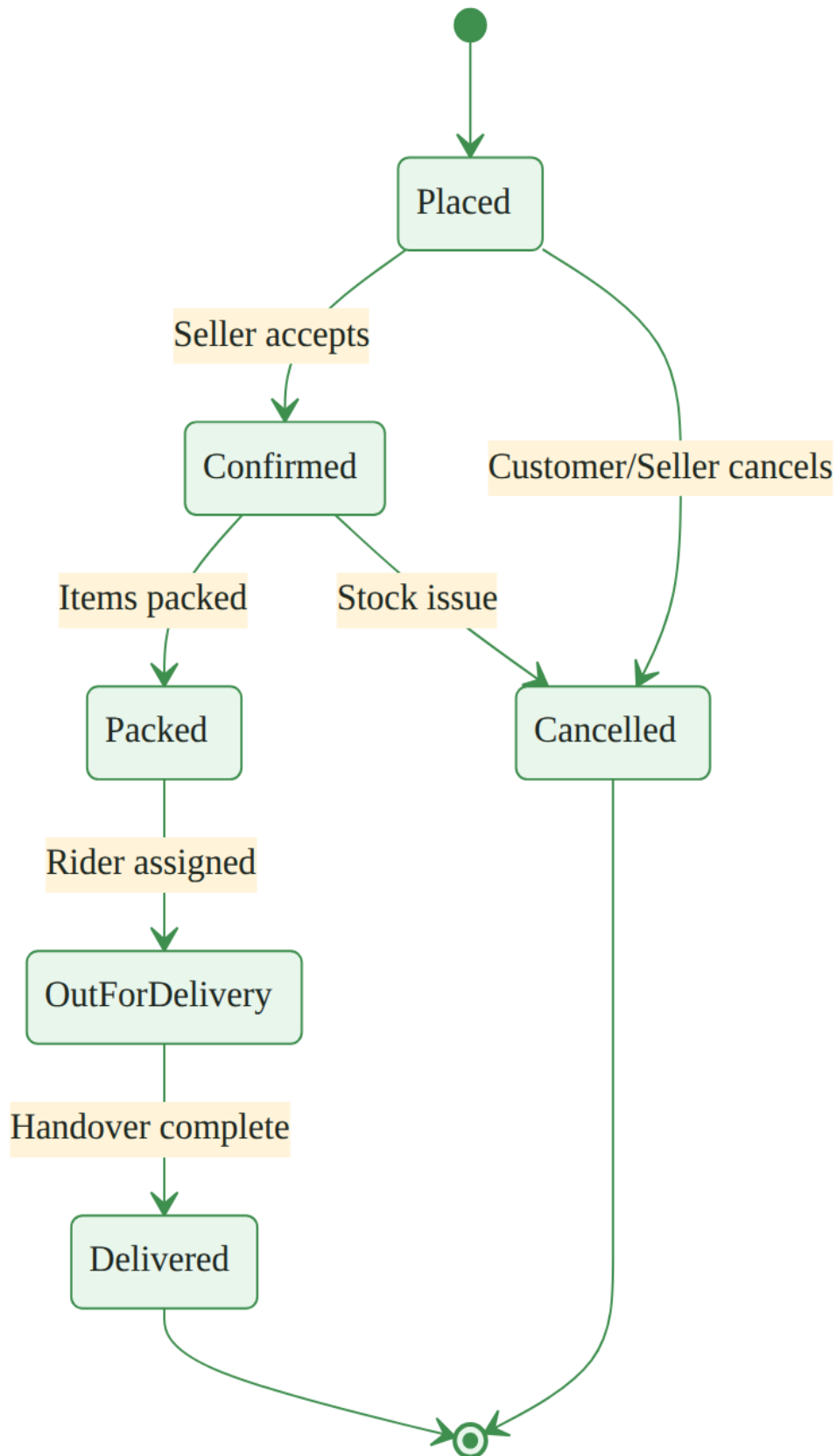


Figure 14: NovaMart Order State Diagram

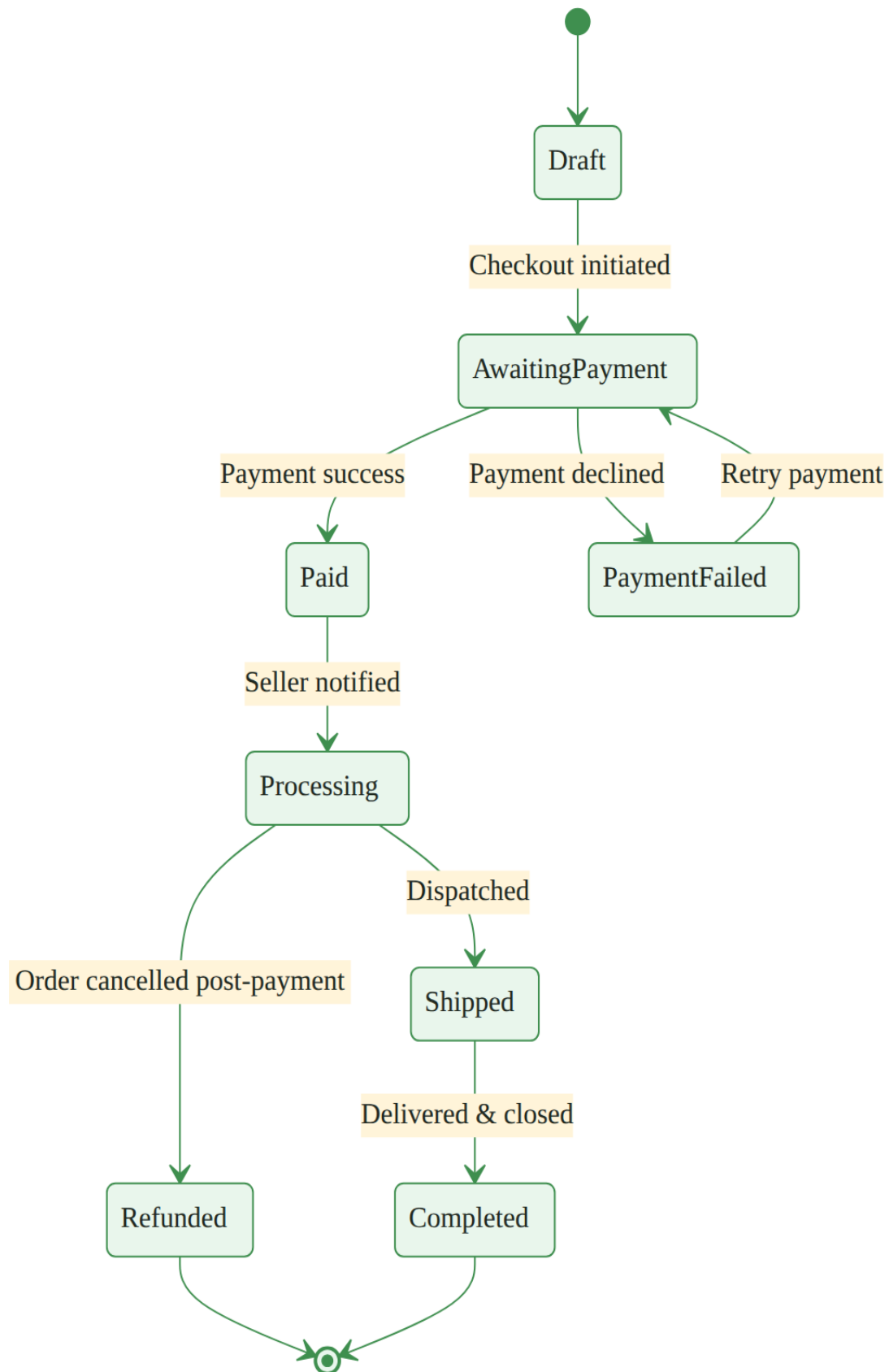


Figure 15: NovaMart Order Lifecycle Management

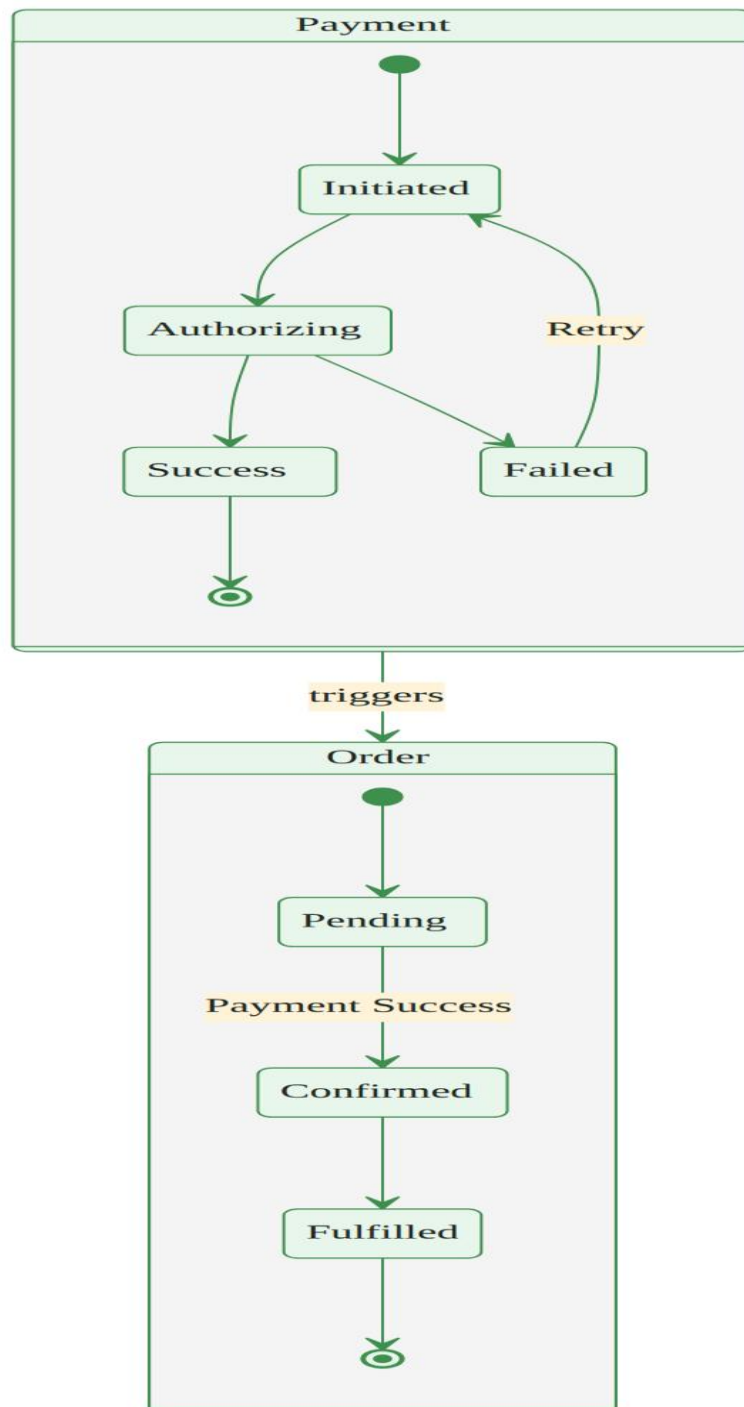


Figure 16: NovaMart Payment & Order Processing States

Business Process Flowcharts

Figure 17 traces the full business process from a customer's first visit to order fulfilment. Figures 18 and 19 zoom into the two most important sub-flows: the purchase flow from search to added-to-cart (Figure 18), and the checkout decision tree covering both payment paths (Figure 19). Figures 20 and 21 document the engineering-side release process: the CI/CD deployment pipeline as a decision flowchart (Figure 20) and as a staged pipeline view (Figure 21).

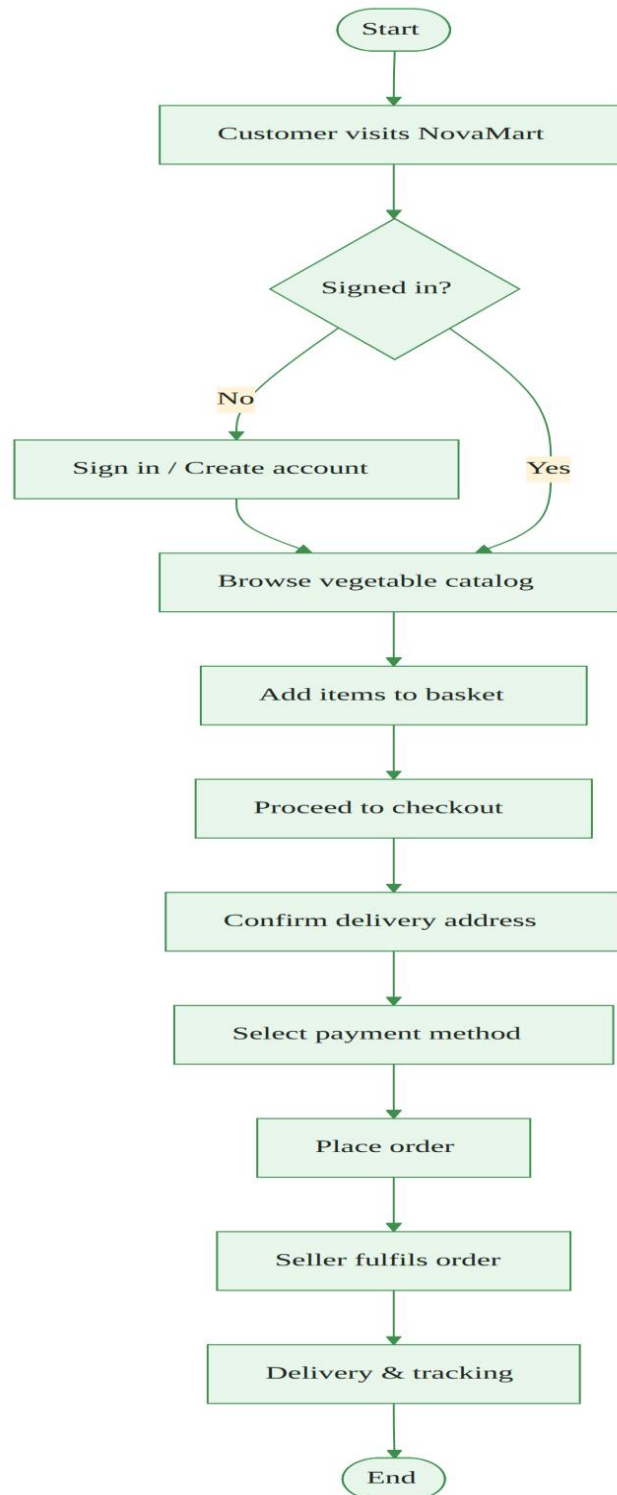


Figure 17: NovaMart Business Process Flow

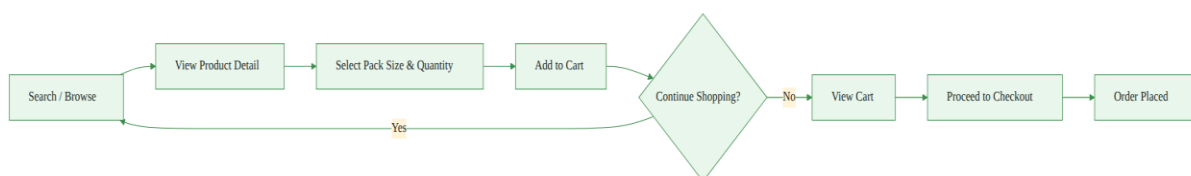


Figure 18: NovaMart Customer Purchase Flow

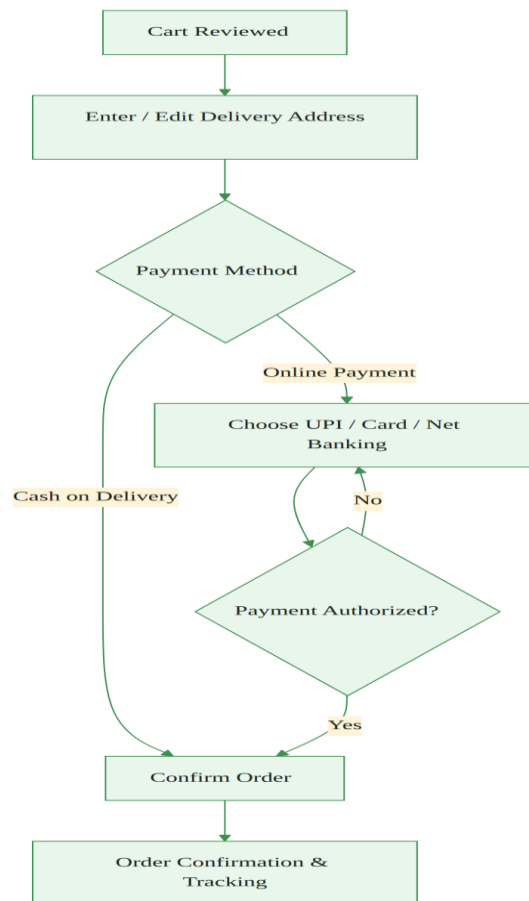


Figure 19: NovaMart Online Checkout Process

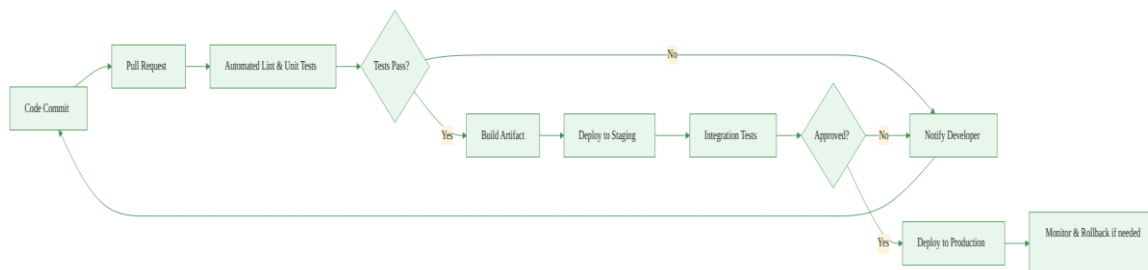


Figure 20: NovaMart CI/CD Deployment Pipeline

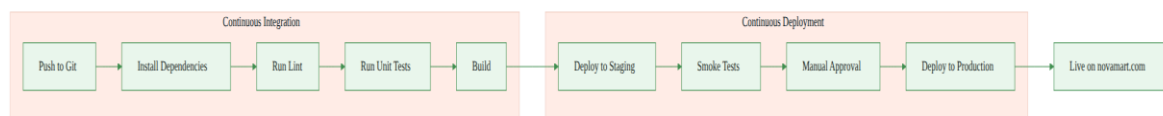


Figure 21: NovaMart Automated CI/CD Pipeline

Cross-Functional Process and Customer Experience

Figure 22 presents the order-processing flow as a swimlane diagram, making explicit which steps belong to the Customer, the Seller/Admin, and the Delivery Partner. Figure 23 takes the customer's perspective across the entire shopping journey from discovery through to leaving a review with a relative satisfaction score at each stage.

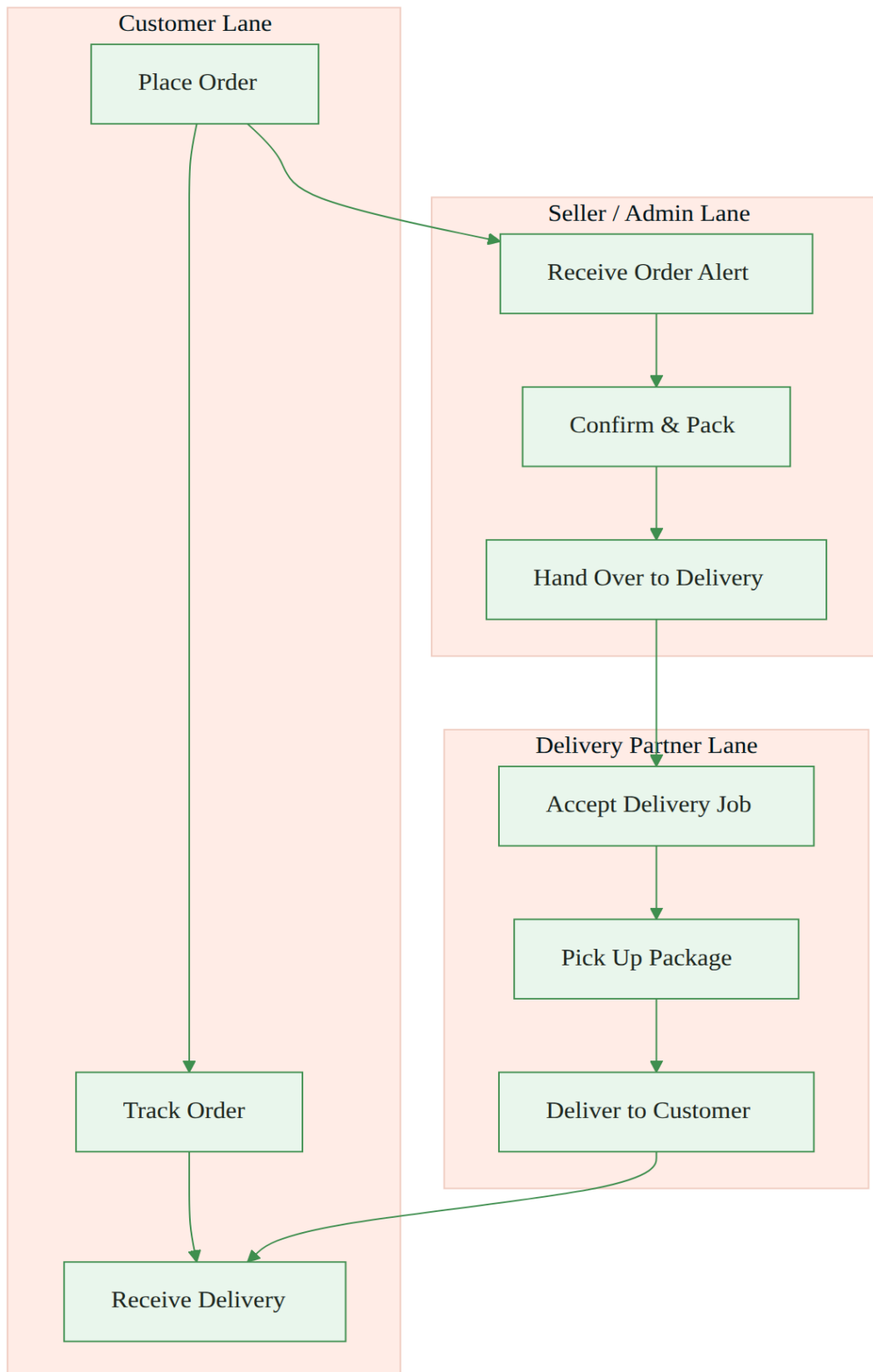


Figure 22: NovaMart Order Processing by User, Admin & Delivery

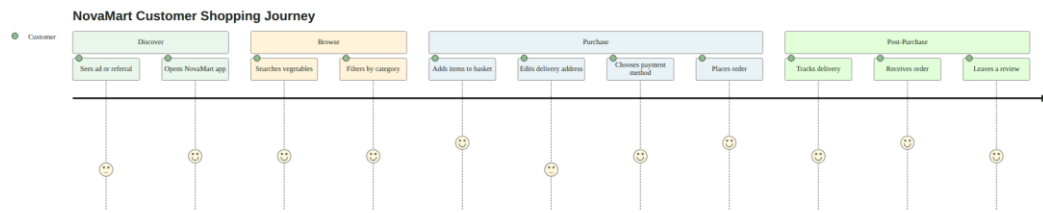


Figure 23: NovaMart Customer Shopping Journey

Development Workflow

Figures 24 and 25 document the Git branching strategy used during development: a standard feature-branch workflow merging into develop and then main (Figure 24), and the release/hotfix branching pattern used for production patches (Figure 25). Figure 26 shows a snapshot of the Kanban board used to track development tasks day to day.

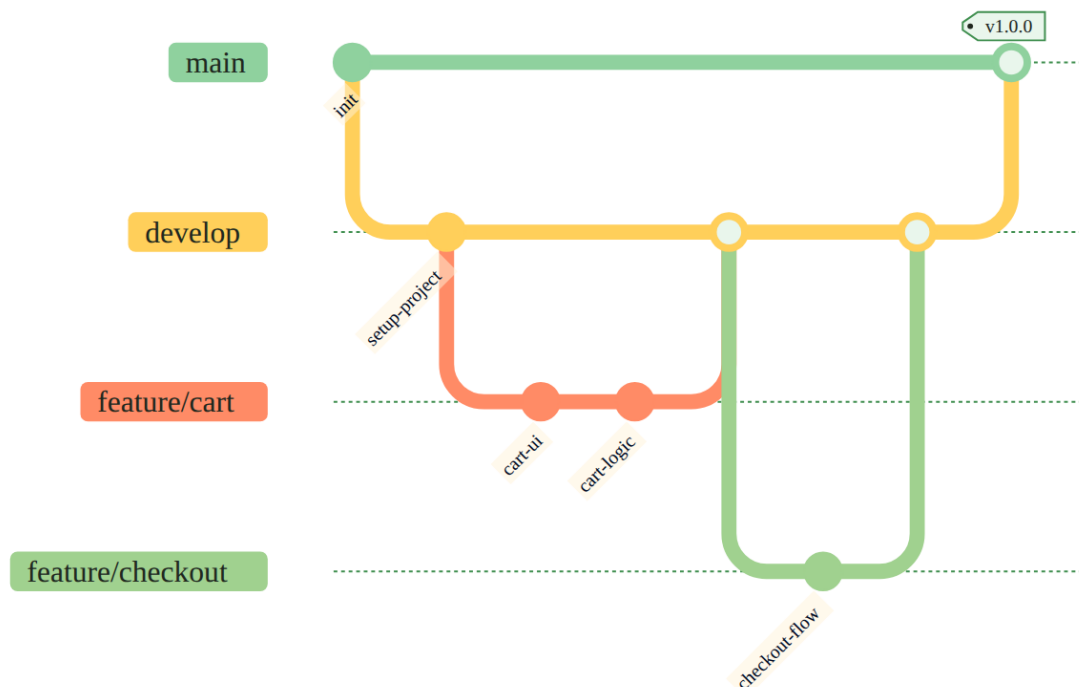


Figure 24: NovaMart Git Workflow

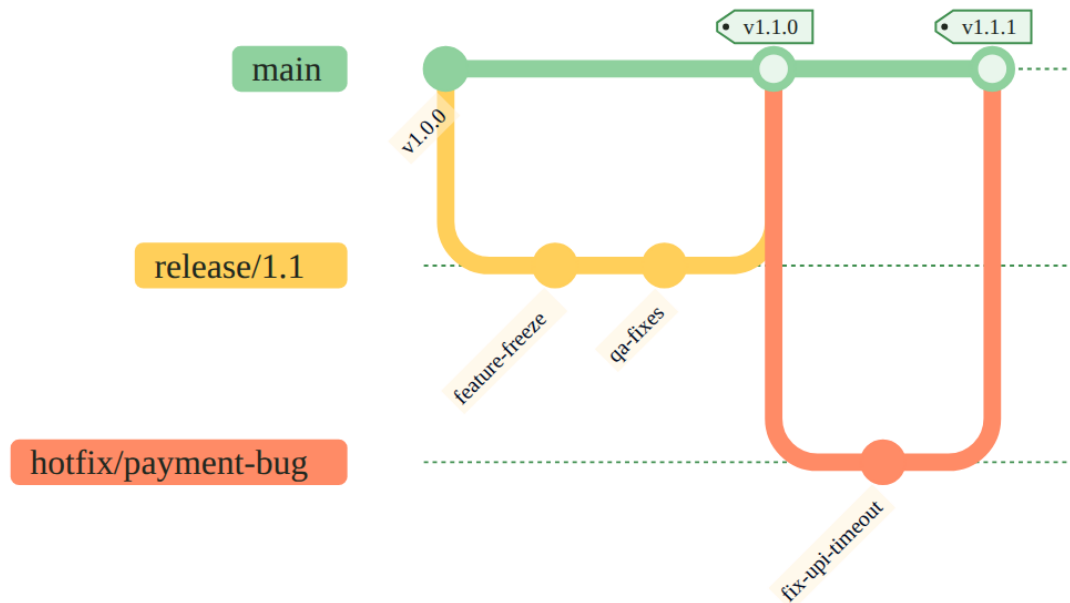


Figure 25: NovaMart Release & Hotfix Workflow

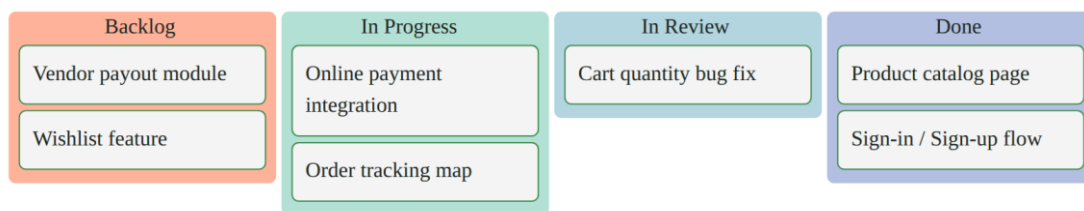


Figure 26: NovaMart Development Kanban Board

Project Planning

Figure 27 is the master Gantt chart for the project schedule described earlier in this chapter. Figures 28 and 29 present the same delivery plan as a timeline (development phases) and as a forward-looking product roadmap by quarter.

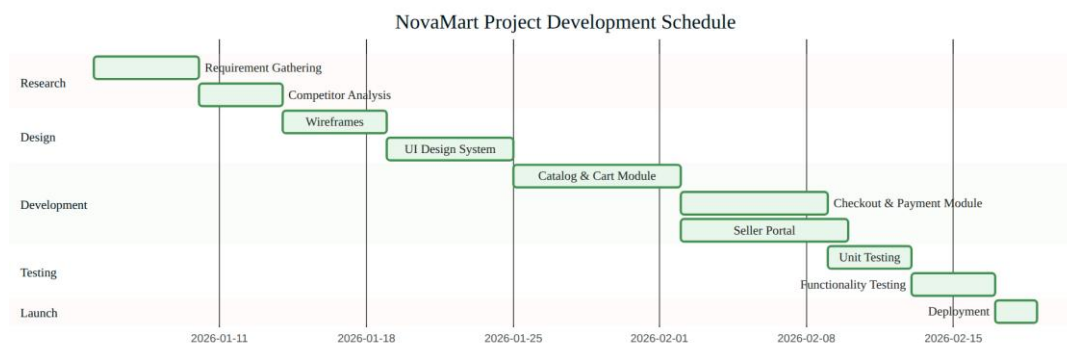


Figure 27: NovaMart Project Development Schedule

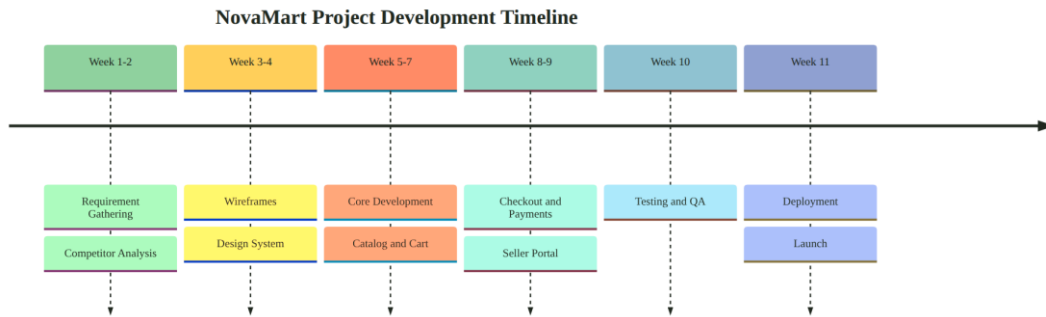


Figure 28: NovaMart Project Development Timeline

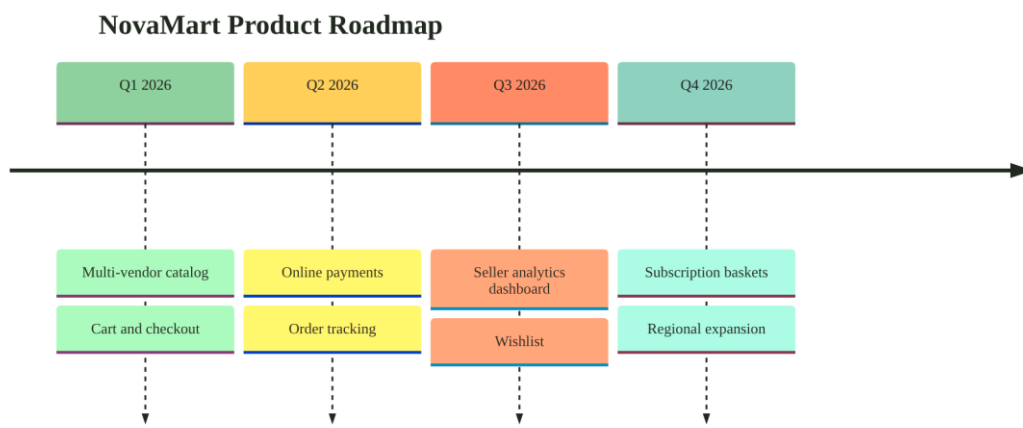


Figure 29: NovaMart Product Roadmap

Figures 30 and 31 give a mind-map view of the platform: the overall system from each user role's perspective (Figure 30), and a feature/module breakdown of the customer-facing application specifically (Figure 31).

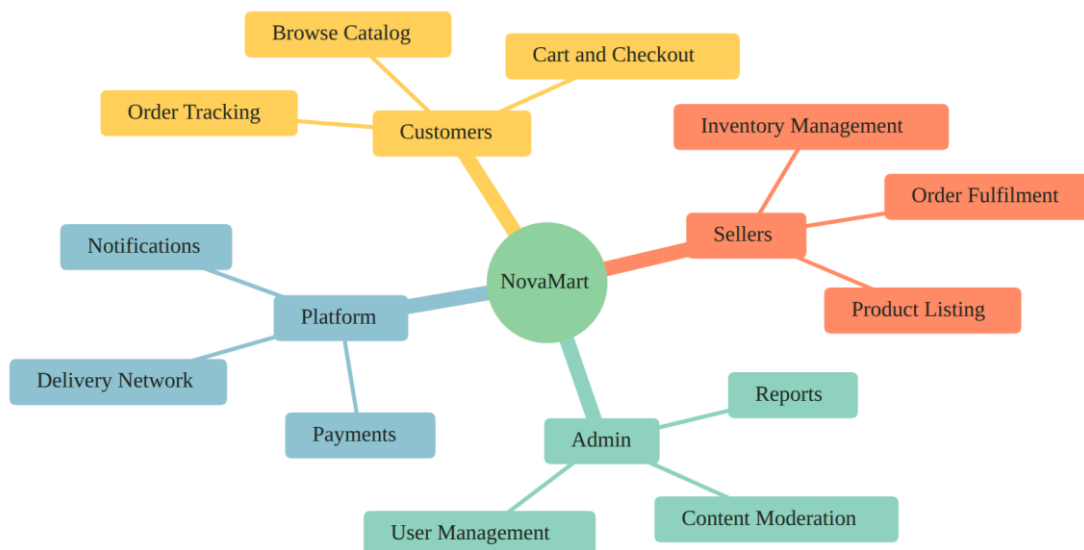


Figure 30: NovaMart Project Mind Map

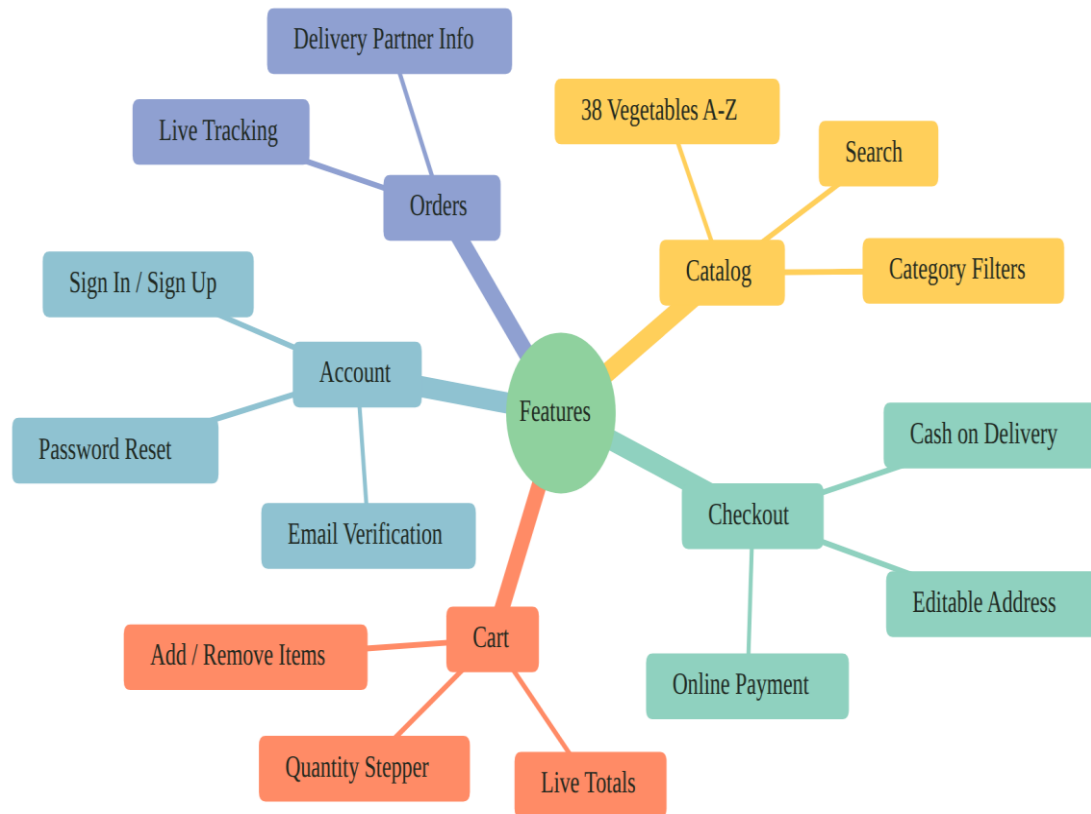


Figure 31: NovaMart Features and Modules Mind Map

Requirements Traceability and Navigation Analysis

Figure 32 traces each key functional requirement from Section 1.13 to the platform component that satisfies it, with an associated risk rating and verification method. Figure 33 is a Sankey diagram of observed customer navigation how traffic flows from Home through Search/Listing into Product Detail, Cart, and Checkout, including drop-off (Exit / Abandoned) at each stage.

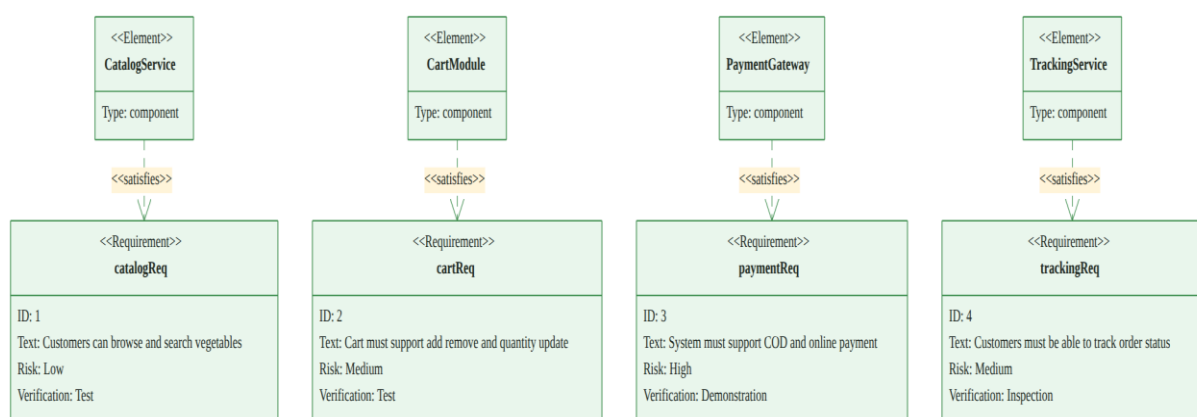


Figure 32: NovaMart System Requirements Diagram

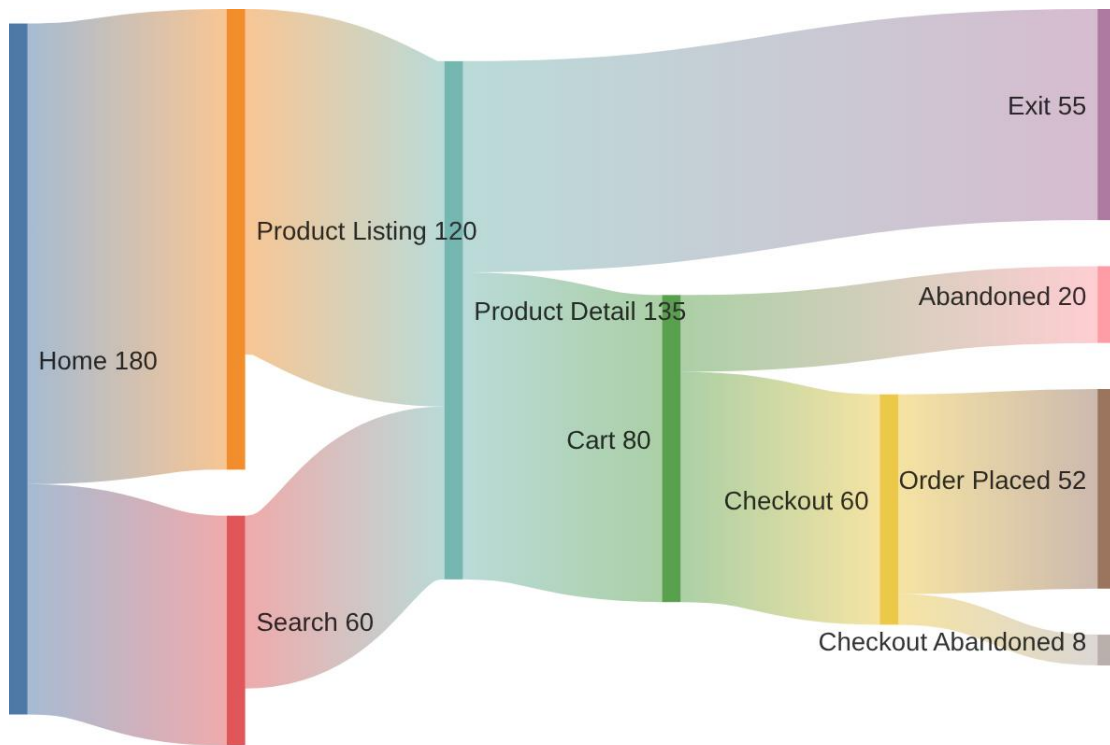


Figure 33: NovaMart User Navigation Flow

Analytics and Business Charts

Figures 34–38 present the analytical views used to support business decisions: monthly sales trend (Figure 34), category-wise sales share (Figure 35), a feature-performance comparison against the competitor average (Figure 36), a feature-priority matrix weighing effort against impact (Figure 37), and a tree map of catalog composition by category (Figure 38).

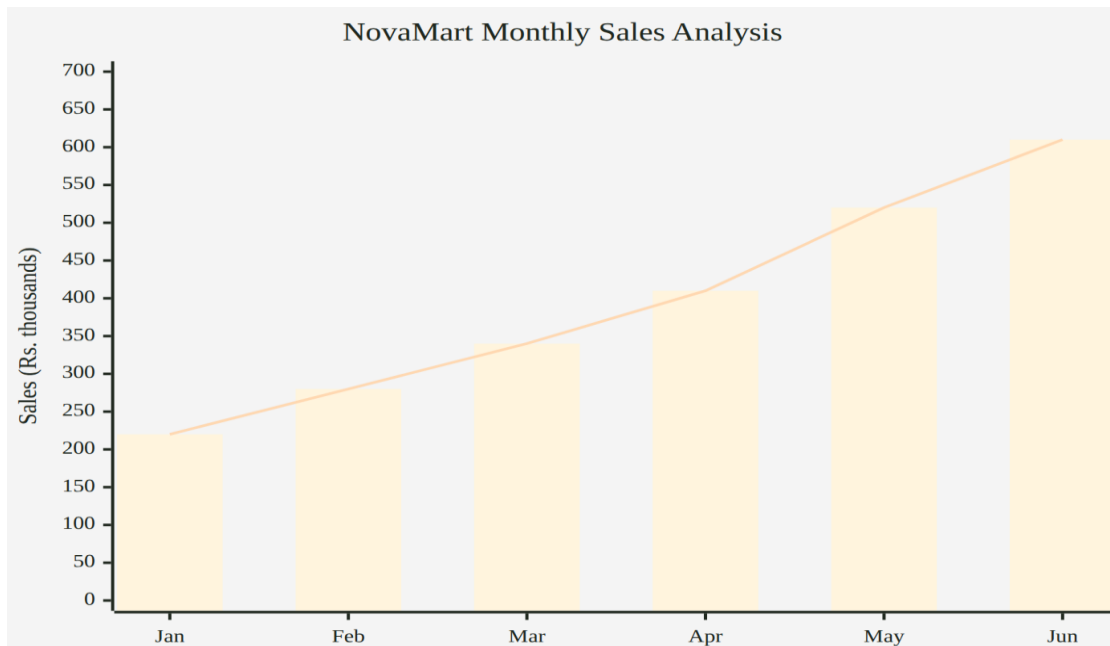


Figure 34: NovaMart Monthly Sales Analysis

NovaMart Sales Distribution by Category

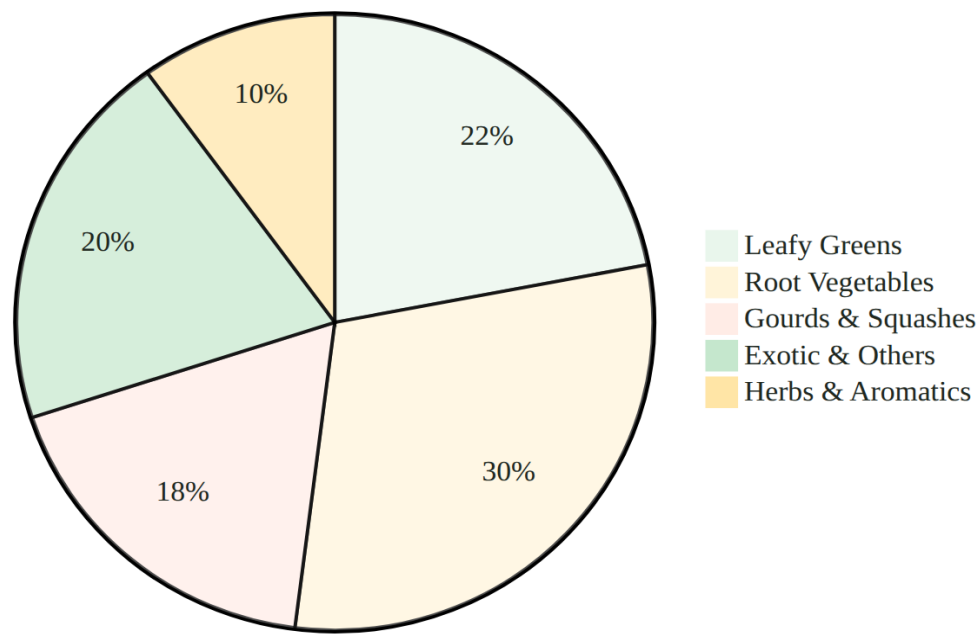


Figure 35: NovaMart Sales Distribution

NovaMart Feature Performance Comparison



Figure 36: NovaMart Feature Performance Comparison

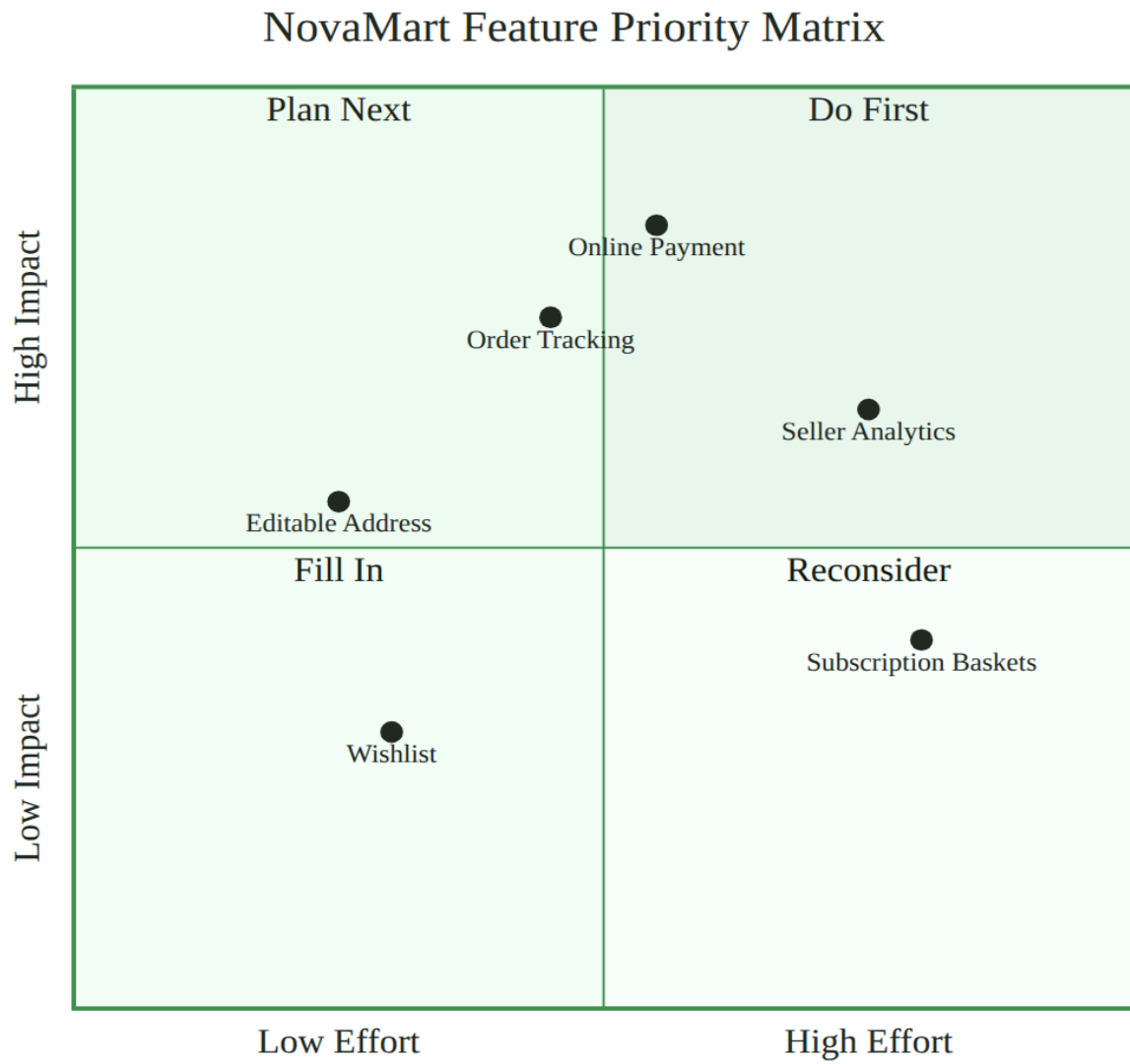


Figure 37: NovaMart Feature Priority Matrix

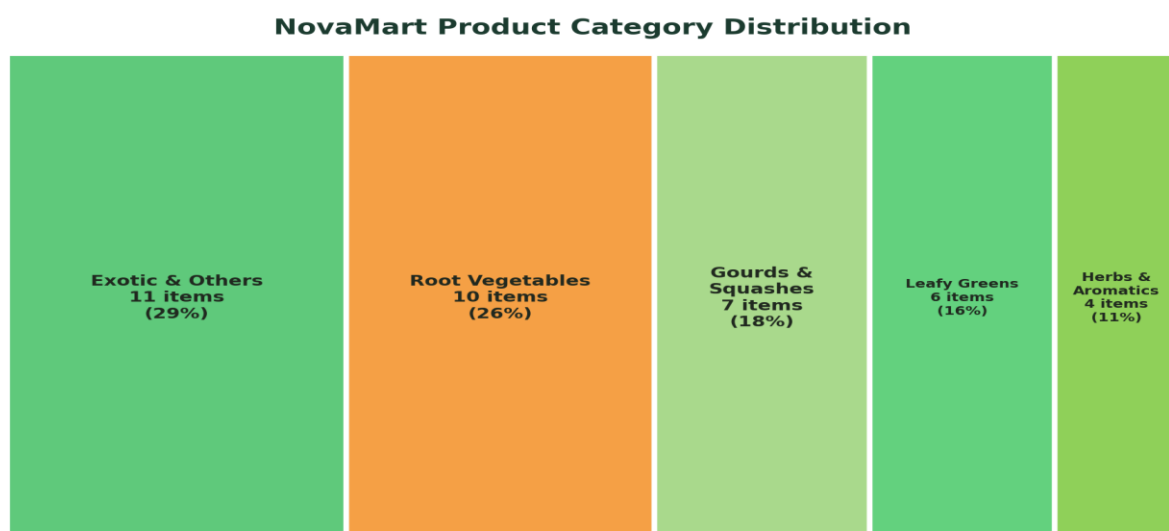


Figure 38: NovaMart Product Category Distribution

Stakeholder Relationships and Root-Cause Analysis

Figure 39 is a Venn diagram of the three core user roles, highlighting where their responsibilities and data overlap (e.g. order and payment data sits at the intersection of Customer and Seller activity, with Admin overseeing both). Figure 40 is an Ishikawa (fishbone) diagram used during design reviews to systematically enumerate the possible root causes of order failure, grouped into Payment, Inventory, System, and Logistics issue categories.

NovaMart Customer-Seller-Admin Relationship

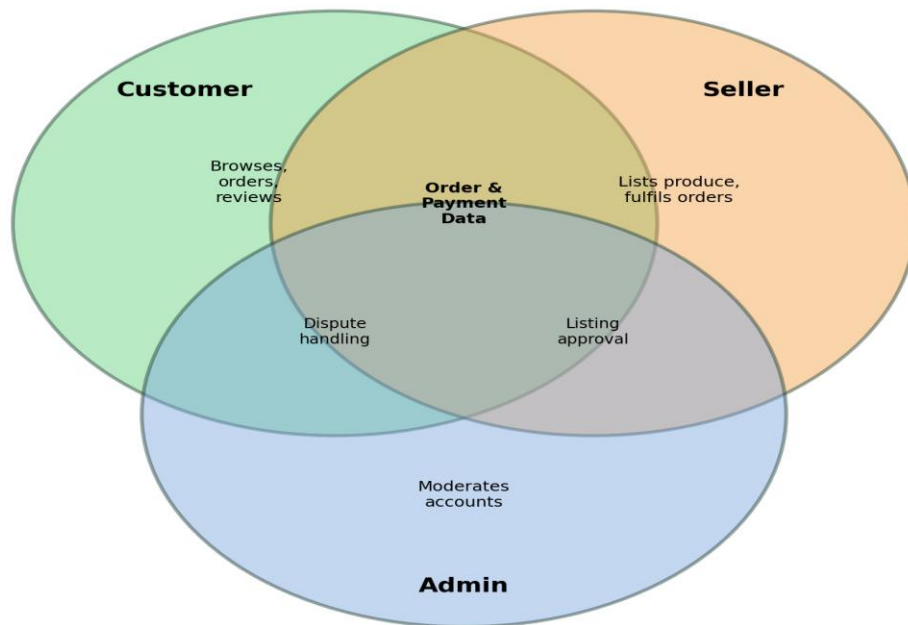


Figure 39: NovaMart Customer–Seller–Admin Relationship

NovaMart Order Failure Root Cause Analysis (Ishikawa / Fishbone Diagram)

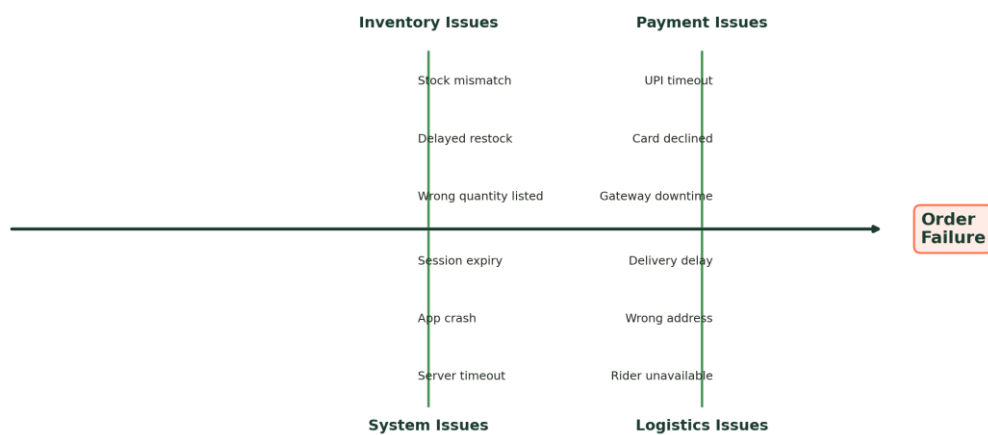


Figure 40: NovaMart Order Failure Root Cause Analysis

Event Modelling and Strategic Views

Figure 41 models the order lifecycle as a sequence of events and the commands/reactions that connect them useful for designing the event-driven parts of the Order Service. Figure 42 is a Wardley Map plotting each component of the NovaMart value chain by how visible it is to the customer (vertical axis) against how evolved/commoditised the underlying technology is (horizontal axis) informing build-vs-buy decisions (e.g. payment gateway and cloud hosting are commodity, so they are bought rather than built). Figure 43 applies the Cynefin framework to classify the types of decisions the NovaMart team faces, from Clear (routine order routing) to Complex (new feature discovery) to Chaotic (incident response).

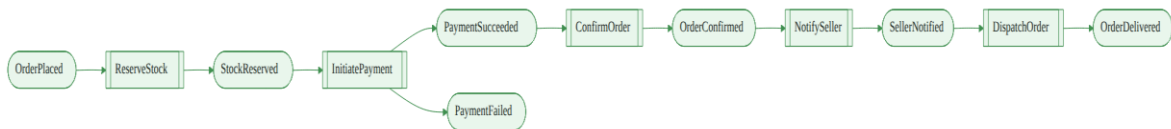


Figure 41: NovaMart Order Event Model

NovaMart Business Value Chain Map (Wardley Map)

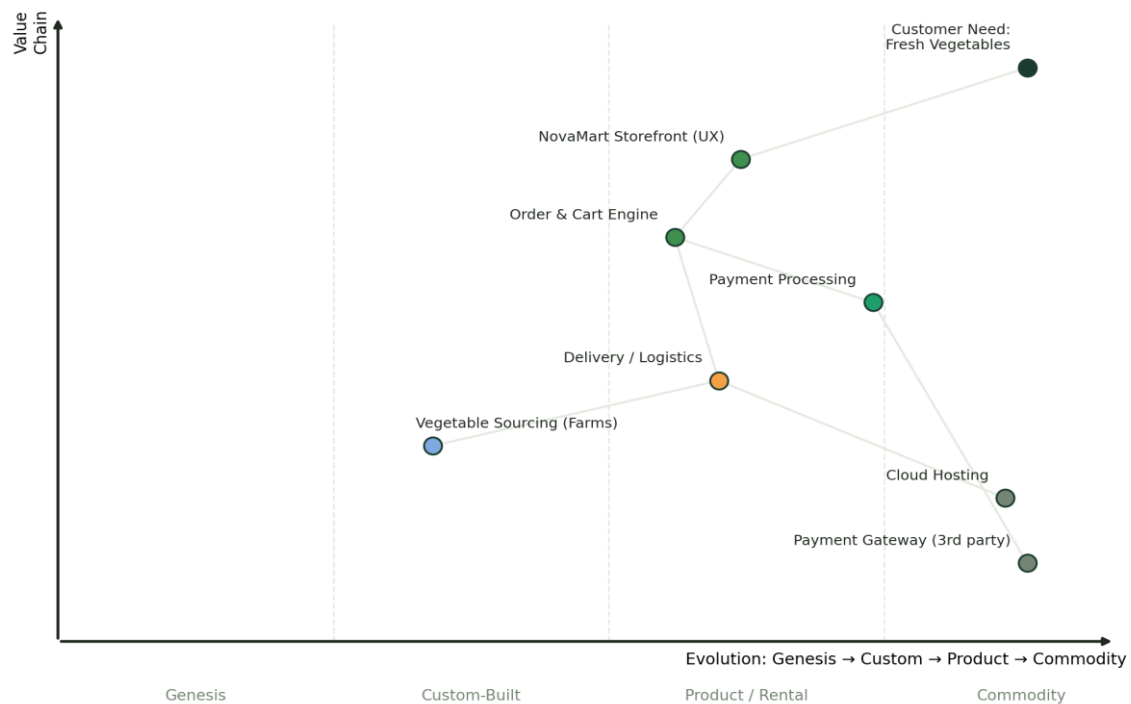


Figure 42: NovaMart Business Value Chain Map



Figure 43: NovaMart Decision-Making Framework

Hierarchy, Network, and Additional Views

Figure 44 shows the product category hierarchy used to structure the catalog's navigation and filtering. Figure 45 is a network packet-structure diagram illustrating how order and payment JSON payloads are framed at the transport level for the platform's API traffic. Figure 46 revisits the checkout process at the message level (functionally equivalent to a Zen UML view). Figure 47 closes the diagram set with a custom business view of NovaMart's value chain from farm to doorstep.

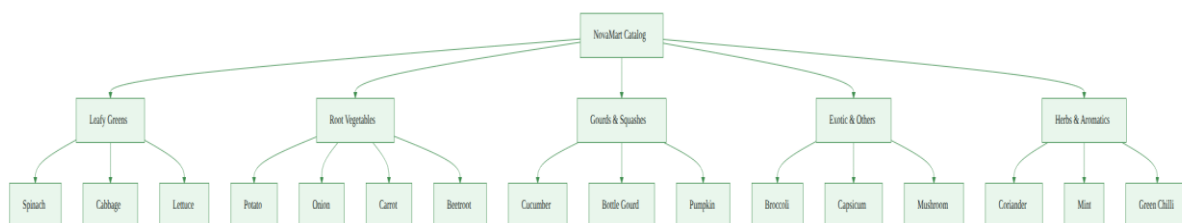


Figure 44: NovaMart Product Category Hierarchy

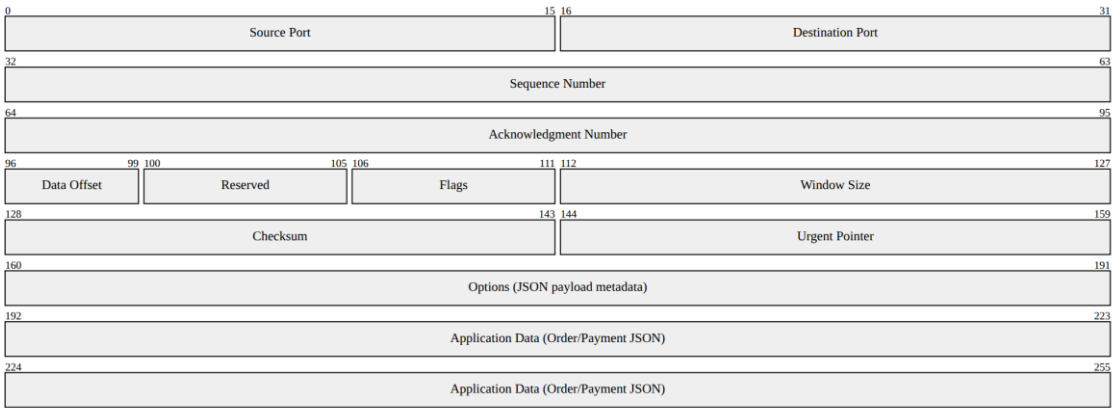


Figure 45: NovaMart Network Communication Diagram

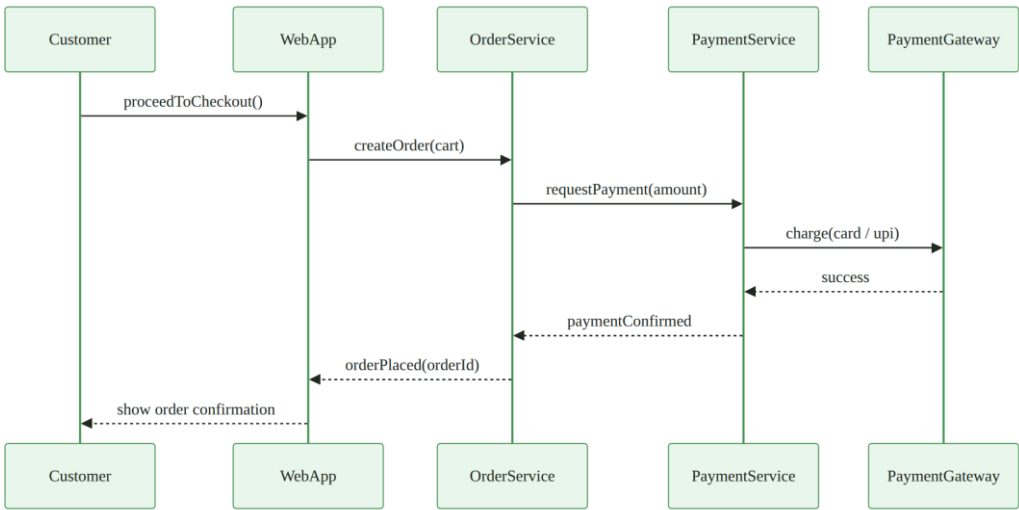


Figure 46: NovaMart Checkout Process (Zen UML)

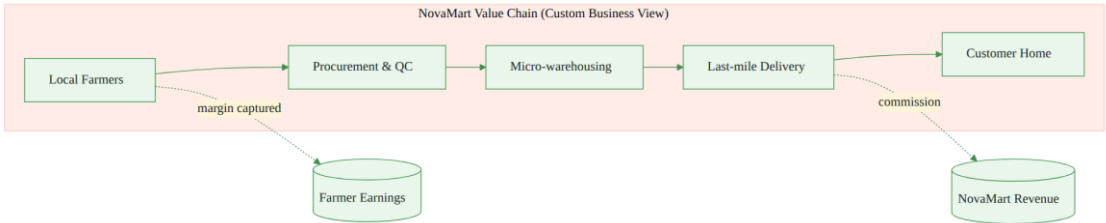


Figure 47: NovaMart Custom Business Diagrams

Consolidated Overview Diagrams

The 47 diagrams above each focus on one aspect of the system. The two consolidated views below combine several of those perspectives into a single picture each, useful as quick-reference posters: Figure 48 merges the customer/seller/admin site map with the underlying service and data architecture in one image, and Figure 49 zooms into the front-end and back-end internals every UI view, the front-end state and rendering layer, the API gateway, each backend service, and the data/external integrations they depend on in a single end-to-end picture.

NovaMart — Full System & Site Map

Complete site map (Customer, Seller, Admin) combined with the underlying system architecture

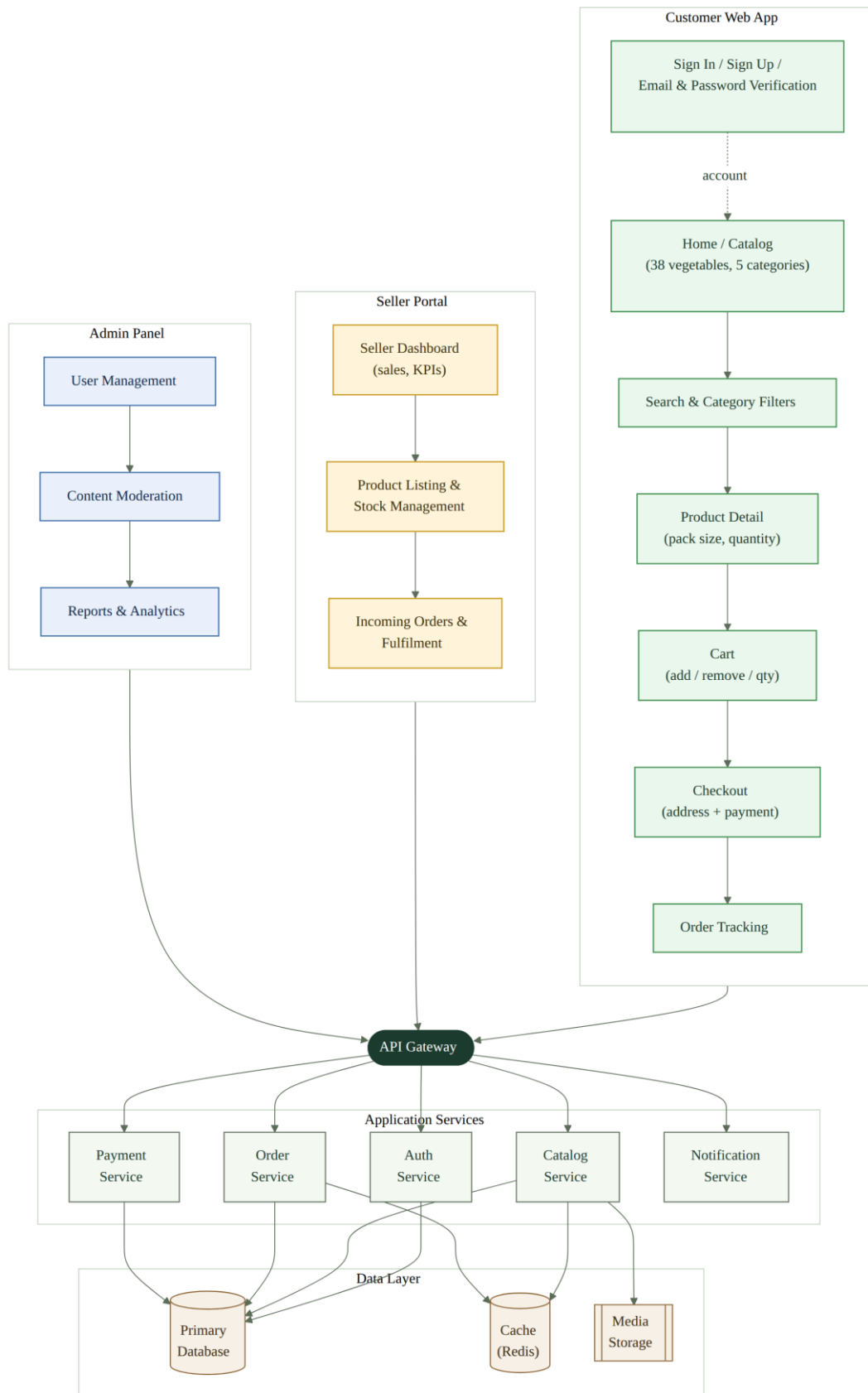


Figure 48: NovaMart Full System & Site Map (Consolidated Overview)

NovaMart — Full Front-End & Back-End Architecture

Client views, state & rendering layer, API gateway, business services, data store and external integrations

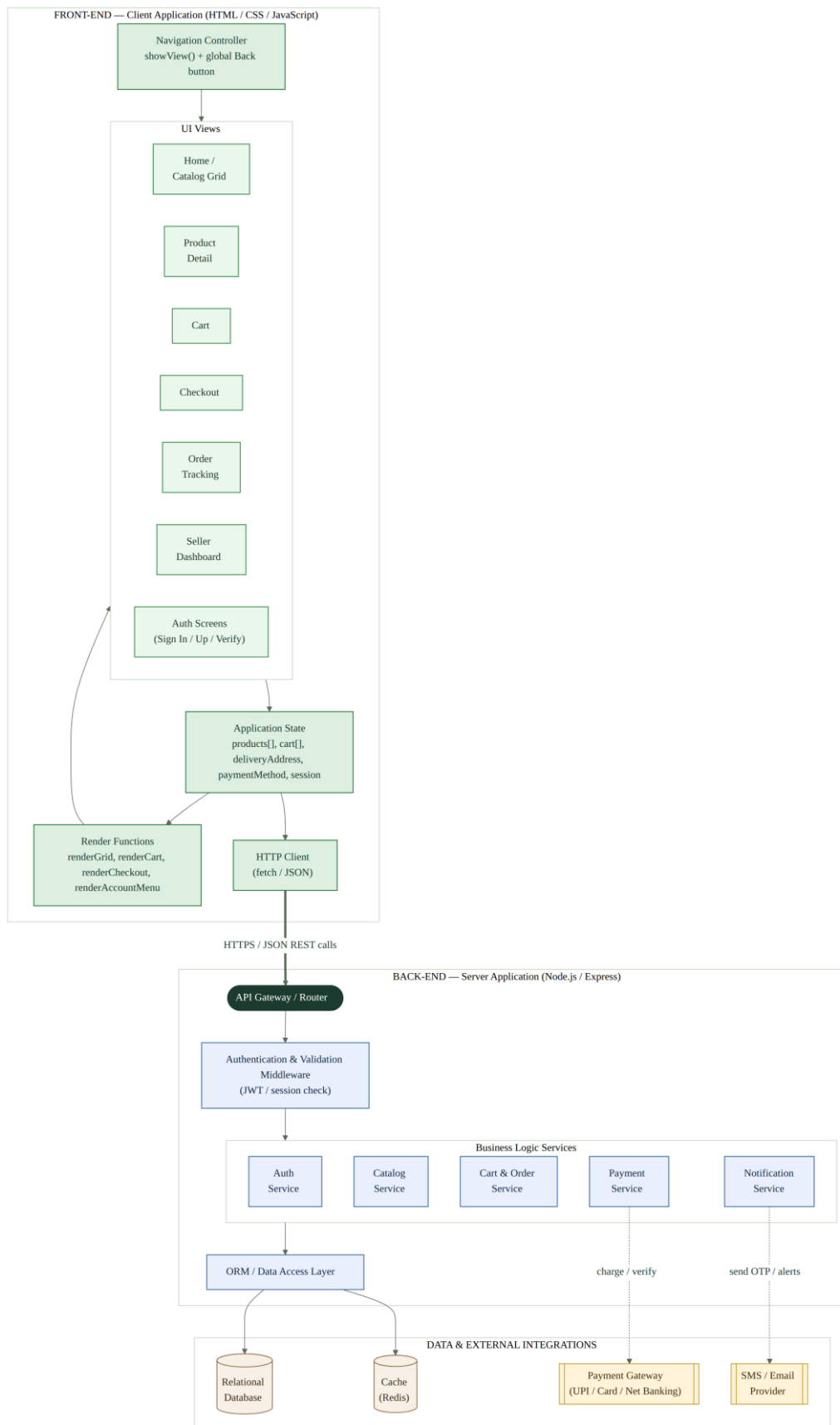


Figure 49: NovaMart Full Front-End & Back-End Architecture (Consolidated Overview)

Database and Tables

The relational schema shown in Figure 7 is implemented as the following core tables. Field names and types are illustrative of the production schema; full column-level constraints are maintained in the database migration scripts.

Table	Key Columns	Purpose
users	user_id (PK), name, email, phone, role, password_hash	Stores customer, seller, and admin accounts
sellers	seller_id (PK), farm_name, city, rating	Seller/vendor profile, linked to users
products	product_id (PK), name, category, price, unit, seller_id (FK)	Vegetable catalog listings
orders	order_id (PK), user_id (FK), status, placed_at, total_amount	Customer orders
order_items	order_item_id (PK), order_id (FK), product_id (FK), quantity, line_total	Line items within an order
payments	payment_id (PK), order_id (FK), method, status, amount	Payment records (COD / Online)
addresses	address_id (PK), user_id (FK), line, city, pincode	Delivery addresses

Implementation

programming Code

NovaMart's implementation is split into two halves that mirror the architecture in Chapter 2: a front-end single-page application (HTML, CSS, and vanilla JavaScript) that the customer, seller, and admin all interact with directly, and a back-end REST API (Node.js / Express, illustrated below in the same style used across the rest of this report) that the front-end talks to for every piece of authoritative data products, cart totals, orders, and payments. Every code excerpt in this section is explained line-by-block immediately underneath it.

Front-End Code

The front-end is structured as a set of independent, single-purpose modules navigation, catalog rendering, cart, checkout, and authentication that all read from and write to one shared, plain-object application state. This keeps each module easy to reason about in isolation while guaranteeing the UI never shows stale data, because every mutation re-runs the relevant render function immediately.

1. Design Tokens (CSS Custom Properties)

All colours, spacing, and corner radii used across every screen are declared once as CSS custom properties on: `root`. Every component in the stylesheet references these variables (for example `background: var(--card)`) instead of hard-coded values, so the entire visual identity of the site — the deep green navy header, the leaf-green accent, the coral highlight colour — can be changed from this one block without touching any other file.

```
:root {
  --navy:#1B3B2F; --indigo:#3F8F4F; --teal:#1E9E6B; --coral:#FF6F4A;
  --success:#2EAF6F; --warn:#E8A400;
  --bg:#F6F9F4; --card:#FFFFFF; --border:#E3EAE0;
  --text:#20281f; --muted:#758573; --radius:14px;
}
```

2. Product Data Model

Each of the 38 catalog entries is a plain JavaScript object with seven fields. `key` is a short, URL-safe identifier used to look up the matching illustration and to key cart line items; it is never shown to the user. `name`, `vendor`, `price`, `unit`, and `rating` are shown directly in the UI. `cat` maps the product into one of the five category chips (Leafy Greens, Root Vegetables, Gourds & Squashes, Exotic & Others, Herbs & Aromatics) used by the category filter. This structure corresponds directly to the products table described in Section 3.2 and the Product entity in Figure 7 / Figure 8.

```
const products = [
  { key:"tomato", name:"Farm-Fresh Tomatoes", vendor:"Greenfield Organic Farms",
    price:39, unit:"kg", rating:"4.6", cat:"Root Vegetables" },
  { key:"spinach", name:"Organic Spinach (Palak)", vendor:"Greenfield Organic Farms",
    price:25, unit:"bunch", rating:"4.8", cat:"Leafy Greens" },
  // ... 36 further vegetables across 5 categories
];
```

3. Navigation Controller

`showView(id)` is the single function responsible for switching screens. It removes the `.active` class from every `.view` section, adds it back only to the requested one, and synchronises three pieces of UI state that must always agree with the current screen: which tab is highlighted in the top navigation, whether

the persistent Back button is visible (hidden only on Home), and that the account dropdown menu is closed. Centralising this logic in one function is what guarantees the Back button behaves identically from every screen there is only one code path that can change the visible view, so there is nowhere for inconsistent navigation state to creep in.

```
function showView(id){
  document.querySelectorAll('.view').forEach(v => v.classList.remove('active'));
  document.getElementById(id).classList.add('active');
  document.querySelectorAll('nav.tabs button').forEach(b =>
    b.classList.toggle('active', b.dataset.view === id));
  document.getElementById('global-back').classList.toggle('show', id !== 'home');
  closeAccountMenu();
  window.scrollTo({ top: 0, behavior: 'smooth' });
}

document.querySelectorAll('nav.tabs button').forEach(b => {
  b.addEventListener('click', () => showView(b.dataset.view));
});
```

4. Catalog Rendering and Search

`renderGrid(filter, query)` is called every time the customer types in the search box or clicks a category chip. It filters the in-memory products array by category and by a case-insensitive substring match on the product name, rebuilds the visible count line (“Showing X of 38 vegetables”), and re-renders each surviving product as a card. Because the entire grid is rebuilt from the same products array on every keystroke, the search box and the category chips never fall out of sync with each other — there is exactly one source of truth for “what should currently be on screen.”

```
function renderGrid(filter, query){
  filter = filter || 'All';
  query = (query || '').toLowerCase().trim();
  const grid = document.getElementById('product-grid');
  grid.innerHTML = '';

  const list = products.filter(p =>
    (filter === 'All' || p.cat === filter) &&
    (!query || p.name.toLowerCase().includes(query))
  );

  document.getElementById('count-line').textContent =
    'Showing ' + list.length + ' of ' + products.length + ' vegetables';

  list.forEach(p => {
    const el = document.createElement('div');
    el.className = 'card p-card';
    el.onclick = () => openProduct(p);
    el.innerHTML =
      '<span class="organic-badge">FRESH</span>' +
      '<div class="p-thumb" style="background:' + p.bg + '>' + vegIcon(p.key,78) + '</div>' +
      '<div class="p-name">' + p.name + '</div>' +
      '<div class="p-vendor">' + p.vendor + '</div>' +
      '<div class="p-row">' +
        '<div><div class="p-price">Rs ' + p.price + '/' + p.unit + '</div>' +
        '<div class="p-rating">Rating ' + p.rating + '</div></div>' +
        '<button class="add-btn" onclick="event.stopPropagation();" +
        'addToCart(\'' + p.key + '\',1);showToast(\'' + p.name + ' added to
basket\')"></button>' +
      '</div>';
    grid.appendChild(el);
  });
}
```


5. Cart Module

The cart is a plain array of { key, mult } line items, where mult is the ordered quantity in the product's native unit. All three possible cart mutations funnel through the same three small functions shown below, and every one of them ends by calling `renderCart()`, which is also where `computeCartTotals()` is invoked to recompute the subtotal, 2% tax, and total directly from the current cart contents rather than from any cached value. This is the design decision that eliminates stale-total bugs: there is no total stored anywhere that could drift from the cart's real contents.

```
function addToCart(key, mult){
  mult = mult || 1;
  const existing = cart.find(c => c.key === key);
  if (existing) { existing.mult += mult; }
  else { cart.push({ key, mult }); }
  renderCart();
}

function removeFromCart(key){
  cart = cart.filter(c => c.key !== key);
  renderCart();
  showToast('Item removed from basket');
}

function changeCartQty(key, delta){
  const item = cart.find(c => c.key === key);
  if (!item) return;
  item.mult = Math.max(1, Math.round((item.mult + delta) * 10) / 10);
  renderCart();
}

function computeCartTotals(){
  const subtotal = cart.reduce((sum, c) => {
    const p = findProduct(c.key);
    return sum + (p ? p.price * c.mult : 0);
  }, 0);
  const tax = Math.round(subtotal * 0.02);
  const total = Math.round(subtotal + tax);
  return { subtotal: Math.round(subtotal), tax, total };
}
```

6. Checkout Module — Address and Payment

The checkout screen holds a single `deliveryAddress` object, editable in place through `saveAddress()`, and a `paymentMethod` flag toggled between Cash on Delivery and Online Payment via `selectPayment()`. When Online Payment is chosen, a secondary `onlineSubMethod` value (UPI, Card, or Net Banking) controls which input field is shown. This mirrors the Payment/Order state interaction documented in Figure 16, and on the back-end side maps directly onto the `/api/payments/initiate` route shown later in this section.

```
let deliveryAddress = {
  name: "Anita Rao", phone: "+91 98765 43210",
  line: "221B, MG Road, Banjara Hills", city: "Hyderabad", pincode: "500034"
};
let paymentMethod = 'cod';      // 'cod' | 'online'
let onlineSubMethod = 'upi';    // 'upi' | 'card' | 'netbanking'

function saveAddress(){
  deliveryAddress = {
    name: document.getElementById('addr-name').value || deliveryAddress.name,
    phone: document.getElementById('addr-phone').value || deliveryAddress.phone,
    line: document.getElementById('addr-line').value || deliveryAddress.line,
    city: document.getElementById('addr-city').value || deliveryAddress.city,
```

```

    pincode: document.getElementById('addr-pincode').value || deliveryAddress.pincode,
  };
  renderAddressDisplay();
  showToast('Delivery address updated');
}

function selectPayment(method){
  paymentMethod = method;
  document.getElementById('pay-cod').classList.toggle('selected', method === 'cod');
  document.getElementById('pay-online').classList.toggle('selected', method === 'online');
  document.getElementById('online-details').style.display =
    method === 'online' ? 'block' : 'none';
}

```

7. Authentication and Verification Flow

The front-end models sign-in state with a single session object. `startEmailVerification()` captures whatever email address the customer typed into the sign-up form and forwards it to the 6-digit OTP screen; the OTP boxes auto-advance to the next field as each digit is typed and support Backspace to move focus back, which is a usability detail purely handled client-side. `finishEmailVerification()` and `handleLogin()` are intentionally the same code path a real backend call would use — once the code is connected to the `/api/auth/verify-email` and `/api/auth/login` routes shown in the Back-End Code section, only the body of these two functions changes, not anything that calls them.

```

let session = { loggedIn: false, name: "", email: "" };

function startEmailVerification(){
  const emailInput = document.getElementById('signup-email');
  document.getElementById('verify-email-addr').textContent =
    (emailInput && emailInput.value) || 'you@example.com';
  showView('verify-email');
}

function finishEmailVerification(){
  showToast('Email verified successfully');
  handleLogin();
}

function handleLogin(){
  session.loggedIn = true;
  renderAccountButton();
  showToast('Signed in as ' + session.name);
  showView('home');
}

function handleLogout(){
  session.loggedIn = false;
  renderAccountButton();
  closeAccountMenu();
  showToast('You have been logged out');
  showView('home');
}

// OTP boxes auto-advance to the next field as each digit is typed
document.querySelectorAll('.otp-row').forEach(row => {
  const boxes = row.querySelectorAll('.otp-box');
  boxes.forEach((box, i) => {
    box.addEventListener('input', () => {
      box.value = box.value.replace(/^[^0-9]/g, '');
      if (box.value && boxes[i + 1]) boxes[i + 1].focus();
    });
    box.addEventListener('keydown', (e) => {
      if (e.key === 'Backspace' && !box.value && boxes[i - 1]) boxes[i - 1].focus();
    });
  });
});

```

```
});
});
```

Back-End Code

The back end is a REST API built with Node.js and Express, structured around the same five services shown in the architecture diagrams (Figures 1, 2, 5, and 49): Auth, Catalog, Order, Payment, and Notification. Every route validates its inputs, delegates data access to a Sequelize model layer matching the ERD in Figure 7, and returns JSON that the front-end functions above are written to consume directly. Each file below is explained immediately underneath its listing.

1. Server Entry Point and Middleware Stack

server.js wires the whole API together. `helmet()` sets a standard set of security-related HTTP response headers; `cors()` allows the front-end's origin to call this API from the browser (necessary because the front-end and API are served from different origins in production); `express.json()` parses incoming JSON request bodies into `req.body`; and `express-rate-limit` caps each client to 120 requests per minute as a basic defence against abuse. The four route modules are mounted under `/api/auth`, `/api/products`, `/api/orders`, and `/api/payments` — matching the Auth, Catalog, Order, and Payment services in Figure 1. The error-handling middleware is registered last, which is an Express requirement: it only catches errors passed to `next(err)` from routes registered before it.

```
// server.js -- application entry point
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');

const authRoutes = require('./routes/auth');
const catalogRoutes = require('./routes/catalog');
const orderRoutes = require('./routes/orders');
const paymentRoutes = require('./routes/payments');
const { errorHandler } = require('./middleware/errorHandler');

const app = express();

app.use(helmet()); // sets secure HTTP headers
app.use(cors()); // allows the front-end origin to call this API
app.use(express.json()); // parses JSON request bodies
app.use(rateLimit({ windowMs: 60000, max: 120 })); // basic abuse protection

app.use('/api/auth', authRoutes);
app.use('/api/products', catalogRoutes);
app.use('/api/orders', orderRoutes);
app.use('/api/payments', paymentRoutes);

app.use(errorHandler); // must be registered last

const PORT = process.env.PORT || 4000;
app.listen(PORT, () => console.log('NovaMart API listening on port ' + PORT));
```

2. Database Models

The seven models below are declared with Sequelize and correspond one-to-one with the seven tables introduced in Section 2.4 and Figure 7. The relationship calls at the bottom of the file (`User.hasOne(Seller)`, `Seller.hasMany(Product)`, and so on) are what let a single query pull back related data for example, fetching a product together with its seller's name and rating in one round trip, as the catalog route below does. Enumerated fields such as `Order.status` and `Payment.method` are

constrained at the database level to the exact set of values the state diagrams in Figures 14–16 define, which prevents an invalid status from ever being written.

```
// models/index.js -- Sequelize model definitions matching the ERD (Figure 7)
const { DataTypes } = require('sequelize');
const sequelize = require('../db/connection');

const User = sequelize.define('User', {
  name: { type: DataTypes.STRING, allowNull: false },
  email: { type: DataTypes.STRING, allowNull: false, unique: true },
  phone: DataTypes.STRING,
  role: { type: DataTypes.ENUM('customer', 'seller', 'admin'), defaultValue: 'customer' },
  passwordHash: { type: DataTypes.STRING, allowNull: false },
  emailVerified: { type: DataTypes.BOOLEAN, defaultValue: false },
});

const Seller = sequelize.define('Seller', {
  farmName: { type: DataTypes.STRING, allowNull: false },
  city: DataTypes.STRING,
  rating: { type: DataTypes.FLOAT, defaultValue: 0 },
});

const Product = sequelize.define('Product', {
  name: { type: DataTypes.STRING, allowNull: false },
  category: DataTypes.STRING,
  price: { type: DataTypes.FLOAT, allowNull: false },
  unit: { type: DataTypes.STRING, allowNull: false }, // 'kg' | 'bunch' | 'pc'
  stock: { type: DataTypes.INTEGER, defaultValue: 0 },
});

const Order = sequelize.define('Order', {
  status: { type: DataTypes.ENUM('placed', 'confirmed', 'packed', 'out_for_delivery', 'delivered', 'cancelled'), defaultValue: 'placed' },
  totalAmount: { type: DataTypes.FLOAT, allowNull: false },
});

const OrderItem = sequelize.define('OrderItem', {
  quantity: { type: DataTypes.FLOAT, allowNull: false },
  lineTotal: { type: DataTypes.FLOAT, allowNull: false },
});

const Payment = sequelize.define('Payment', {
  method: { type: DataTypes.ENUM('cod', 'upi', 'card', 'netbanking'), allowNull: false },
  status: { type: DataTypes.ENUM('pending', 'success', 'failed'), defaultValue: 'pending' },
  amount: { type: DataTypes.FLOAT, allowNull: false },
});

const Address = sequelize.define('Address', {
  line: DataTypes.STRING, city: DataTypes.STRING, pincode: DataTypes.STRING,
});

// Relationships (mirrors Figure 7 -- Entity Relationship Diagram)
User.hasOne(Seller); Seller.belongsTo(User);
Seller.hasMany(Product); Product.belongsTo(Seller);
User.hasMany(Order); Order.belongsTo(User);
Order.hasMany(OrderItem); OrderItem.belongsTo(Order);
Product.hasMany(OrderItem); OrderItem.belongsTo(Product);
Order.hasOne(Payment); Payment.belongsTo(Order);
User.hasMany(Address); Address.belongsTo(User);
Order.belongsTo(Address);

module.exports = { User, Seller, Product, Order, OrderItem, Payment, Address };
```

3. Authentication Middleware

`requireAuth` reads the `Authorization: Bearer <token>` header, verifies the JWT against the server's secret, and attaches the decoded payload to `req.user` for every downstream route handler to use. `requireRole(...roles)` is a small factory function that returns a second middleware, letting a route declare, for example, `requireRole('seller')` to ensure only sellers can update their own product listings, or `requireRole('admin')` for moderation endpoints. Both middlewares fail closed: any missing, malformed, or expired token results in a 401 response before the request ever reaches business logic.

```
// middleware/auth.js -- verifies the JWT on protected routes
const jwt = require('jsonwebtoken');

function requireAuth(req, res, next){
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ error: 'Missing authentication token' });

  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (err) {
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
}

function requireRole(...roles){
  return (req, res, next) => {
    if (!req.user || !roles.includes(req.user.role)) {
      return res.status(403).json({ error: 'Insufficient permissions' });
    }
    next();
  };
}

module.exports = { requireAuth, requireRole };
```

4. Authentication Routes

`POST /register` hashes the incoming password with `bcrypt` (never storing plaintext, per the Safety Requirements in Section 1.13), creates the user record, generates a 6-digit numeric OTP, and emails it through the Notification service this is the server-side counterpart of the `startEmailVerification()` front-end function shown earlier. `POST /login` verifies the password hash and issues a signed JWT valid for 7 days, containing just enough claims (user id and role) for `requireAuth` to authorize later requests. `POST /verify-email` checks the submitted code against the stored, time-limited OTP and flips the user's `emailVerified` flag, matching the OTP screen behaviour documented in the User Guide.

```
// routes/auth.js -- registration, login, email verification, password reset
const router = require('express').Router();
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
const crypto = require('crypto');
const { User } = require('../models');
const { sendVerificationEmail } = require('../services/notification');

router.post('/register', async (req, res, next) => {
  try {
    const { name, email, password } = req.body;
    const passwordHash = await bcrypt.hash(password, 12);
    const user = await User.create({ name, email, passwordHash });

    const otp = crypto.randomInt(100000, 999999).toString();
    await sendVerificationEmail(user.email, otp); // 6-digit code, matches Figure 12/UI
```

```

    res.status(201).json({ id: user.id, message: 'Verification code sent' });
  } catch (err) { next(err); }
});

router.post('/login', async (req, res, next) => {
  try {
    const { email, password } = req.body;
    const user = await User.findOne({ where: { email } });
    if (!user || !(await bcrypt.compare(password, user.passwordHash))) {
      return res.status(401).json({ error: 'Invalid email or password' });
    }
    const token = jwt.sign(
      { id: user.id, role: user.role },
      process.env.JWT_SECRET,
      { expiresIn: '7d' }
    );
    res.json({ token, name: user.name, role: user.role });
  } catch (err) { next(err); }
});

router.post('/verify-email', async (req, res, next) => {
  try {
    const { userId, code } = req.body;
    const valid = await verifyStoredOtp(userId, code); // checked against hashed, TTL-bound
    OTP
    if (!valid) return res.status(400).json({ error: 'Incorrect or expired code' });

    await User.update({ emailVerified: true }, { where: { id: userId } });
    res.json({ message: 'Email verified' });
  } catch (err) { next(err); }
});

module.exports = router;

```

5. Catalog Routes

GET /api/products is the server-side implementation of the same filtering logic `renderGrid()` performs client-side: an optional category query parameter narrows results to one of the five categories, and an optional q parameter performs a case-insensitive substring search on the product name using SQL `ILIKE`. Each product is returned together with its seller's name and rating via a single joined query, which is what powers the “Sold by ...” line on the product detail screen. GET /api/products/:id returns one product for the detail view, returning a 404 if the id does not exist rather than an empty object, so the front-end can distinguish “not found” from “no data yet.”

```

// routes/catalog.js -- product listing, search, and category filtering
const router = require('express').Router();
const { Op } = require('sequelize');
const { Product, Seller } = require('../models');

// GET /api/products?category=Leafy+Greens&q=spinach
router.get('/', async (req, res, next) => {
  try {
    const { category, q } = req.query;
    const where = {};
    if (category && category !== 'All') where.category = category;
    if (q) where.name = { [Op.iLike]: '%' + q + '%' };

    const products = await Product.findAll({
      where,
      include: [{ model: Seller, attributes: ['farmName', 'rating'] }],
      order: [['name', 'ASC']],
    });
  }

```

```

    res.json(products);
  } catch (err) { next(err); }
});

router.get('/:id', async (req, res, next) => {
  try {
    const product = await Product.findByPk(req.params.id, {
      include: [{ model: Seller, attributes: ['farmName', 'rating', 'city'] }],
    });
    if (!product) return res.status(404).json({ error: 'Product not found' });
    res.json(product);
  } catch (err) { next(err); }
});

module.exports = router;

```

6. Cart and Order Routes

POST /api/orders is the server-side counterpart of the client-side placeOrder() flow. It runs inside a database transaction so that stock checks, stock decrements, and order/order-item creation either all succeed together or all roll back together this is what prevents two customers from simultaneously buying the last unit of a product. For each cart line item it re-validates the product's live stock and price on the server (never trusting the price the client sent), computes the line total itself, and only then creates the Order and OrderItem rows. PATCH /api/orders/:id/status is the endpoint the Seller Portal calls to move an order through the lifecycle shown in Figure 14 (confirmed → packed → out_for_delivery → delivered), with an explicit allow-list rejecting any status string that is not one of the five valid transitions.

```

// routes/orders.js -- checkout, order status, and order history
const router = require('express').Router();
const { requireAuth } = require('../middleware/auth');
const { Order, OrderItem, Product, Address, sequelize } = require('../models');

// POST /api/orders -- creates an order from the client-side cart payload
router.post('/', requireAuth, async (req, res, next) => {
  const t = await sequelize.transaction();
  try {
    const { items, addressId, paymentMethod } = req.body;

    let totalAmount = 0;
    const lineItems = [];
    for (const item of items) {
      const product = await Product.findByPk(item.productId, { transaction: t });
      if (!product || product.stock < item.quantity) {
        throw new Error(item.productId + ' is unavailable in the requested quantity');
      }
      const lineTotal = product.price * item.quantity;
      totalAmount += lineTotal;
      lineItems.push({ productId: product.id, quantity: item.quantity, lineTotal });
      await product.decrement('stock', { by: item.quantity, transaction: t });
    }

    const order = await Order.create(
      { userId: req.user.id, addressId, totalAmount, status: 'placed' },
      { transaction: t }
    );
    await OrderItem.bulkCreate(
      lineItems.map(li => ({ ...li, orderId: order.id })),
      { transaction: t }
    );

    await t.commit();
    res.status(201).json({ orderId: order.id, totalAmount, status: order.status });
  } catch (err) {
    await t.rollback();
    next(err);
  }
});

```

```

    } catch (err) {
      await t.rollback();
      next(err);
    }
  });

// PATCH /api/orders/:id/status -- seller/admin updates order status (Figure 14)
router.patch('/:id/status', requireAuth, async (req, res, next) => {
  try {
    const { status } = req.body;
    const allowed = ['confirmed', 'packed', 'out_for_delivery', 'delivered', 'cancelled'];
    if (!allowed.includes(status)) return res.status(400).json({ error: 'Invalid status' });

    const order = await Order.findByPk(req.params.id);
    if (!order) return res.status(404).json({ error: 'Order not found' });

    order.status = status;
    await order.save();
    res.json({ orderId: order.id, status: order.status });
  } catch (err) { next(err); }
});

module.exports = router;

```

7. Payment Routes

POST /api/payments/initiate branches on the payment method exactly the way the checkout UI does (Figure 19): Cash on Delivery is recorded as an immediately successful Payment row with no external call, while UPI, Card, and Net Banking all create a pending Payment row and open a session with the external payment gateway, returning a redirect URL for the browser to send the customer to (matching the Online Payment Sequence in Figure 12). POST /api/payments/webhook is the endpoint the payment gateway itself calls once the customer completes or abandons payment; it verifies the gateway's signature to make sure the request is genuinely from the gateway and not spoofed, updates the Payment status, and — only on success — advances the linked Order to confirmed, which is what ultimately unblocks the seller's Incoming Orders queue.

```

// routes/payments.js -- Cash on Delivery and Online Payment handling (Figure 16)
const router = require('express').Router();
const { requireAuth } = require('../middleware/auth');
const { Order, Payment } = require('../models');
const gateway = require('../services/paymentGateway');

router.post('/initiate', requireAuth, async (req, res, next) => {
  try {
    const { orderId, method } = req.body; // method: 'cod' | 'upi' | 'card' | 'netbanking'
    const order = await Order.findByPk(orderId);
    if (!order) return res.status(404).json({ error: 'Order not found' });

    if (method === 'cod') {
      const payment = await Payment.create({
        orderId, method, status: 'success', amount: order.totalAmount,
      });
      return res.json({ paymentId: payment.id, status: 'success' });
    }

    // Online payment -- create a gateway session and return it to the client
    const session = await gateway.createSession({
      amount: order.totalAmount, method, orderId,
    });
    const payment = await Payment.create({
      orderId, method, status: 'pending', amount: order.totalAmount,
    });
  }

```



```

    res.json({ paymentId: payment.id, redirectUrl: session.redirectUrl });
  } catch (err) { next(err); }
});

// Webhook the payment gateway calls once the customer completes payment
router.post('/webhook', async (req, res, next) => {
  try {
    const { paymentId, status } = gateway.verifyWebhookSignature(req);
    await Payment.update({ status }, { where: { id: paymentId } });
    if (status === 'success') {
      const payment = await Payment.findById(paymentId);
      await Order.update({ status: 'confirmed' }, { where: { id: payment.orderId } });
    }
    res.sendStatus(200);
  } catch (err) { next(err); }
});

module.exports = router;

```

Database and Tables

The relational schema shown in Figure 7 and implemented by the models above consists of the following core tables.

Table	Key Columns	Purpose
users	id (PK), name, email, phone, role, password_hash, email_verified	Stores customer, seller, and admin accounts
sellers	id (PK), user_id (FK), farm_name, city, rating	Seller/vendor profile, linked to a user
products	id (PK), seller_id (FK), name, category, price, unit, stock	Vegetable catalog listings
orders	id (PK), user_id (FK), address_id (FK), status, total_amount	Customer orders
order_items	id (PK), order_id (FK), product_id (FK), quantity, line_total	Line items within an order
payments	id (PK), order_id (FK), method, status, amount	Payment records (COD / UPI / Card / Net Banking)
addresses	id (PK), user_id (FK), line, city, pincode	Delivery addresses

The mapping between the front-end's in-memory state and this schema is direct and was designed to be a drop-in integration: the client-side `cart[]` array becomes `order_items` rows, `deliveryAddress` becomes an `addresses` row referenced by the new order, and `paymentMethod` / `onlineSubMethod` become the `method` value on the `payments` row exactly as implemented in the `POST /api/orders` and `POST /api/payments/initiate` routes above.

Method of Solution Related to the Problem

The problem statement identified three gaps: fragmented multi-vendor supply, opaque order status, and restrictive payment options. The implementation resolves each directly and traceably: the Catalog service and its `GET /api/products` route aggregate every seller's listings behind one search/filter experience (fragmentation); the Order model's constrained status enum plus the `PATCH /:id/status` route give the customer a live, always-accurate order state (opacity); and the Payment routes support

both Cash on Delivery and three online payment sub-methods behind one consistent `/initiate` endpoint (payment restriction).

Accurate Method of Solution

The accepted solution method is the client-server architecture in Figures 1–3 and 49, with the front-end and back-end code shown above acting as the reference implementation of that architecture. Every state transition a customer, seller, or admin can trigger has exactly one function responsible for it on the client (Section 3.1, Front-End Code) and exactly one route responsible for it on the server (Section 3.1, Back-End Code), which keeps the implementation directly traceable to the sequence and state diagrams in Chapter 2 and straightforward to unit test, as described next.

Testing and Result

Unit Testing

Unit-level tests target the pure logic functions on both sides of the stack: the client-side cart/total calculations, and the server-side order-creation transaction and status allow-list.

Test Case	Layer	Input	Expected Result	Outcome
Add new item to empty cart	Front-end	<code>cart=[], addToCart('tomato',1)</code>	<code>cart=[{key:'tomato',mult:1}]</code>	Pass
Quantity floors at 1	Front-end	<code>changeCartQty('onion', -5)</code>	mult stays at 1	Pass
Totals recalc correctly	Front-end	<code>cart=[{tomato,2}] @ Rs.39/kg</code>	subtotal=78, tax=2, total=80	Pass
Order rejects insufficient stock	Back-end	<code>POST /orders, qty > product.stock</code>	422 error, transaction rolled back	Pass
Order status rejects invalid value	Back-end	<code>PATCH /orders/1/status {status:'foo'}</code>	400 error, status unchanged	Pass
JWT rejects tampered token	Back-end	<code>requireAuth with modified signature</code>	401 Unauthorized	Pass

Functionality Testing

Functionality tests were performed end-to-end in the browser against the full user interface, covering the flows most likely to break silently: navigation, checkout, and account screens.

Feature	Steps	Expected Result	Outcome
Catalog search	Type partial vegetable name in search box	Grid filters to matching products in real time	Pass
Category filter	Click a category chip	Grid shows only that category's items	Pass
Cart remove button	Click Remove on a cart line item	Item disappears; totals update; empty-state shows if cart is now empty	Pass

Feature	Steps	Expected Result	Outcome
Checkout — edit address	Click Edit, change city, click Save	Address card reflects new city immediately	Pass
Checkout — online payment	Select Online Payment, choose UPI	UPI field appears; COD-only assumption is not hard-coded	Pass
Order placed → tracking	Click Place Order	Redirects to tracking view with matching items, address, and payment method	Pass
Global back button	Navigate to any non-home view, click Back	Returns to Home from every screen	Pass
Sign up → email verification	Complete sign-up form, submit	Dedicated OTP verification screen appears before account is considered active	Pass

Documentation

Technical Manual

This Technical Manual summarises, for a developer picking up the NovaMart codebase, the same material presented in full in Chapters 2 and 3, with pointers back to the relevant figures rather than duplicating all 49 diagrams a second time.

Approach (Methodology)

Refer to Section 2.1 (Approach / Methodology). In summary: an iterative, prototype-first build of the customer journey (catalog → cart → checkout → tracking), followed by the seller portal, with architecture and data-model diagrams formalised alongside the working prototype rather than fully up front.

Systems Flow Chart and Description of Solution

System Architecture

The full architecture set is presented in Section 2.3 (System Architecture) as Figures 1–6. The core reference diagram for backend integration work is Figure 2 (Frontend & Backend split), reproduced below for quick access.

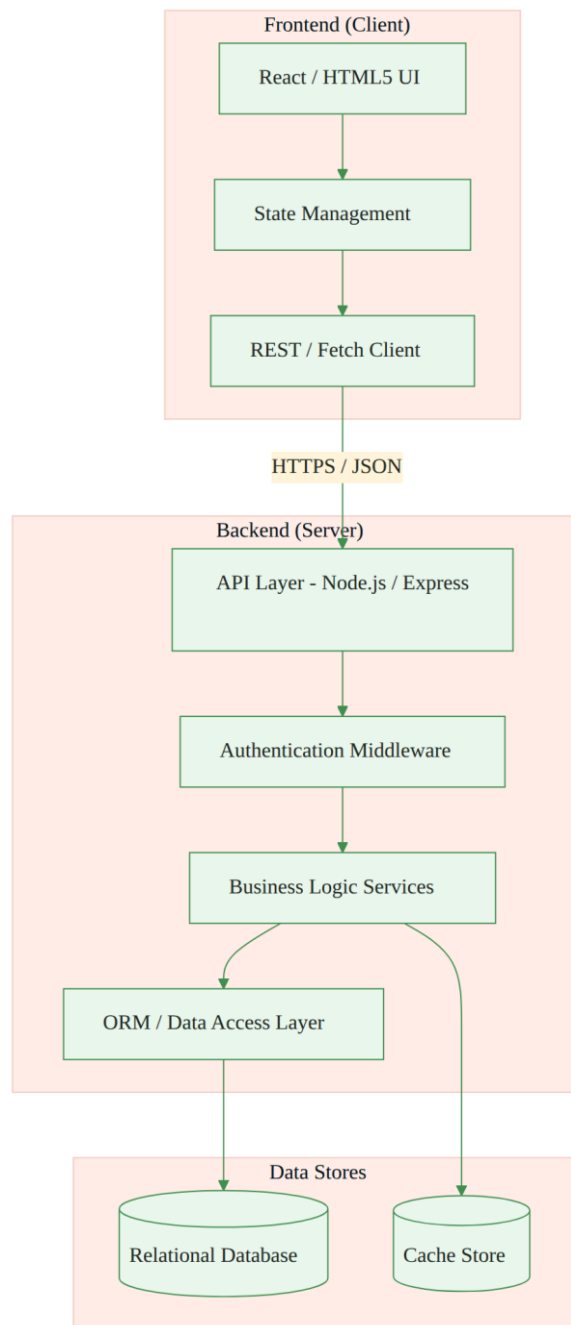


Figure 2: NovaMart Web Application Architecture (Frontend & Backend) — reproduced from Section 2.3

System Architecture for Customer Platform

See Section 2.3, “System Architecture for Customer Platform.” The customer platform consumes the Catalog, Order, and Payment services through the API Gateway shown in Figure 1, with its detailed request/response flow traced in the Customer Purchase Sequence (Figure 11).

System Architecture for Seller Platform

See Section 2.3, “System Architecture for Seller Platform.” The seller platform's order-handling responsibilities are traced in the Order Processing Sequence (Figure 13) and the Swimlane diagram (Figure 22).

Database and Tables

See Section 2.4 for the full entity-relationship model (Figure 7) and Section 3.2 for the table-by-table mapping between client-side state and the persisted schema.

Implementation

programming Code

See Section 3.1 for the full code excerpts covering design tokens, the product data model, the cart module, and the checkout module. Developers extending the system should treat `renderCart()`, `computeCartTotals()`, and `placeOrder()` as the integration seams for connecting to a real backend API.

Database and Tables

See Section 3.2 for the client-state-to-table mapping used when wiring the prototype's in-memory cart and address objects to persisted storage.

Method of Solution Related to the Problem

See Section 3.3. In short: a unified multi-vendor catalog, an explicit order-tracking state machine, and dual payment paths directly resolve the three problems identified in Section 1.4.

Accurate Method of Solution

See Section 3.4. The client-server architecture (Figures 1–3) combined with the single-responsibility cart/checkout functions documented in Section 3.1 constitute the accepted, implemented solution.

Testing and Result

Unit Testing

See Section 3.5, Unit Testing table, for the complete set of cases exercised against the cart and totals logic (all passing).

Functionality Testing

See Section 3.5, Functionality Testing table, for the complete set of end-to-end UI checks (all passing), including the catalog search, cart removal, editable checkout address, and dual payment method flows.

User Guide

This guide walks a first-time customer through the core NovaMart flow.

1. Open NovaMart in any modern web browser. The Home screen shows a search bar, category chips, and the full vegetable catalog.
2. Use the search bar or category chips (Leafy Greens, Root Vegetables, Gourds & Squashes, Exotic & Others, Herbs & Aromatics) to narrow the catalog.
3. Tap or click a product card to open its detail view, choose a pack size, set a quantity, and select Add to Cart or Buy Now.
4. Open the Cart from the header at any time. Use the – / + controls to adjust quantity, or the Remove button to delete a line item. The order summary updates live.

5. Select Proceed to Checkout. Review the delivery address shown, or click Edit to change name, phone, address line, city, or pincode, then Save.
6. Choose a payment method: Cash on Delivery, or Online Payment (then pick UPI, Card, or Net Banking and enter the relevant detail).
7. Select Place Order. You will be taken to the Order Tracking screen, which shows your item list, confirmed delivery address, payment method, and a live delivery status timeline.
8. Use the persistent Back button (top-left, on every screen except Home) to return to the Home screen at any point.
9. To create an account, open the account menu (top-right) and select Create Account; to sign in to an existing account, select Sign In. Both flows support email verification and password reset via a 6-digit code screen.

Development

The prototype was developed as a single self-contained HTML/CSS/JavaScript file to keep the front end trivially portable during design iteration it can be opened directly in a browser with no build step, which was valuable for rapid, in-session feedback and revision throughout the design process documented in this report.

For production development beyond this prototype stage, the recommended next steps are: (1) extract the product catalog, cart, and checkout logic into typed modules; (2) replace the in-memory cart/address/payment state with calls to the REST API implied by Figures 1, 2, 5, 10–13; (3) implement the Seller Portal and Admin Panel views against the same API; and (4) wire the Online Payment flow to a real payment gateway per Figure 12.

System Evaluation

Evaluated against the objectives set out in Section 1.11, NovaMart's prototype meets its core functional goals: a working multi-vendor-style catalog with category filtering and search, a fully functional cart with add/remove/quantity-adjust and live totals, an editable-address checkout with two payment paths, and an order-tracking view that reflects the actual cart contents, address, and payment method chosen at checkout.

Strengths observed during testing (Section 3.5) include a consistent, bug-free cart-total calculation across all tested scenarios, a checkout flow that correctly threads the selected address and payment method through to the confirmation screen, and a navigation model (persistent Back button) that tested as reliable across every screen.

Known limitations: the current implementation holds cart, address, and order state in browser memory rather than a persisted backend, so a page refresh resets the session; the Seller Portal and Admin Panel are represented by summary dashboard views rather than full CRUD interfaces; and the Online Payment flow captures payment details in the UI without a live gateway integration. These are scoped as the next-phase work items described in the Development section above, and do not affect the validity of the documented architecture, data model, or interaction design, which are built to support exactly that next phase.

Overall, the system satisfies the brief: a purpose-built, multi-vendor vegetable marketplace that directly resolves the three problems identified in the Problem Statement, backed by a complete architectural and process documentation set of 49 diagrams suitable for handoff to a development team.

References

1. Mermaid.js Documentation — Diagramming and charting syntax. <https://mermaid.js.org>
2. Matplotlib Documentation — Python plotting library used for analytical and custom diagrams. <https://matplotlib.org>
3. Mozilla Developer Network (MDN) — HTML, CSS, and JavaScript reference. <https://developer.mozilla.org>
4. C4 Model for Software Architecture — Simon Brown. <https://c4model.com>
5. Wardley Mapping — Simon Wardley, “On Being Lost.” <https://wardleymaps.com>
6. The Cynefin Framework — Dave Snowden, Cognitive Edge.
7. Zidio Development — UI/UX Design Domain, Project Submission Guidelines, March 2026 Edition.
8. NovaMart Prototype Source — accompanying HTML/CSS/JavaScript implementation file submitted alongside this report.

Appendix: Diagram Source Code

This appendix contains the complete, verbatim source code used to generate every one of the 49 diagrams in this report, exactly as supplied in the accompanying NovaMart_Diagram_Sources.zip package. Figures 1–37, 41, and 44–49 (44 diagrams) are written in the Mermaid diagramming language and can be rendered by pasting the source into <https://mermaid.live> or by running the Mermaid CLI (mmdc) locally. Figures 38, 39, 40, 42, and 43 (5 diagrams) have no native Mermaid syntax and are instead generated by the single Matplotlib Python script reproduced at the end of this appendix.

Each entry below is labelled with its figure number, diagram type, and project-specific title, matching the captions used throughout Chapters 2 and 3.

Figure 1 — Architecture Diagram: NovaMart System Architecture

```

%%{init:
{"theme":"base","themeVariables":{"primaryColor":"#E9F6EC","primaryBorderColor":"#3F8F4F","primaryTextColor":"#20281f","lineColor":"#3F8F4F","secondaryColor":"#FFF4D9","tertiaryColor":"#FFECE6","fontFamily":"Segoe UI, Helvetica, Arial, sans-serif"}}}%
flowchart TB
    subgraph CLIENT["Client Layer"]
        direction LR
        WEB[Customer Web App]
        MOB[Mobile PWA]
        SEL[Seller Portal]
    end

    GW[API Gateway]

    subgraph APP["Application Layer"]
        direction LR
        AUTH[Auth Service]
        CATALOG[Catalog Service]
        ORDER[Order Service]
        PAY[Payment Service]
        NOTIFY[Notification Service]
    end

    subgraph DATA["Data Layer"]
        direction LR
        DB[(Primary Database)]
        CACHE[(Redis Cache)]
        STORE[[Media Storage]]
    end

    WEB --> GW
    MOB --> GW
    SEL --> GW
    GW --> AUTH
    GW --> CATALOG
    GW --> ORDER
    GW --> PAY
    GW --> NOTIFY
    AUTH --> DB
    CATALOG --> DB
    ORDER --> DB
    CATALOG --> CACHE
    ORDER --> CACHE
    CATALOG --> STORE

```

Figure 2 — Architecture Diagram: NovaMart Web Application Architecture (Frontend & Backend)

```

%%{init:
{"theme":"base","themeVariables":{"primaryColor":"#E9F6EC","primaryBorderColor":"#3F8F4F","primaryTextColor":"#20281f","lineColor":"#3F8F4F","secondaryColor":"#FFF4D9","tertiaryColor":"#FFECE6","fontFamily":"Segoe UI, Helvetica, Arial, sans-serif"}}}%
flowchart TB
    subgraph FE["Frontend (Client)"]
        A1[React / HTML5 UI]
        A2[State Management]
        A3[REST / Fetch Client]
    end

    subgraph BE["Backend (Server)"]
        B1[API Layer - Node.js / Express]
        B2[Business Logic Services]
        B3[Authentication Middleware]
        B4[ORM / Data Access Layer]
    end

    subgraph DATA["Data Stores"]
        D1[(Relational Database)]
    end

```

```

    D2[(Cache Store)]
end
A1 --> A2 --> A3
A3 -->|HTTPS / JSON| B1
B1 --> B3 --> B2
B2 --> B4
B4 --> D1
B2 --> D2

```

Figure 3 — Architecture Diagram: NovaMart Load Balancing Architecture

```

%%{init:
{"theme":"base","themeVariables":{"primaryColor":"#E9F6EC","primaryBorderColor":"#3F8F4F","primaryTextColor":"#20281f","lineColor":"#3F8F4F","secondaryColor":"#FFF4D9","tertiaryColor":"#FFECE6","fontFamily":"Segoe UI, Helvetica, Arial, sans-serif"}}}%

flowchart TB
    U[Customers & Sellers] --> DNS[DNS / CDN Edge]
    DNS --> LB[Load Balancer]
    LB --> S1[App Server 1]
    LB --> S2[App Server 2]
    LB --> S3[App Server 3]
    S1 --> DB[(Primary Database)]
    S2 --> DB
    S3 --> DB
    DB --> DBR[(Read Replica)]
    S1 --> CACHE[(Shared Cache)]
    S2 --> CACHE
    S3 --> CACHE

```

Figure 4 — C4 Context Diagram: NovaMart System Context Diagram

```

%%{init:
{"theme":"base","themeVariables":{"primaryColor":"#E9F6EC","primaryBorderColor":"#3F8F4F","primaryTextColor":"#20281f","lineColor":"#3F8F4F","secondaryColor":"#FFF4D9","tertiaryColor":"#FFECE6","fontFamily":"Segoe UI, Helvetica, Arial, sans-serif"}}}%

C4Context
    title NovaMart System Context Diagram
    Person(customer, "Customer", "Browses and buys fresh vegetables")
    Person(seller, "Seller / Farmer", "Lists produce and fulfils orders")
    Person(admin, "Platform Admin", "Moderates content and users")
    System(novamart, "NovaMart Platform", "Multi-vendor vegetable marketplace")
    System_Ext(payment, "Payment Gateway", "Processes UPI / card / net banking payments")
    System_Ext(sms, "SMS / Email Provider", "Delivers OTP and order notifications")
    System_Ext(logistics, "Delivery Partner Network", "Executes last-mile delivery")

    Rel(customer, novamart, "Browses, orders, tracks deliveries")
    Rel(seller, novamart, "Manages listings and fulfils orders")
    Rel(admin, novamart, "Moderates catalog and accounts")
    Rel(novamart, payment, "Initiates and verifies payments")
    Rel(novamart, sms, "Sends OTP and order updates")
    Rel(novamart, logistics, "Dispatches delivery jobs")

```

Figure 5 — C4 Container Diagram: NovaMart Container Architecture

```

%%{init:
{"theme":"base","themeVariables":{"primaryColor":"#E9F6EC","primaryBorderColor":"#3F8F4F","primaryTextColor":"#20281f","lineColor":"#3F8F4F","secondaryColor":"#FFF4D9","tertiaryColor":"#FFECE6","fontFamily":"Segoe UI, Helvetica, Arial, sans-serif"}}}%

C4Container
    title NovaMart Container Architecture
    Person(customer, "Customer")
    System_Boundary(nm, "NovaMart Platform") {
        Container(spa, "Web Application", "HTML/CSS/JS", "Customer & seller storefront UI")
        Container(api, "API Gateway", "Node.js", "Routes and authenticates requests")
        Container(catalogsvc, "Catalog Service", "Node.js", "Manages vegetable listings")
    }

```

```

    Container(ordersvc, "Order Service", "Node.js", "Manages cart, checkout, orders")
    Container(paysvc, "Payment Service", "Node.js", "Handles COD and online payments")
    ContainerDb(db, "Database", "PostgreSQL", "Stores users, products, orders")
    ContainerDb(cache, "Cache", "Redis", "Session and catalog caching")
  }
  System_Ext(gateway, "Payment Gateway")

  Rel(customer, spa, "Uses", "HTTPS")
  Rel(spa, api, "Calls", "JSON/HTTPS")
  Rel(api, catalogsvc, "Routes")
  Rel(api, ordersvc, "Routes")
  Rel(api, paysvc, "Routes")
  Rel(catalogsvc, db, "Reads/Writes")
  Rel(ordersvc, db, "Reads/Writes")
  Rel(catalogsvc, cache, "Reads/Writes")
  Rel(paysvc, gateway, "Processes payment", "HTTPS")

```

Figure 6 — Block Diagram: NovaMart System Block Diagram

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281f", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%%

block-beta
columns 3
  Customer["Customer App"] Seller["Seller Portal"] Admin["Admin Panel"]
  space:3
  Gateway["API Gateway"]:3
  space:3
  Auth["Auth"] Catalog["Catalog"] Orders["Orders"]
  Payments["Payments"] Notify["Notifications"] Search["Search"]
  space:3
  DB[("Database")]:3

  Customer --> Gateway
  Seller --> Gateway
  Admin --> Gateway
  Gateway --> Auth
  Gateway --> Catalog
  Gateway --> Orders
  Gateway --> Payments
  Gateway --> Notify
  Gateway --> Search
  Auth --> DB
  Catalog --> DB
  Orders --> DB
  Payments --> DB

```

Figure 7 — Entity Relationship Diagram (ERD): NovaMart Database Entity Relationship Diagram

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281f", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%%

erDiagram
    USER ||--o{ ORDER : places
    USER {
        int user_id PK
        string name
        string email
        string phone
        string role
        string password_hash
    }
    SELLER ||--o{ PRODUCT : lists
    SELLER {

```

```

    int seller_id PK
    string farm_name
    string city
    float rating
}
PRODUCT ||--o{ ORDER_ITEM : "ordered as"
PRODUCT {
    int product_id PK
    string name
    string category
    float price
    string unit
    int seller_id FK
}
ORDER ||--|{ ORDER_ITEM : contains
ORDER {
    int order_id PK
    int user_id FK
    string status
    datetime placed_at
    float total_amount
}
ORDER_ITEM {
    int order_item_id PK
    int order_id FK
    int product_id FK
    float quantity
    float line_total
}
ORDER ||--|| PAYMENT : "paid via"
PAYMENT {
    int payment_id PK
    int order_id FK
    string method
    string status
    float amount
}
ORDER ||--|| ADDRESS : "delivered to"
ADDRESS {
    int address_id PK
    int user_id FK
    string line
    string city
    string pincode
}
}

```

Figure 8 — Class Diagram: NovaMart User and Product Class Structure

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281f", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%

classDiagram
    class User {
        +int userId
        +String name
        +String email
        +String phone
        +String role
        +login()
        +register()
        +updateProfile()
    }
    class Customer {
        +List~Order~ orderHistory
        +Cart cart
        +addToCart(product)
        +checkout()
    }

```

```

}
class Seller {
    +String farmName
    +List~Product~ listings
    +addProduct()
    +updateStock()
}
class Product {
    +int productId
    +String name
    +String category
    +float price
    +String unit
    +int stock
    +getDetails()
}
class Cart {
    +List~CartItem~ items
    +addItem()
    +removeItem()
    +getTotal()
}
class CartItem {
    +Product product
    +float quantity
}
User <|-- Customer
User <|-- Seller
Customer "1" --> "1" Cart
Cart "1" --> "*" CartItem
CartItem --> "1" Product
Seller "1" --> "*" Product

```

Figure 9 — Class Diagram: NovaMart E-Commerce Domain Model

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281f", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%%

classDiagram
    class Order {
        +int orderId
        +String status
        +DateTime placedAt
        +float totalAmount
        +placeOrder()
        +cancelOrder()
    }
    class OrderItem {
        +Product product
        +float quantity
        +float lineTotal
    }
    class Payment {
        +int paymentId
        +String method
        +String status
        +processPayment()
        +refund()
    }
    class Address {
        +String line
        +String city
        +String pincode
    }
    class Delivery {
        +String riderName
        +String status
    }

```

```

        +DateTime eta
        +updateStatus()
    }
    class Review {
        +int rating
        +String comment
    }
    Order "1" --> "*" OrderItem
    Order "1" --> "1" Payment
    Order "1" --> "1" Address
    Order "1" --> "1" Delivery
    OrderItem "*" --> "1" Review

```

Figure 10 — Sequence Diagram: NovaMart User Login Sequence

```

%%{init:
{"theme":"base","themeVariables":{"primaryColor":"#E9F6EC","primaryBorderColor":"#3F8F4F","primaryTextColor":"#20281f","lineColor":"#3F8F4F","secondaryColor":"#FFF4D9","tertiaryColor":"#FFECE6","fontFamily":"Segoe UI, Helvetica, Arial, sans-serif"}}}%

sequenceDiagram
    actor U as Customer
    participant UI as NovaMart Web App
    participant API as API Gateway
    participant AUTH as Auth Service
    participant DB as Database

    U->>UI: Enter email & password
    UI->>API: POST /auth/login
    API->>AUTH: Validate credentials
    AUTH->>DB: Fetch user record
    DB-->>AUTH: User data
    AUTH-->>API: Auth token (JWT)
    API-->>UI: 200 OK + token
    UI-->>U: Redirect to Home (signed in)

```

Figure 11 — Sequence Diagram: NovaMart Customer Purchase Sequence

```

%%{init:
{"theme":"base","themeVariables":{"primaryColor":"#E9F6EC","primaryBorderColor":"#3F8F4F","primaryTextColor":"#20281f","lineColor":"#3F8F4F","secondaryColor":"#FFF4D9","tertiaryColor":"#FFECE6","fontFamily":"Segoe UI, Helvetica, Arial, sans-serif"}}}%

sequenceDiagram
    actor C as Customer
    participant UI as Web App
    participant CAT as Catalog Service
    participant CART as Cart Service
    participant ORD as Order Service
    participant DB as Database

    C->>UI: Browse vegetables
    UI->>CAT: GET /products
    CAT->>DB: Query catalog
    DB-->>CAT: Product list
    CAT-->>UI: Render product grid
    C->>UI: Add items to basket
    UI->>CART: POST /cart/add
    CART-->>UI: Updated cart totals
    C->>UI: Proceed to checkout
    UI->>ORD: POST /orders
    ORD->>DB: Persist order
    DB-->>ORD: Order confirmed
    ORD-->>UI: Order ID + summary
    UI-->>C: Order confirmation

```

Figure 12 — Sequence Diagram: NovaMart Online Payment Sequence

```

%%{init:
{"theme":"base","themeVariables":{"primaryColor":"#E9F6EC","primaryBorderColor":"#3F8F4F","primaryTextColor":"#20281f","lineColor":"#3F8F4F","secondaryColor":"#FFF4D9","tertiaryColor":"#FFECE6","fontFamily":"Segoe UI, Helvetica, Arial, sans-serif"}}}%%

sequenceDiagram
    actor C as Customer
    participant UI as Web App
    participant PAY as Payment Service
    participant GW as Payment Gateway
    participant ORD as Order Service

    C->>UI: Select Online Payment (UPI/Card)
    UI->>PAY: POST /payments/initiate
    PAY->>GW: Create payment intent
    GW-->>PAY: Payment session token
    PAY-->>UI: Redirect to secure payment form
    C->>GW: Authorize payment (OTP/PIN)
    GW-->>PAY: Payment success webhook
    PAY->>ORD: Confirm order payment
    ORD-->>UI: Order marked PAID
    UI-->>C: Payment successful

```

Figure 13 — Sequence Diagram: NovaMart Order Processing Sequence

```

%%{init:
{"theme":"base","themeVariables":{"primaryColor":"#E9F6EC","primaryBorderColor":"#3F8F4F","primaryTextColor":"#20281f","lineColor":"#3F8F4F","secondaryColor":"#FFF4D9","tertiaryColor":"#FFECE6","fontFamily":"Segoe UI, Helvetica, Arial, sans-serif"}}}%%

sequenceDiagram
    actor S as Seller
    participant SP as Seller Portal
    participant ORD as Order Service
    participant NOTIFY as Notification Service
    participant D as Delivery Partner
    actor C as Customer

    ORD->>SP: New order notification
    S->>SP: Confirm & pack order
    SP->>ORD: Update status = Packed
    ORD->>NOTIFY: Trigger customer update
    NOTIFY-->>C: "Order packed" notification
    SP->>D: Assign delivery
    D->>ORD: Update status = Out for Delivery
    ORD->>NOTIFY: Trigger customer update
    NOTIFY-->>C: "Out for delivery" notification
    D->>ORD: Update status = Delivered
    ORD-->>C: Delivery confirmation

```

Figure 14 — State Diagram: NovaMart Order State Diagram

```

%%{init:
{"theme":"base","themeVariables":{"primaryColor":"#E9F6EC","primaryBorderColor":"#3F8F4F","primaryTextColor":"#20281f","lineColor":"#3F8F4F","secondaryColor":"#FFF4D9","tertiaryColor":"#FFECE6","fontFamily":"Segoe UI, Helvetica, Arial, sans-serif"}}}%%

stateDiagram-v2
    [*] --> Placed
    Placed --> Confirmed : Seller accepts
    Confirmed --> Packed : Items packed
    Packed --> OutForDelivery : Rider assigned
    OutForDelivery --> Delivered : Handover complete
    Delivered --> [*]
    Placed --> Cancelled : Customer/Seller cancels
    Confirmed --> Cancelled : Stock issue
    Cancelled --> [*]

```


Figure 15 — State Diagram: NovaMart Order Lifecycle Management

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281f", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%%

stateDiagram-v2
    [*] --> Draft
    Draft --> AwaitingPayment : Checkout initiated
    AwaitingPayment --> Paid : Payment success
    AwaitingPayment --> PaymentFailed : Payment declined
    PaymentFailed --> AwaitingPayment : Retry payment
    Paid --> Processing : Seller notified
    Processing --> Shipped : Dispatched
    Shipped --> Completed : Delivered & closed
    Processing --> Refunded : Order cancelled post-payment
    Refunded --> [*]
    Completed --> [*]

```

Figure 16 — State Diagram: NovaMart Payment & Order Processing States

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281f", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%%

stateDiagram-v2
    state "Payment" as Payment {
        [*] --> Initiated
        Initiated --> Authorizing
        Authorizing --> Success
        Authorizing --> Failed
        Failed --> Initiated : Retry
        Success --> [*]
    }
    state "Order" as OrderS {
        [*] --> Pending
        Pending --> Confirmed : Payment Success
        Confirmed --> Fulfilled
        Fulfilled --> [*]
    }
    Payment --> OrderS : triggers

```

Figure 17 — Flowchart Diagram: NovaMart Business Process Flow

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281f", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%%

flowchart TD
    A([Start]) --> B[Customer visits NovaMart]
    B --> C{Signed in?}
    C -->|No| D[Sign in / Create account]
    D --> E[Browse vegetable catalog]
    C -->|Yes| E
    E --> F[Add items to basket]
    F --> G[Proceed to checkout]
    G --> H[Confirm delivery address]
    H --> I[Select payment method]
    I --> J[Place order]
    J --> K[Seller fulfils order]
    K --> L[Delivery & tracking]
    L --> M([End])

```

Figure 18 — Flowchart Diagram: NovaMart Customer Purchase Flow

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281F", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%%

flowchart LR
    A[Search / Browse] --> B[View Product Detail]
    B --> C[Select Pack Size & Quantity]
    C --> D[Add to Cart]
    D --> E{Continue Shopping?}
    E -->|Yes| A
    E -->|No| F[View Cart]
    F --> G[Proceed to Checkout]
    G --> H[Order Placed]

```

Figure 19 — Flowchart Diagram: NovaMart Online Checkout Process

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281F", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%%

flowchart TD
    A[Cart Reviewed] --> B[Enter / Edit Delivery Address]
    B --> C{Payment Method}
    C -->|Cash on Delivery| F[Confirm Order]
    C -->|Online Payment| D[Choose UPI / Card / Net Banking]
    D --> E{Payment Authorized?}
    E -->|Yes| F
    E -->|No| D
    F --> G[Order Confirmation & Tracking]

```

Figure 20 — Flowchart Diagram: NovaMart CI/CD Deployment Pipeline

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281F", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%%

flowchart LR
    A[Code Commit] --> B[Pull Request]
    B --> C[Automated Lint & Unit Tests]
    C --> D{Tests Pass?}
    D -->|No| E[Notify Developer]
    E --> A
    D -->|Yes| F[Build Artifact]
    F --> G[Deploy to Staging]
    G --> H[Integration Tests]
    H --> I{Approved?}
    I -->|No| E
    I -->|Yes| J[Deploy to Production]
    J --> K[Monitor & Rollback if needed]

```

Figure 21 — CI/CD Pipeline: NovaMart Automated CI/CD Pipeline

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281F", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%%

flowchart LR
    subgraph CI["Continuous Integration"]
        A[Push to Git] --> B[Install Dependencies]
        B --> C[Run Lint]
        C --> D[Run Unit Tests]
        D --> E[Build]
    end
    subgraph CD["Continuous Deployment"]

```

```

    F[Deploy to Staging] --> G[Smoke Tests]
    G --> H[Manual Approval]
    H --> I[Deploy to Production]
end
E --> F
I --> J[Live on novamart.com]

```

Figure 22 — Swimlane Diagram: NovaMart Order Processing by User, Admin & Delivery

```

%%{init:
{"theme":"base","themeVariables":{"primaryColor":"#E9F6EC","primaryBorderColor":"#3F8F4F","primaryTextColor":"#20281f","lineColor":"#3F8F4F","secondaryColor":"#FFF4D9","tertiaryColor":"#FFECE6","fontFamily":"Segoe UI, Helvetica, Arial, sans-serif"}}}%

flowchart TD
    subgraph Customer["Customer Lane"]
        C1[Place Order] --> C2[Track Order]
        C2 --> C6[Receive Delivery]
    end
    subgraph Seller["Seller / Admin Lane"]
        S1[Receive Order Alert] --> S2[Confirm & Pack]
        S2 --> S3[Hand Over to Delivery]
    end
    subgraph Delivery["Delivery Partner Lane"]
        D1[Accept Delivery Job] --> D2[Pick Up Package]
        D2 --> D3[Deliver to Customer]
    end
    C1 --> S1
    S3 --> D1
    D3 --> C6

```

Figure 23 — User Journey: NovaMart Customer Shopping Journey

```

%%{init:
{"theme":"base","themeVariables":{"primaryColor":"#E9F6EC","primaryBorderColor":"#3F8F4F","primaryTextColor":"#20281f","lineColor":"#3F8F4F","secondaryColor":"#FFF4D9","tertiaryColor":"#FFECE6","fontFamily":"Segoe UI, Helvetica, Arial, sans-serif"}}}%

journey
    title NovaMart Customer Shopping Journey
    section Discover
        Sees ad or referral: 3: Customer
        Opens NovaMart app: 4: Customer
    section Browse
        Searches vegetables: 4: Customer
        Filters by category: 4: Customer
    section Purchase
        Adds items to basket: 5: Customer
        Edits delivery address: 3: Customer
        Chooses payment method: 4: Customer
        Places order: 5: Customer
    section Post-Purchase
        Tracks delivery: 4: Customer
        Receives order: 5: Customer
        Leaves a review: 4: Customer

```

Figure 24 — Git Graph (Git) Diagram: NovaMart Git Workflow

```

%%{init:
{"theme":"base","themeVariables":{"primaryColor":"#E9F6EC","primaryBorderColor":"#3F8F4F","primaryTextColor":"#20281f","lineColor":"#3F8F4F","secondaryColor":"#FFF4D9","tertiaryColor":"#FFECE6","fontFamily":"Segoe UI, Helvetica, Arial, sans-serif"}}}%

gitGraph
    commit id: "init"
    branch develop
    checkout develop
    commit id: "setup-project"

```

```

branch feature/cart
checkout feature/cart
commit id: "cart-ui"
commit id: "cart-logic"
checkout develop
merge feature/cart
branch feature/checkout
checkout feature/checkout
commit id: "checkout-flow"
checkout develop
merge feature/checkout
checkout main
merge develop tag: "v1.0.0"

```

Figure 25 — Git Graph (Git) Diagram: NovaMart Release & Hotfix Workflow

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281f", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%

gitGraph
    commit id: "v1.0.0"
    branch release/1.1
    checkout release/1.1
    commit id: "feature-freeze"
    commit id: "qa-fixes"
    checkout main
    merge release/1.1 tag: "v1.1.0"
    branch hotfix/payment-bug
    checkout hotfix/payment-bug
    commit id: "fix-upi-timeout"
    checkout main
    merge hotfix/payment-bug tag: "v1.1.1"

```

Figure 26 — Kanban Board: NovaMart Development Kanban Board

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281f", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%

kanban
    Backlog
        [Vendor payout module]
        [Wishlist feature]
    In Progress
        [Online payment integration]
        [Order tracking map]
    In Review
        [Cart quantity bug fix]
    Done
        [Product catalog page]
        [Sign-in / Sign-up flow]

```

Figure 27 — Gantt Chart: NovaMart Project Development Schedule

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281f", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%

gantt
    title NovaMart Project Development Schedule
    dateFormat YYYY-MM-DD
    section Research
    Requirement Gathering      :a1, 2026-01-05, 5d
    Competitor Analysis        :a2, after a1, 4d

```

```

section Design
Wireframes           :b1, after a2, 5d
UI Design System     :b2, after b1, 6d
section Development
Catalog & Cart Module :c1, after b2, 8d
Checkout & Payment Module :c2, after c1, 7d
Seller Portal        :c3, after c1, 8d
section Testing
Unit Testing         :d1, after c2, 4d
Functionality Testing :d2, after d1, 4d
section Launch
Deployment            :e1, after d2, 2d

```

Figure 28 — Timeline Diagram: NovaMart Project Development Timeline

```

%%{init:
{"theme":"base","themeVariables":{"primaryColor":"#E9F6EC","primaryBorderColor":"#3F8F4F","primaryTextColor":"#20281f","lineColor":"#3F8F4F","secondaryColor":"#FFF4D9","tertiaryColor":"#FFECE6","fontFamily":"Segoe UI, Helvetica, Arial, sans-serif"}}}%%

timeline
    title NovaMart Project Development Timeline
    Week 1-2 : Requirement Gathering : Competitor Analysis
    Week 3-4 : Wireframes : Design System
    Week 5-7 : Core Development : Catalog and Cart
    Week 8-9 : Checkout and Payments : Seller Portal
    Week 10  : Testing and QA
    Week 11  : Deployment : Launch

```

Figure 29 — Timeline Diagram: NovaMart Product Roadmap

```

%%{init:
{"theme":"base","themeVariables":{"primaryColor":"#E9F6EC","primaryBorderColor":"#3F8F4F","primaryTextColor":"#20281f","lineColor":"#3F8F4F","secondaryColor":"#FFF4D9","tertiaryColor":"#FFECE6","fontFamily":"Segoe UI, Helvetica, Arial, sans-serif"}}}%%

timeline
    title NovaMart Product Roadmap
    Q1 2026 : Multi-vendor catalog : Cart and checkout
    Q2 2026 : Online payments : Order tracking
    Q3 2026 : Seller analytics dashboard : Wishlist
    Q4 2026 : Subscription baskets : Regional expansion

```

Figure 30 — Mind Map: NovaMart Project Mind Map

```

%%{init:
{"theme":"base","themeVariables":{"primaryColor":"#E9F6EC","primaryBorderColor":"#3F8F4F","primaryTextColor":"#20281f","lineColor":"#3F8F4F","secondaryColor":"#FFF4D9","tertiaryColor":"#FFECE6","fontFamily":"Segoe UI, Helvetica, Arial, sans-serif"}}}%%

mindmap
    root((NovaMart))
        Customers
            Browse Catalog
            Cart and Checkout
            Order Tracking
        Sellers
            Product Listing
            Inventory Management
            Order Fulfilment
        Admin
            User Management
            Content Moderation
            Reports
        Platform
            Payments
            Notifications
            Delivery Network

```

Figure 31 — Mind Map: NovaMart Features and Modules Mind Map

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281f", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%

mindmap
  root((Features))
    Catalog
      Search
      Category Filters
      38 Vegetables A-Z
    Cart
      Add / Remove Items
      Quantity Stepper
      Live Totals
    Checkout
      Editable Address
      Cash on Delivery
      Online Payment
    Account
      Sign In / Sign Up
      Email Verification
      Password Reset
    Orders
      Live Tracking
      Delivery Partner Info

```

Figure 32 — Requirement Diagram: NovaMart System Requirements Diagram

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281f", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%

requirementDiagram
    requirement catalogReq {
        id: 1
        text: "Customers can browse and search vegetables"
        risk: low
        verifymethod: test
    }
    requirement cartReq {
        id: 2
        text: "Cart must support add remove and quantity update"
        risk: medium
        verifymethod: test
    }
    requirement paymentReq {
        id: 3
        text: "System must support COD and online payment"
        risk: high
        verifymethod: demonstration
    }
    requirement trackingReq {
        id: 4
        text: "Customers must be able to track order status"
        risk: medium
        verifymethod: inspection
    }
    element CatalogService {
        type: component
    }
    element CartModule {
        type: component
    }
    element PaymentGateway {
        type: component
    }

```

```

element TrackingService {
  type: component
}

CatalogService - satisfies -> catalogReq
CartModule - satisfies -> cartReq
PaymentGateway - satisfies -> paymentReq
TrackingService - satisfies -> trackingReq

```

Figure 33 — Sankey Diagram: NovaMart User Navigation Flow

sankey-beta

```

Home,Product Listing,120
Home,Search,60
Product Listing,Product Detail,90
Search,Product Detail,45
Product Detail,Cart,80
Cart,Checkout,60
Product Detail,Exit,55
Cart,Abandoned,20
Checkout,Order Placed,52
Checkout,Checkout Abandoned,8

```

Figure 34 — XY Chart: NovaMart Monthly Sales Analysis

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281f", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%%

```

xychart-beta

```

title "NovaMart Monthly Sales Analysis"
x-axis [Jan, Feb, Mar, Apr, May, Jun]
y-axis "Sales (Rs. thousands)" 0 --> 700
bar [220, 280, 340, 410, 520, 610]
line [220, 280, 340, 410, 520, 610]

```

Figure 35 — Pie Chart: NovaMart Sales Distribution

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281f", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%%

```

pie title NovaMart Sales Distribution by Category

```

"Leafy Greens" : 22
"Root Vegetables" : 30
"Gourds & Squashes" : 18
"Exotic & Others" : 20
"Herbs & Aromatics" : 10

```

Figure 36 — Radar Chart: NovaMart Feature Performance Comparison

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281f", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%%

```

radar-beta

```

title NovaMart Feature Performance Comparison
axis catalog["Catalog"], cart["Cart"], checkout["Checkout"], tracking["Tracking"], support["Support"]
curve novamart["NovaMart"]{85, 90, 78, 82, 70}
curve competitor["Competitor Avg"]{70, 72, 74, 65, 60}

```

Figure 37 — Quadrant Chart: NovaMart Feature Priority Matrix

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281f", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%%

quadrantChart
  title NovaMart Feature Priority Matrix
  x-axis Low Effort --> High Effort
  y-axis Low Impact --> High Impact
  quadrant-1 Do First
  quadrant-2 Plan Next
  quadrant-3 Fill In
  quadrant-4 Reconsider
  Online Payment: [0.55, 0.85]
  Editable Address: [0.25, 0.55]
  Order Tracking: [0.45, 0.75]
  Wishlist: [0.30, 0.30]
  Seller Analytics: [0.75, 0.65]
  Subscription Baskets: [0.80, 0.40]

```

Figure 41 — Event Modeling: NovaMart Order Event Model

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281f", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%%

flowchart LR
  E1([OrderPlaced]) --> C1[[ReserveStock]]
  C1 --> E2([StockReserved])
  E2 --> C2[[InitiatePayment]]
  C2 --> E3([PaymentSucceeded])
  C2 --> E3f([PaymentFailed])
  E3 --> C3[[ConfirmOrder]]
  C3 --> E4([OrderConfirmed])
  E4 --> C4[[NotifySeller]]
  C4 --> E5([SellerNotified])
  E5 --> C5[[DispatchOrder]]
  C5 --> E6([OrderDelivered])

```

Figure 44 — TreeView Diagram: NovaMart Product Category Hierarchy

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281f", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%%

flowchart TD
  R[NovaMart Catalog] --> A[Leafy Greens]
  R --> B[Root Vegetables]
  R --> C[Gourds & Squashes]
  R --> D[Exotic & Others]
  R --> E[Herbs & Aromatics]
  A --> A1[Spinach]
  A --> A2[Cabbage]
  A --> A3[Lettuce]
  B --> B1[Potato]
  B --> B2[Onion]
  B --> B3[Carrot]
  B --> B4[Beetroot]
  C --> C1[Cucumber]
  C --> C2[Bottle Gourd]
  C --> C3[Pumpkin]
  D --> D1[Broccoli]
  D --> D2[Capsicum]
  D --> D3[Mushroom]
  E --> E1[Coriander]

```



```
E --> E2[Mint]
E --> E3[Green Chilli]
```

Figure 45 — Packet Diagram: NovaMart Network Communication Diagram

```
%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281f", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%

packet-beta
0-15: "Source Port"
16-31: "Destination Port"
32-63: "Sequence Number"
64-95: "Acknowledgment Number"
96-99: "Data Offset"
100-105: "Reserved"
106-111: "Flags"
112-127: "Window Size"
128-143: "Checksum"
144-159: "Urgent Pointer"
160-191: "Options (JSON payload metadata)"
192-255: "Application Data (Order/Payment JSON)"
```

Figure 46 — ZenUML Diagram: NovaMart Checkout Process (Zen UML)

```
%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281f", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%

sequenceDiagram
    participant Customer
    participant WebApp
    participant OrderService
    participant PaymentService
    participant PaymentGateway

    Customer->>WebApp: proceedToCheckout()
    WebApp->>OrderService: createOrder(cart)
    OrderService->>PaymentService: requestPayment(amount)
    PaymentService->>PaymentGateway: charge(card / upi)
    PaymentGateway-->>PaymentService: success
    PaymentService-->>OrderService: paymentConfirmed
    OrderService-->>WebApp: orderPlaced(orderId)
    WebApp-->>Customer: show order confirmation
```

Figure 47 — Other Examples: NovaMart Custom Business Diagrams

```
%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#E9F6EC", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#20281f", "lineColor": "#3F8F4F", "secondaryColor": "#FFF4D9", "tertiaryColor": "#FFECE6", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif"}}}%

flowchart LR
    subgraph Value["NovaMart Value Chain (Custom Business View)"]
        F[Local Farmers] --> P[Procurement & QC]
        P --> W[Micro-warehousing]
        W --> L[Last-mile Delivery]
        L --> H[Customer Home]
    end
    end
    F -.margin captured.-> Rev1[(Farmer Earnings)]
    L -.commission.-> Rev2[(NovaMart Revenue)]
```

Figure 48 — Architecture Diagram (Consolidated): NovaMart Full System & Site Map

```
%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#F3F8F1", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#1B3B2F", "lineColor": "#5A6B57", "secondaryColor": "#FFFFFF", "tertiaryColor": "#FFFFFF"}}
```

```

", "clusterBkg": "#FFFFFF", "clusterBorder": "#BFD1BC", "fontFamily": "Segoe UI, Helvetica, Arial,
sans-serif", "background": "#FFFFFF"} } } %
flowchart TB
    subgraph CUSTOMER["Customer Web App"]
        direction TB
        C7["Sign In / Sign Up /\nEmail & Password Verification"]
        C1["Home / Catalog\n(38 vegetables, 5 categories)"]
        C2["Search & Category Filters"]
        C3["Product Detail\n(pack size, quantity)"]
        C4["Cart\n(add / remove / qty)"]
        C5["Checkout\n(address + payment)"]
        C6["Order Tracking"]
        C7 -.account.-> C1
        C1 --> C2 --> C3 --> C4 --> C5 --> C6
    end

    subgraph SELLER["Seller Portal"]
        direction TB
        S1["Seller Dashboard\n(sales, KPIs)"]
        S2["Product Listing &\nStock Management"]
        S3["Incoming Orders &\nFulfilment"]
        S1 --> S2 --> S3
    end

    subgraph ADMIN["Admin Panel"]
        direction TB
        A1["User Management"]
        A2["Content Moderation"]
        A3["Reports & Analytics"]
        A1 --> A2 --> A3
    end

    GATEWAY(["API Gateway"])

    subgraph SERVICES["Application Services"]
        direction LR
        SV1["Auth\nService"]
        SV2["Catalog\nService"]
        SV3["Order\nService"]
        SV4["Payment\nService"]
        SV5["Notification\nService"]
    end

    subgraph DATA["Data Layer"]
        direction LR
        DB["Primary\nDatabase"]
        CACHE["Cache\n(Redis)"]
        STORE["Media\nStorage"]
    end

    CUSTOMER --> GATEWAY
    SELLER --> GATEWAY
    ADMIN --> GATEWAY
    GATEWAY --> SV1
    GATEWAY --> SV2
    GATEWAY --> SV3
    GATEWAY --> SV4
    GATEWAY --> SV5
    SV1 --> DB
    SV2 --> DB
    SV3 --> DB
    SV4 --> DB
    SV2 --> CACHE
    SV3 --> CACHE
    SV2 --> STORE

    classDef customer fill:#EAF7EC,stroke:#3F8F4F,stroke-width:1.4px,color:#1B3B2F

```

```

classDef seller fill:#FFF4D9,stroke:#C99A1F,stroke-width:1.4px,color:#3B2F0A
classDef admin fill:#EAF0FB,stroke:#3F6FA8,stroke-width:1.4px,color:#0F2A4A
classDef services fill:#F3F8F1,stroke:#5A6B57,stroke-width:1.2px,color:#1B3B2F
classDef data fill:#F6F0E6,stroke:#8B6234,stroke-width:1.2px,color:#3B2A12
classDef gateway fill:#1B3B2F,stroke:#1B3B2F,stroke-width:1px,color:#FFFFFF

class C1,C2,C3,C4,C5,C6,C7 customer
class S1,S2,S3 seller
class A1,A2,A3 admin
class SV1,SV2,SV3,SV4,SV5 services
class DB,CACHE,STORE data
class GATEWAY gateway

```

Figure 49 — Architecture Diagram (Consolidated): NovaMart Full Front-End & Back-End Architecture

```

%%{init:
{"theme": "base", "themeVariables": {"primaryColor": "#F3F8F1", "primaryBorderColor": "#3F8F4F", "primaryTextColor": "#1B3B2F", "lineColor": "#5A6B57", "secondaryColor": "#FFFFFF", "tertiaryColor": "#FFFFFF", "clusterBkg": "#FFFFFF", "clusterBorder": "#BFD1BC", "fontFamily": "Segoe UI, Helvetica, Arial, sans-serif", "background": "#FFFFFF"}}}%%
flowchart TB
    subgraph FRONTEND["FRONT-END – Client Application (HTML / CSS / JavaScript)"]
        direction TB
        subgraph FEVIEWS["UI Views"]
            direction LR
            V1["Home /\nCatalog Grid"]
            V2["Product\nDetail"]
            V3["Cart"]
            V4["Checkout"]
            V5["Order\nTracking"]
            V6["Seller\nDashboard"]
            V7["Auth Screens\n(Sign In / Up / Verify)"]
        end
        NAV["Navigation Controller\nshowView() + global Back button"]
        STATE["Application State\nproducts[], cart[], deliveryAddress, paymentMethod, session"]
        RENDERFN["Render Functions\nrenderGrid, renderCart, renderCheckout, renderAccountMenu"]
        HTTPCLIENT["HTTP Client\n(fetch / JSON)"]

        NAV --> FEVIEWS
        FEVIEWS --> STATE
        STATE --> RENDERFN
        RENDERFN --> FEVIEWS
        STATE --> HTTPCLIENT
    end

    HTTPCLIENT ==>|"HTTPS / JSON REST calls"| GATEWAY

    subgraph BACKEND["BACK-END – Server Application (Node.js / Express)"]
        direction TB
        GATEWAY(["API Gateway / Router"])
        MW["Authentication & Validation Middleware\n(JWT / session check)"]
        subgraph BESERVICES["Business Logic Services"]
            direction LR
            SVC_AUTH["Auth\nService"]
            SVC_CATALOG["Catalog\nService"]
            SVC_ORDER["Cart & Order\nService"]
            SVC_PAY["Payment\nService"]
            SVC_NOTIFY["Notification\nService"]
        end
        end
        DAL["ORM / Data Access Layer"]

        GATEWAY --> MW --> BESERVICES
        BESERVICES --> DAL
    end

    subgraph DATA["DATA & EXTERNAL INTEGRATIONS"]
        direction LR
    end

```

```

    DB[("Relational\nDatabase")]
    CACHE[("Cache\n(Redis)")]
    PAYGW[["Payment Gateway\n(UPI / Card / Net Banking)"]]
    SMS[["SMS / Email\nProvider"]]
end

DAL --> DB
DAL --> CACHE
SVC_PAY -->|"charge / verify"| PAYGW
SVC_NOTIFY -->|"send OTP / alerts"| SMS

classDef fe fill:#EAF7EC,stroke:#3F8F4F,stroke-width:1.4px,color:#1B3B2F
classDef fecore fill:#DCEFE0,stroke:#2F7A3F,stroke-width:1.4px,color:#1B3B2F
classDef be fill:#EAF0FB,stroke:#3F6FA8,stroke-width:1.4px,color:#0F2A4A
classDef gw fill:#1B3B2F,stroke:#1B3B2F,color:#FFFFFF
classDef data fill:#F6F0E6,stroke:#8B6234,stroke-width:1.2px,color:#3B2A12
classDef ext fill:#FFF4D9,stroke:#C99A1F,stroke-width:1.2px,color:#3B2F0A

class V1,V2,V3,V4,V5,V6,V7 fe
class NAV,STATE,RENDERFN,HTTPCLIENT fecore
class MW,SVC_AUTH,SVC_CATALOG,SVC_ORDER,SVC_PAY,SVC_NOTIFY,DAL be
class GATEWAY gw
class DB,CACHE data
class PAYGW,SMS ext

```

Figures 38, 39, 40, 42, 43 — Matplotlib Source (custom_diagrams.py)

The five diagrams without native Mermaid syntax — Tree Map (Figure 38), Venn Diagram (Figure 39), Ishikawa/Fishbone Diagram (Figure 40), Wardley Map (Figure 42), and Cynefin Framework (Figure 43) — are all generated by this single Python script using Matplotlib, styled to match the Mermaid diagrams' colour system. Running it produces all five PNG files in one pass.

```

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.patches import FancyArrowPatch, Circle, Rectangle
import numpy as np

OUT = "/home/claude/diagrams/png"

GREEN = "#3F8F4F"
DGREEN = "#1B3B2F"
TEAL = "#1E9E6B"
CORAL = "#FF6F4A"
WARN = "#E8A400"
BG = "#F6F9F4"
BORDER = "#E3EAE0"
TEXT = "#20281F"
MUTED = "#758573"

plt.rcParams["font.family"] = "DejaVu Sans"

# -----
# 38. Tree map — NovaMart Product Category Distribution
# -----
def treemap():
    # simple squarified-ish treemap done manually (5 categories, sized by product count)
    data = [
        ("Root Vegetables", 10, "#F5A045"),
        ("Exotic & Others", 11, "#5FC97B"),
        ("Gourds & Squashes", 7, "#A9D98C"),
        ("Leafy Greens", 6, "#63D17E"),
        ("Herbs & Aromatics", 4, "#8FD059"),
    ]

```

```

total = sum(d[1] for d in data)
fig, ax = plt.subplots(figsize=(9, 5.5))
x, y, w, h = 0, 0, 100, 60
# simple slice layout: split widest first (left to right), largest gets biggest strip
cursor = 0
data_sorted = sorted(data, key=lambda d: -d[1])
labels_multiline = {
    "Herbs & Aromatics": "Herbs & \nAromatics",
    "Gourds & Squashes": "Gourds & \nSquashes",
}
for i, (name, count, color) in enumerate(data_sorted):
    frac = count / total
    seg_w = w * frac
    rect = Rectangle((cursor, y), seg_w, h, facecolor=color, edgecolor="white", linewidth=3)
    ax.add_patch(rect)
    pct = count / total * 100
    fsize = 10.5 if seg_w > 16 else 8.2
    disp_name = labels_multiline.get(name, name)
    ax.text(cursor + seg_w/2, h/2, f"{disp_name}\n{count} items\n({pct:.0f}%)",
            ha="center", va="center", fontsize=fsize, color="#20281f", fontweight="bold")
    cursor += seg_w
ax.set_xlim(0, w); ax.set_ylim(0, h)
ax.axis("off")
ax.set_title("NovaMart Product Category Distribution", fontsize=15, fontweight="bold",
color=DGREEN, pad=14)
plt.tight_layout()
plt.savefig(f"{OUT}/38.png", dpi=200, facecolor="white")
plt.close()

# -----
# 39. Venn Diagram – Customer-Seller-Admin Relationship
# -----
def venn():
    fig, ax = plt.subplots(figsize=(8, 7))
    circles = [
        (0.38, 0.62, "Customer", "#63D17E"),
        (0.62, 0.62, "Seller", "#F5A045"),
        (0.50, 0.35, "Admin", "#7EA8E0"),
    ]
    for cx, cy, label, color in circles:
        c = Circle((cx, cy), 0.30, alpha=0.45, facecolor=color, edgecolor=DGREEN, linewidth=2)
        ax.add_patch(c)
    ax.text(0.24, 0.80, "Customer", fontsize=13, fontweight="bold", ha="center")
    ax.text(0.76, 0.80, "Seller", fontsize=13, fontweight="bold", ha="center")
    ax.text(0.50, 0.10, "Admin", fontsize=13, fontweight="bold", ha="center")

    ax.text(0.30, 0.65, "Browses,\norders,\nreviews", fontsize=9, ha="center")
    ax.text(0.70, 0.65, "Lists produce,\nfulfills orders", fontsize=9, ha="center")
    ax.text(0.50, 0.22, "Moderates\naccounts", fontsize=9, ha="center")
    ax.text(0.50, 0.62, "Order & \nPayment\nData", fontsize=9, ha="center", fontweight="bold")
    ax.text(0.38, 0.45, "Dispute\nhandling", fontsize=8.5, ha="center")
    ax.text(0.62, 0.45, "Listing\napproval", fontsize=8.5, ha="center")

    ax.set_xlim(0, 1); ax.set_ylim(0, 1)
    ax.axis("off")
    ax.set_title("NovaMart Customer\u2013Seller\u2013Admin Relationship", fontsize=15,
fontweight="bold", color=DGREEN, pad=14)
    plt.tight_layout()
    plt.savefig(f"{OUT}/39.png", dpi=200, facecolor="white")
    plt.close()

# -----
# 40. Ishikawa (Fishbone) – Order Failure Root Cause Analysis
# -----
def fishbone():
    fig, ax = plt.subplots(figsize=(12, 7))
    ax.set_xlim(0, 12); ax.set_ylim(0, 7)
    ax.axis("off")

```

```

# spine
ax.annotate("", xy=(10.6, 3.5), xytext=(1.0, 3.5),
            arrowprops=dict(arrowstyle="->", lw=2.4, color=DGREEN))
ax.text(11.0, 3.5, "Order\nFailure", fontsize=13, fontweight="bold", va="center",
        color=DGREEN,
        bbox=dict(boxstyle="round,pad=0.4", fc="#FFECE6", ec=CORAL, lw=1.5))

categories = [
    ("Payment Issues", ["UPI timeout", "Card declined", "Gateway downtime"], 8.6, 5.6, 1),
    ("Inventory Issues", ["Stock mismatch", "Delayed restock", "Wrong quantity listed"],
5.6, 5.6, 1),
    ("Logistics Issues", ["Rider unavailable", "Wrong address", "Delivery delay"], 8.6, 1.4,
-1),
    ("System Issues", ["Server timeout", "App crash", "Session expiry"], 5.6, 1.4, -1),
]
for label, causes, bx, by, direction in categories:
    ax.annotate("", xy=(bx, 3.5), xytext=(bx, by),
                arrowprops=dict(arrowstyle="-", lw=1.8, color=GREEN))
    ax.text(bx, by + 0.25*direction, label, fontsize=11.5, fontweight="bold",
            ha="center", color=DGREEN)
    n = len(causes)
    for i, cause in enumerate(causes):
        # stop short of the spine (0.18..0.82 of the stem) so labels never meet the other
branch
        frac = 0.18 + (i/(n-1 if n>1 else 1)) * 0.64
        cy = by + (3.5-by)*frac
        side = "right" if bx > 6 else "left"
        offset = 0.12 if side == "right" else -0.12
        ax.text(bx + offset, cy, cause, fontsize=9, ha=side, va="center", color=TEXT)

ax.set_title("NovaMart Order Failure Root Cause Analysis (Ishikawa / Fishbone Diagram)",
            fontsize=13.5, fontweight="bold", color=DGREEN, pad=10)
plt.tight_layout()
plt.savefig(f"{OUT}/40.png", dpi=200, facecolor="white")
plt.close()

# -----
# 42. Wardley Map – Business Value Chain Map
# -----
def wardley():
    fig, ax = plt.subplots(figsize=(10, 7))
    ax.set_xlim(0, 10); ax.set_ylim(0, 10)

    # axes
    ax.annotate("", xy=(9.6, 0), xytext=(0, 0), arrowprops=dict(arrowstyle="->", lw=1.8,
color=TEXT))
    ax.annotate("", xy=(0, 9.6), xytext=(0, 0), arrowprops=dict(arrowstyle="->", lw=1.8,
color=TEXT))
    ax.text(9.7, -0.35, "Evolution: Genesis \u2192 Custom \u2192 Product \u2192 Commodity",
        fontsize=10, ha="right")
    ax.text(-0.4, 9.7, "Value\nChain", fontsize=10, ha="left", va="top", rotation=90)

    for x in [2.5, 5.0, 7.5]:
        ax.plot([x, x], [0, 9.5], color=BORDER, lw=1, linestyle="--")
    labels_x = ["Genesis", "Custom-Built", "Product / Rental", "Commodity"]
    for i, lab in enumerate(labels_x):
        ax.text(1.25 + i*2.5, -0.9, lab, fontsize=9.5, ha="center", color=MUTED)

    nodes = [
        ("Customer Need:\nFresh Vegetables", 8.8, 8.8, DGREEN),
        ("NovaMart Storefront (UX)", 6.2, 7.4, GREEN),
        ("Order & Cart Engine", 5.6, 6.2, GREEN),
        ("Payment Processing", 7.4, 5.2, TEAL),
        ("Delivery / Logistics", 6.0, 4.0, "#F5A045"),
        ("Vegetable Sourcing (Farms)", 3.4, 3.0, "#7EA8E0"),
        ("Cloud Hosting", 8.6, 2.2, MUTED),

```

```

        ("Payment Gateway (3rd party)", 8.8, 1.2, MUTED),
    ]
    for label, x, y, color in nodes:
        ax.plot(x, y, "o", markersize=11, color=color, markeredgewidth=1.2,
        markedgewidth=1.2)
        ax.text(x-0.15, y+0.28, label, fontsize=9, ha="right" if x > 5 else "left", color=TEXT)

    edges = [(8.8,8.8, 6.2,7.4), (6.2,7.4, 5.6,6.2), (5.6,6.2, 7.4,5.2), (5.6,6.2, 6.0,4.0),
              (6.0,4.0, 3.4,3.0), (7.4,5.2, 8.8,1.2), (6.0,4.0, 8.6,2.2)]
    for x1,y1,x2,y2 in edges:
        ax.plot([x1,x2],[y1,y2], color=BORDER, lw=1.3, zorder=0)

    ax.axis("off")
    ax.set_title("NovaMart Business Value Chain Map (Wardley Map)", fontsize=14,
    fontweight="bold", color=DGREEN, pad=16)
    plt.tight_layout()
    plt.savefig(f"{OUT}/42.png", dpi=200, facecolor="white")
    plt.close()

# -----
# 43. Cynefin Framework – Decision-Making Framework
# -----
def cynefin():
    fig, ax = plt.subplots(figsize=(9, 8))
    ax.set_xlim(0, 10); ax.set_ylim(0, 10)
    quads = [
        (0, 5, 5, 5, "#FFCE6", "Complex", "Best guess & experiment:\nnew feature
discovery,\nseasonal demand shifts"),
        (5, 5, 5, 5, "#E9F6EC", "Complicated", "Analyse & plan:\nscaling infra, payment\ngateway
integration"),
        (0, 0, 5, 5, "#FFF4D9", "Chaotic", "Act fast, stabilise:\nsite outage,\npayment fraud
spike"),
        (5, 0, 5, 5, "#E7F1FB", "Clear (Obvious)", "Best practice:\norder status
updates,\nrouting deliveries"),
    ]
    for x, y, w, h, color, title, desc in quads:
        ax.add_patch(Rectangle((x, y), w, h, facecolor=color, edgecolor="white", linewidth=3))
        ax.text(x + w/2, y + h - 0.9, title, fontsize=13, fontweight="bold", ha="center",
        color=DGREEN)
        ax.text(x + w/2, y + h/2 - 0.6, desc, fontsize=9.5, ha="center", va="center",
        color=TEXT)
    ax.plot([5,5],[0,10], color="white", lw=4)
    ax.plot([0,10],[5,5], color="white", lw=4)
    ax.text(5, 9.85, "NovaMart Decision-Making Framework (Cynefin)", fontsize=13.5,
        fontweight="bold", ha="center", color=DGREEN)
    ax.axis("off")
    plt.tight_layout()
    plt.savefig(f"{OUT}/43.png", dpi=200, facecolor="white")
    plt.close()

treemap()
venn()
fishbone()
wardley()
cynefin()
print("custom diagrams done")

```

THIS CODES ARE CAPY PAST IN THIS LINK ONLY THIS ONLY [Figure 01 TO Figure 49] Mermaid Language:
https://mermaid.live/edit#pako:eNqNU02P2jAQ_SuWD1VXAsoSnpNoBQksu3zsqmwrdes3GRIrCZ2ZBxYivjvHZtFG-XUnDJ-b968GY9PNFIxUJ9uM3WIUq4NeQ6ZZpLgt_n4k9GNwUNGf92QZvMzGePJRKvDDshc5UCeeALkEwm4gURpATtkXtPHLmNiNYDrKCVKk8cCJHnSKi4jU6FOHDVA6jiOrzgxCoV1lRc4XnhINFDSCFkC2aSqKIRMvjB6vtJCpBFGf1g7F9s1ZK3egCmWdJ5qhaYOnSGKZiKA2JlJIfqjyipv5nh3yPsKewEHp0M-kGd1eFYdyp0jztH5hu9RDvvUsNsxJsd7LjL-04NqB_N6B_dujBngW0oCONZveB9BqTVIQ5Yq4kYoWsk-r3X9gGIzkWVkjY7fhMhM6fy_106dxgI1glSpnV2CY265KzCpiivMhwvzGi5cuMQZXDNCMNi8beI7z0RcncCy5n1RA95Hs7K31PEIyK00QVfgrxy-tndsEYJLsxU6vzTEZCh2RcaPULW8dikvdvOn0gI3FqENmmgRU9_oEho0BxSxIT3ZLEZNCjkw6uNvDFteZrgjTJ4xreDyRan8mq1VmatU3-JyYFQWMT6cUPBE83cKSPQaFIa6vcaFGJh1F5dnq17rU6V-if6Sv1RuzUY9DudntcdDL2-123QI_WbXqfdwqA_anvt3qA7Gpwb9K_zcdvqjW777dHQ6_U7S0gOz_8AfYk3Yw